

COMPUTER ORGANISATION

Topic 4

Digital Logic (Part 1)

Learning Objectives

Understand the basic operations of Boolean algebra.

Distinguish among the different types of flip-flops.

Use a Karnaugh map to simplify a Boolean expression.

Present an overview of programmable logic devices.

Outline

- ▶ Boolean Algebra
- ▶ Gates
- ▶ Combinational Circuits
 - ▶ Implementation of Boolean Functions
 - ▶ Multiplexers
 - ▶ Decoders
 - ▶ Read-Only Memory
 - ▶ Adders

Introduction

- ▶ The operation of the digital computer is based on the storage and processing of **binary data**.
- ▶ How these are implemented using **combinational** & **sequential** circuit
- ▶ Combinational and sequential circuits, which are constructed from **gates**

Introduction

► Boolean Algebra

- Introduced in 1854 by George Boole, only used in circuit design in 1938
- Mathematical foundation of digital circuits
- A tool to:
 - Analysis: It is an economical way of describing the function of **digital circuitry**.
 - Design: Given a desired function, Boolean algebra can be applied to develop a **simplified** implementation of that function

Boolean Algebra

- ▶ Use logical variables and operations

▶ **Variable** takes the value of **1 (TRUE)** or **0 (FALSE)**

1	0
High voltage	Low voltage
On	Off
True	False

Boolean Algebra

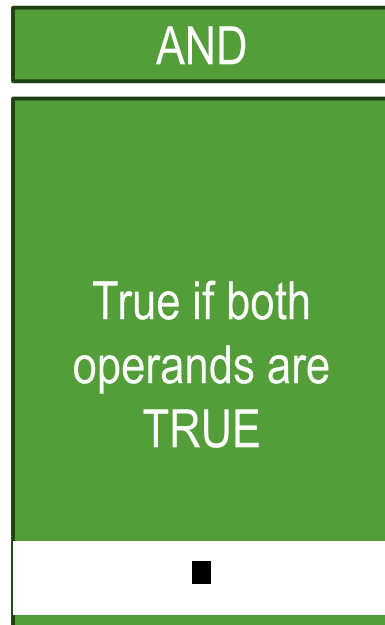
- ▶ Basic logical **operations** – **AND**, **OR**, and **NOT**
 - ▶ represented as *dot*, *plus sign* and *overbar*

$$A \text{ AND } B = A \cdot B$$

$$A \text{ OR } B = A + B$$

$$\text{NOT } A = \overline{A}$$

Basic Operations



$$D = A + (\bar{B} \cdot C)$$

D = 1 if A is 1 or if both B = 0 and C = 1
Otherwise, D = 0.

Other Operations

XOR

True if and only if
exactly one
operand has
value 1

NAND

Complement
(NOT) of the
AND function

NOR

Complement
(NOT) of the OR
function

$$A \text{ NAND } B = \text{NOT } (A \text{ AND } B) = \overline{AB}$$

$$A \text{ NOR } B = \text{NOT } (A \text{ OR } B) = \overline{A + B}$$

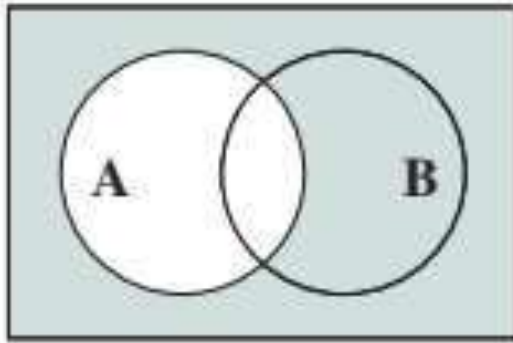
Truth Table

- The resulting value of the operations for all possible values of the operands

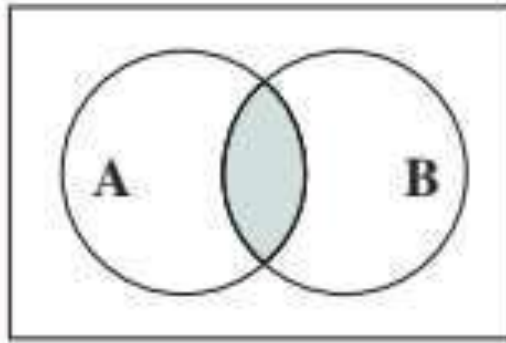
A	B	NOT A ($\overline{A}$)	A AND B ($A \cdot B$)	A OR B ($A + B$)	A NAND B ($\overline{A \cdot B}$)	A NOR B ($\overline{A + B}$)	A XOR B ($A \oplus B$)
0	0	1	0	0	1	1	0
0	1	1	0	1	1	0	1
1	0	0	0	1	1	0	1
1	1	0	1	1	0	0	0

Operation	Expression	Output = 1 if
AND	$A \cdot B \cdot \dots$	All of the set $\{A, B, \dots\}$ are 1.
OR	$A + B + \dots$	Any of the set $\{A, B, \dots\}$ are 1.
NAND	$\overline{A \cdot B \cdot \dots}$	Any of the set $\{A, B, \dots\}$ are 0.
NOR	$\overline{A + B + \dots}$	All of the set $\{A, B, \dots\}$ are 0.
XOR	$A \oplus B \oplus \dots$	The set $\{A, B, \dots\}$ contains an odd number of ones.

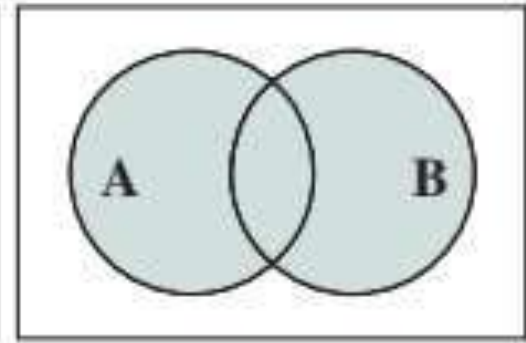
Boolean variables in circles



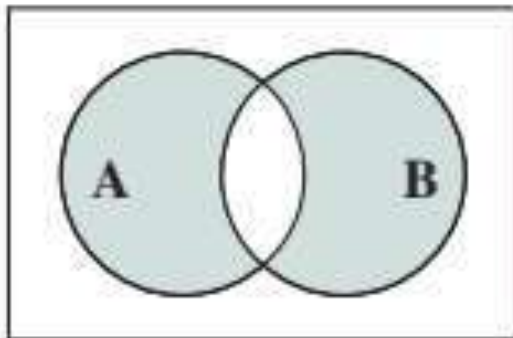
NOT A



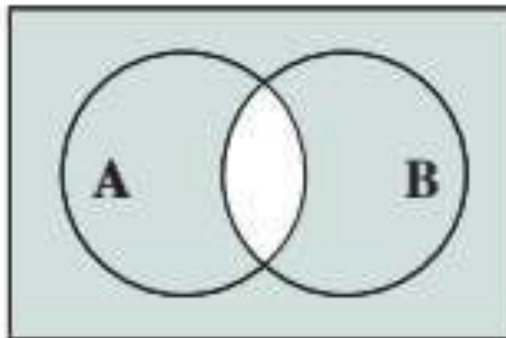
A AND B



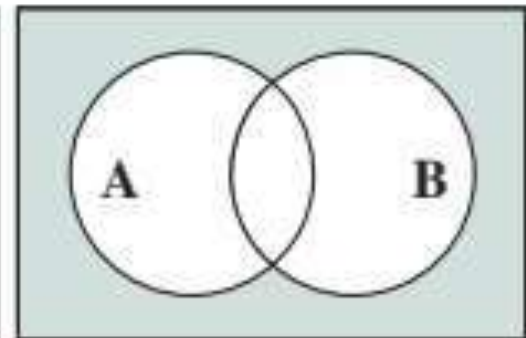
A OR B



A XOR B



A NAND B



A NOR B

Basic Identities

	AND	OR
Commutative Laws	$A.B = B.A$	$A + B = B + A$
Idempotent Laws	$AA = A$	$A+A = A$
Distributive Laws	$A.(B + C) = (A.B) + (A.C)$	$A(B.C) = (A + B) . (A + C)$
Identity Laws	$A.1 = A$	$A + 0 = A$
Associative Laws	$A.(B.C) = (A.B).C$	$A + (B + C) = (A + B) + C$
Inverse Elements	$A.\bar{A} = 0$	$A + \bar{A} = 1$
DeMorgan's Theorems	$\overline{A.B} = \bar{A} + \bar{B}$	$\overline{A + B} = \bar{A}.\bar{B}$

Basic Identities (Examples)

	AND	OR
Commutative Laws	$A.B = B.A$	$A + B = B + A$
	$1 \times 0 = 0 \times 1 = 0$	$1 + 0 = 0 + 1 = 1$
Idempotent Laws	$AA = A$	$A+A = A$
	$1.1 = 1$	$1 + 1 = 1$
Distributive Laws	$A.(B + C) = (A.B) + (A.C)$	$A(B.C) = (A + B) . (A + C)$
	$1.(0+1) = (1.0) + (1.1)$ $1(1) = 0 + 1$ $1 = 1$	$1(1.1) = (1+1) + (1+1)$ $1 = 1 + 1$ $1 = 1$
Identity Laws	$A.1 = A$	$A + 0 = A$
	$1.1 = 1$	$1+1 = 1$

Basic Identities (Examples)

AND

OR

Associative Laws

$$A.(B.C) = (A.B).C$$

$$A + (B + C) = (A + B) + C$$

$$1 \times (0 \times 1) = (1 \times 0) \times 1 = 0$$

$$1 + (0 + 1) = (1 + 0) + 1 = 1$$

DeMorgan's Theorems

$$\overline{A.B} = \overline{A} + \overline{B}$$

$$\overline{A + B} = \overline{A}.\overline{B}$$

$$(1 \times 0)' = 1' + 0' = 0 + 1 = 1$$

$$(1 + 0)' = 1' \times 0' = 0 \times 1 = 0$$

Simplifying expressions #1

$$F = A + \overline{BA}$$

$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

DeMorgan's Theorem

$$\overline{BA} = \overline{A} + \overline{B}$$

$$F = A + \overline{BA} = A + \overline{A} + \overline{B}$$

$$A + \overline{A} = 1$$

Inverse Elements

$$A + \overline{A} = 1$$

$$F = 1 + \overline{B}$$

$$0 + A = A$$

Identity Elements

$$1 + \overline{B} = 1$$

$$F = 1$$

Simplifying expressions #2

	AND	OR
Commutative Laws	$A.B = B.A$	$A + B = B + A$

Simplify $(X + Y) (Y + Z) (Y + X)$

$$(Y + X) (Y + Z) (Y + X)$$

1. Re-arrange the expression

$$(Y + X) (Y + Z) (Y + X)$$

2. Eliminate the repetitive expressions

$$(Y + X) (Y + Z)$$

3. Simplify the expressions

Simplifying expressions #3

	AND	OR
Commutative Laws	$A.B = B.A$	$A + B = B + A$

Simplify the following expressions according to the commutative law:

$$(\bar{A} + D) (D + \bar{A}) (B + E + \bar{A})(\bar{A} + E + B)(A + B + C)$$

$$(\bar{A} + D) (D + \bar{A}) (B + E + \bar{A})(\bar{A} + E + B)(A + B + C)$$

$$(\bar{A} + D) (B + E + \bar{A})(A + B + C)$$

Simplifying expressions #4

	AND	OR
Commutative Laws	$A.B = B.A$	$A + B = B + A$

Simplify the following expressions according to the commutative law:

$$(A + B + C)(D + E + F)(X + Y)(C + A + B)$$

$$(A + B + C)(D + E + F)(X + Y)(C + A + B) = (A + B + C)(D + E + F)(X + Y)(A + B + C)$$

$$(A + B + C)(D + E + F)(X + Y) \cancel{(A + B + C)}$$

$$(A + B + C)(D + E + F)(X + Y)$$

Simplifying expressions #4

	AND	OR
Commutative Laws	$A.B = B.A$	$A + B = B + A$

Simplify the following expressions according to the commutative law:

$$(ABC) + (A'B'C') + (B'C') + A(B+C) + (CBA) + (C'B')$$

$$(ABC) + (A'B'C') + (B'C') + A(B+C) + \cancel{(ABC)} + (C'B')$$

$$(ABC) + (A'B'C') + (B'C') + A(B+C) + (C'B')$$

$$(ABC) + (A'B'C') + (B'C') + A(B+C) + \cancel{(C'B')}$$

$$(ABC) + (A'B'C') + (B'C') + A(B+C)$$

Simplifying expressions #5

Simplify $A = \overline{MNOP}$ using DeMorgan's Theorem

$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

DeMorgan's Theorem

$$A = \overline{M} + \overline{N} + \overline{O} + \overline{P}$$

Simplifying expressions #6

$$A'B'C' + AB'C' + ABC' + A'B'C + AB'C + ABC$$

$$B'C'(A'+A) + AB(C'+C) + A'B'C + AB'C$$

$$B'C'(A'+A) + AB(C'+C) + B'C(A'+A)$$

$$B'C'(1) + AB(1) + B'C(1)$$

$$B'C' + AB + B'C$$

$$B'(C'+C) + AB$$

$$B'(1) + AB$$

$$B' + AB$$

Simplifying expressions #6

$$A'BC + AB'C' + A'B'C' + AB'C + A'BC'$$

$$\boxed{A'BC} + \boxed{AB'C'} + \boxed{A'B'C'} + \boxed{AB'C} + \boxed{A'BC'}$$

$$\boxed{A'B(C+C')} + \boxed{AB'(C'+C)} + \boxed{B'C'(A'+A)}$$

$$A'B(1) + AB'(1) + B'C'(1)$$

$$A'B + AB' + B'C'$$

Quick Activity #1

- ✓ $A'BC + AB'C' + A'B'C' + AB'C + A'BC'$
- ✓ $A'B'C' + A'B'C + A'BC + A'BC' + ABC + AB'C + AB'C' + A'BC$
- ✓ $A + B'A'$
- ✓ $A'B'C' + A'B'C' + ABC' + A'B'C + AB'C + ABC$

Quick Activity #1

- $A'BC + AB'C' + A'B'C' + AB'C + A'BC'$

$$\boxed{A'BC} + \boxed{AB'C'} + \boxed{A'B'C'} + \boxed{AB'C} + \boxed{A'BC'}$$

$$A'B(C+C') + B'C'(A+A') + AB'(C'+C)$$

$$A'B + B'C' + AB'$$

Quick Activity #1

- $A'B'C' + A'B'C + A'BC + A'BC' + ABC + AB'C + AB'C' + A'BC$
 $A'B'C' + A'B'C + A'BC + A'BC' + ABC + AB'C + AB'C'$
 $A'B'(C'+C) + B'C'(A'+A) + A'BC + A'BC' + ABC + AB'C$
 $A'B'(C'+C) + B'C'(A'+A) + A'B(C+C') + AC(B+B')$
 $A'B' + B'C' + A'B + AC$

Outline

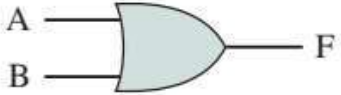
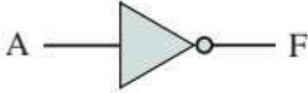
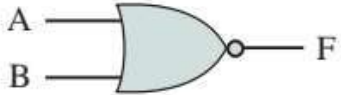
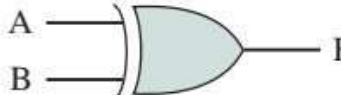
- ▶ Boolean Algebra
- ▶ Gates
- ▶ Combinational Circuits
 - ▶ Implementation of Boolean Functions
 - ▶ Multiplexers
 - ▶ Decoders
 - ▶ Read-Only Memory
 - ▶ Adders

Gates

- ▶ The fundamental building block of all digital logic circuits is the gate.
- ▶ Logical functions are implemented by the **interconnection** of gates.
- ▶ A gate is an electronic circuit that produces an **output signal** that is a simple **Boolean operation** on its input signals
- ▶ The basic gates used in digital logic are **AND**, **OR**, **NOT**, **NAND**, **NOR**, and **XOR**
- ▶ One or two inputs and **one output**

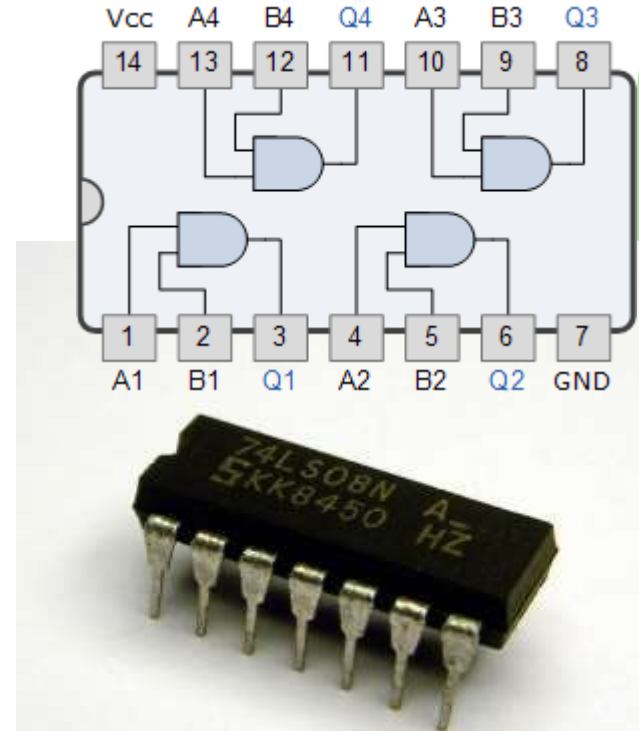
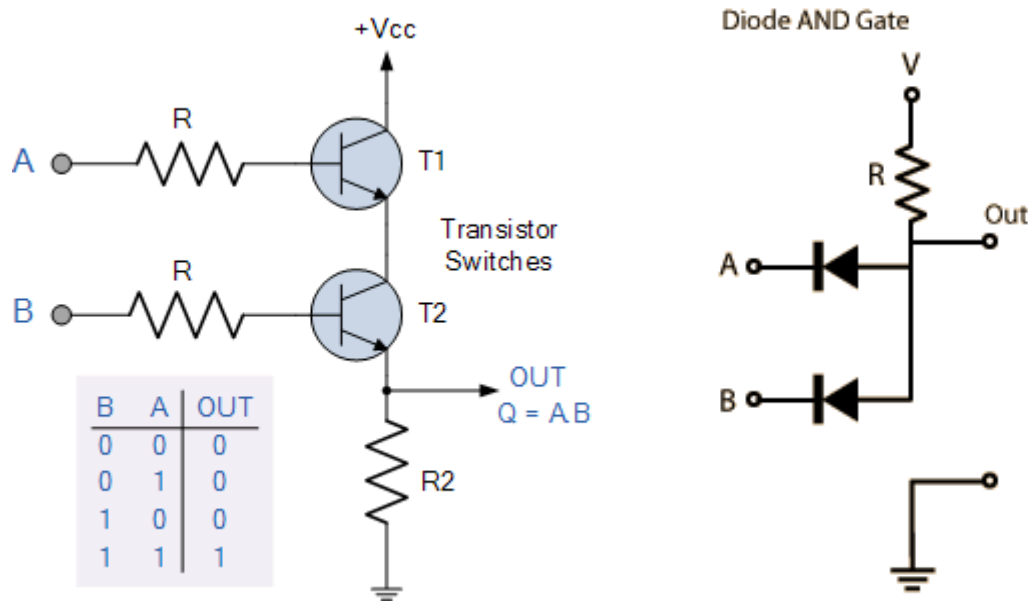
Gates

- Graphic symbol, algebraic notation, and truth table

Name	Graphical Symbol	Algebraic Function	Truth Table															
AND		$F = A \cdot B$ or $F = AB$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	F	0	0	0	0	1	0	1	0	0	1	1	1
A	B	F																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
OR		$F = A + B$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	F	0	0	0	0	1	1	1	0	1	1	1	1
A	B	F																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
NOT		$F = \bar{A}$ or $F = A'$	<table><tr><th>A</th><th>F</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	F	0	1	1	0									
A	F																	
0	1																	
1	0																	
NAND		$F = \overline{AB}$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	F	0	0	1	0	1	1	1	0	1	1	1	0
A	B	F																
0	0	1																
0	1	1																
1	0	1																
1	1	0																
NOR		$F = \overline{A + B}$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	F	0	0	1	0	1	0	1	0	0	1	1	0
A	B	F																
0	0	1																
0	1	0																
1	0	0																
1	1	0																
XOR		$F = A \oplus B$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	F	0	0	0	0	1	1	1	0	1	1	1	0
A	B	F																
0	0	0																
0	1	1																
1	0	1																
1	1	0																

Gates

- Example of AND gate circuit and packaging



http://www.electronics-tutorials.ws/logic/logic_2.html

<http://hyperphysics.phy-astr.gsu.edu/hbase/electronic/diodgate.html>

<http://robokart.com/ic-s/7408-quad-and-gate.html>

Functionally Complete Set

- ▶ **Not all gate types used** in implementation
- ▶ Design and fabrication are **simpler** if only **one or two types of gates** are used
- ▶ Any Boolean function can be implemented using only the gates in the **set**
- ▶ The following are functionally complete sets

AND, OR, NOT

AND, NOT

OR, NOT

NAND

NOR

Functionally Complete Set

AND, OR, NOT

AND, NOT

OR, NOT

NAND

NOR

For AND and NOT we can synthesize OR
using the DeMorgan's theorem

$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

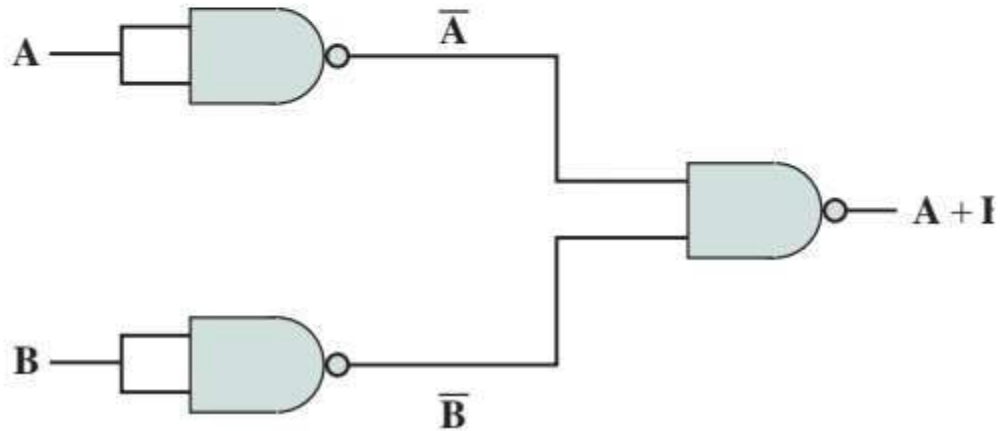
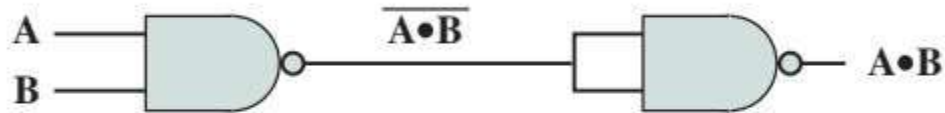
DeMorgan's Theorem

Digital circuits frequently implemented using
either NAND or NOR gates only

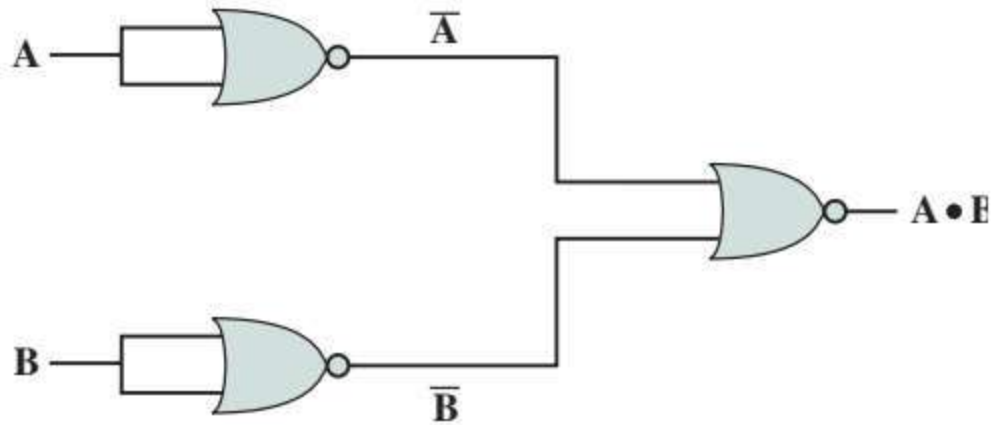
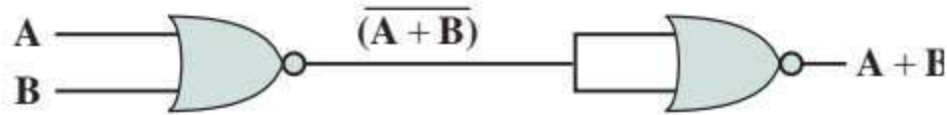
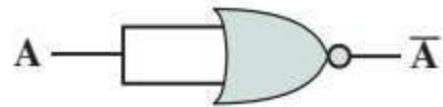
$$A + B = \overline{\overline{A} \cdot \overline{B}}$$

$$A \text{ OR } B = \text{NOT } ((\text{NOT } A) \text{ AND } (\text{NOT } B))$$

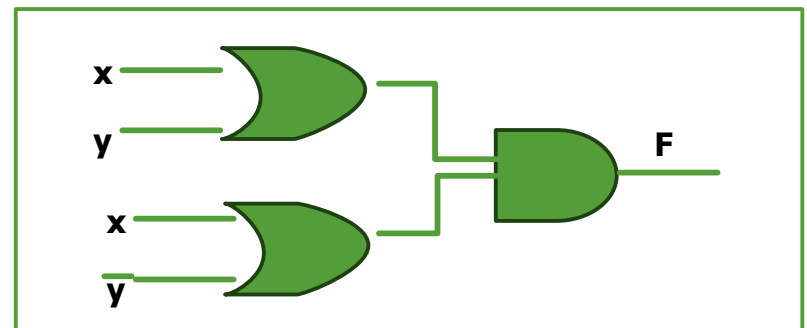
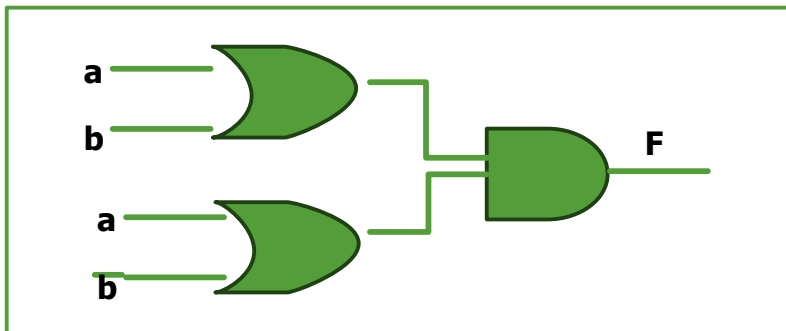
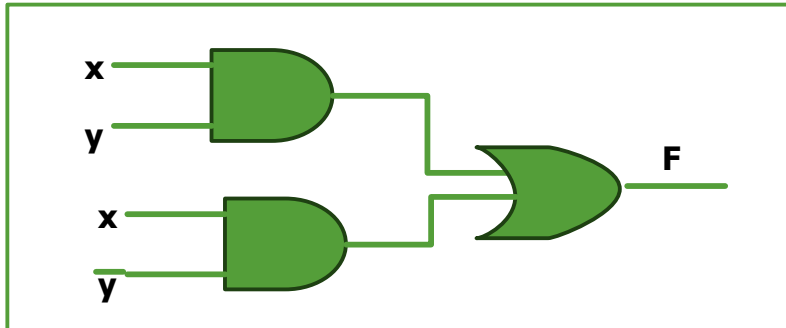
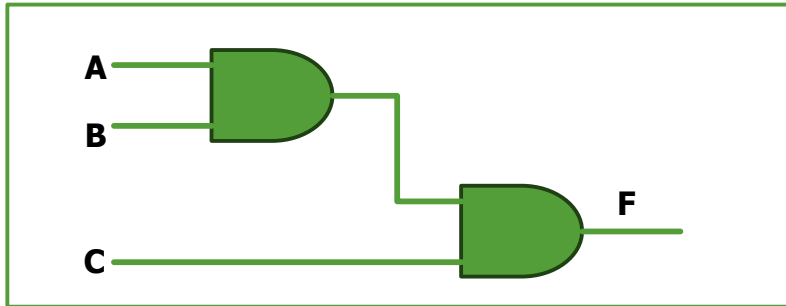
Implementation Using NAND



Implementation Using NOR



Write a function using the block diagram



Draw a block diagram based on the given function

- $F = AB + C$
- $F = \bar{X}Y + X'Y + Y'Z$
- $F = (A+B')(A+B+C)(B'+C')$

—

Outline

- ▶ Boolean Algebra
- ▶ Gates
- ▶ Combinational Circuits
 - ▶ Implementation of Boolean Functions
 - ▶ Multiplexers
 - ▶ Decoders
 - ▶ Read-Only Memory
 - ▶ Adders

Combinational Circuit

- ▶ A combinational circuit is an interconnected set of gates whose output at any time is a function only of the input at that time.
- ▶ As with a single gate, the appearance of the input is followed almost immediately by the appearance of the output, with only gate delays.
- ▶ In general terms, a combinational circuit consists of **n binary inputs** and **m binary outputs**. As with a gate, a combinational circuit can be defined in three ways:

Truth table

For each of the 2^n possible combination of input signals, the binary value of each of the m output signals is listed

Graphical symbols

The interconnected layout of gates

Boolean equations

Each output signal is expressed as Boolean function of its input signals

Implementation of Boolean Functions

- ▶ Typically, there are several alternatives to express a function:

▶ Sum of Product (SOP) $(.) + (.) + (.) \dots$

- ▶ The output is 1 if ANY of the input combinations result in a 1 is true (1 term for each 1)
- ▶ Any combination of variables produce result 1

▶ Product of Sum (POS) $(+).(+).(+) \dots$

- ▶ The output is 1 if NONE of the input combinations that produces 0 is true (1 term for each 0)
- ▶ Any combination of variables produce result 0

Example of SOP

$$\overline{(X \cdot Y \cdot Z)} = \bar{X} + \bar{Y} + \bar{Z}$$

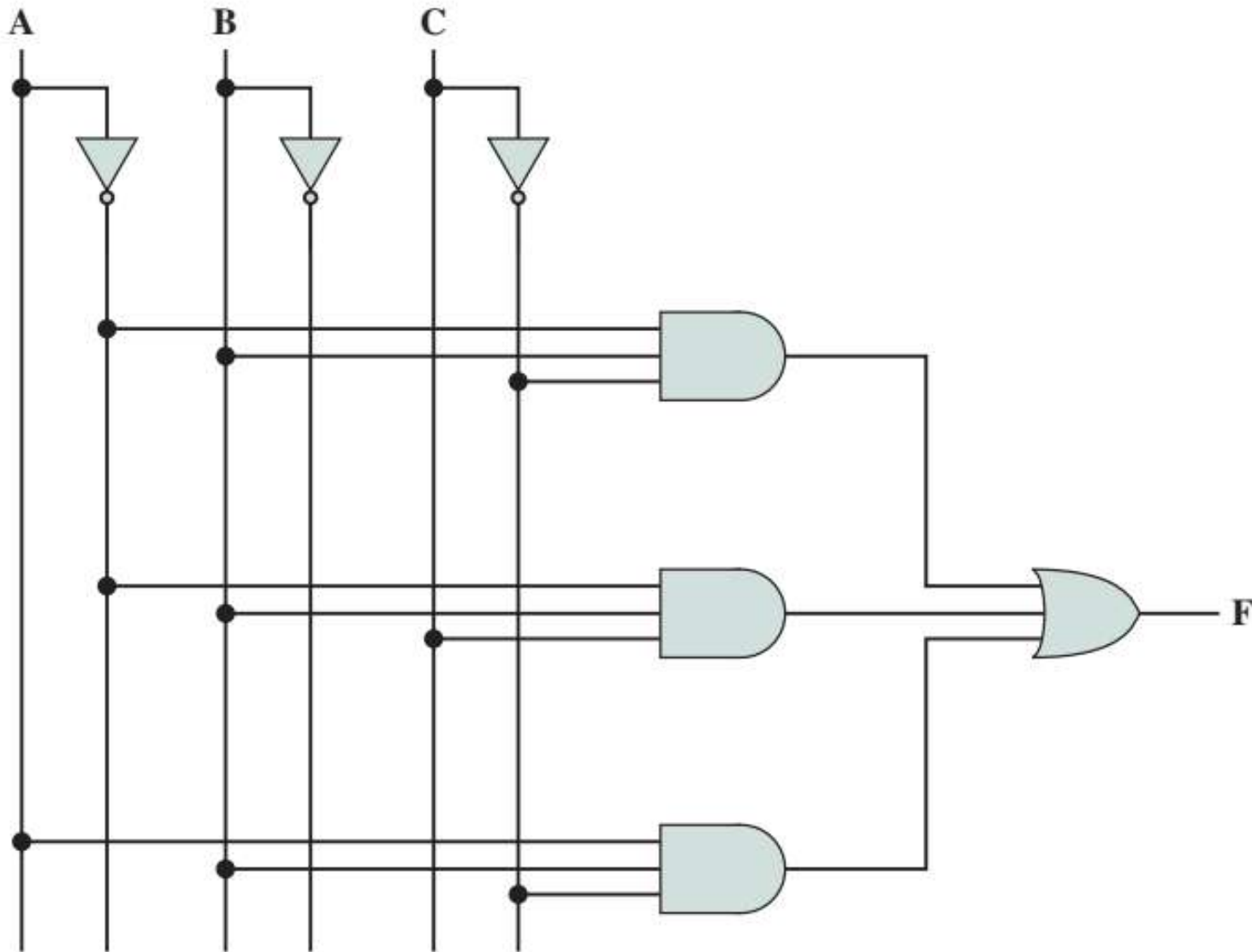
$$F = \overline{\overline{\overline{A}}\overline{\overline{B}}\overline{\overline{C}}} \cdot \overline{\overline{\overline{A}}\overline{\overline{B}}\overline{C}} \cdot \overline{\overline{\overline{A}}\overline{B}\overline{\overline{C}}} \cdot \overline{\overline{\overline{A}}\overline{B}C} \cdot \overline{\overline{A}\overline{\overline{B}}\overline{\overline{C}}}$$

Itemising combinations of values of A, B, and C that cause F to be 1

Implementation of Boolean Functions

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

Sum of Product (SOP)



Example of POS

$$F = (\overline{\overline{A}} + \overline{\overline{B}} + \overline{\overline{C}}) \cdot (\overline{\overline{A}} + \overline{\overline{B}} + \overline{\overline{C}}) \cdot (\overline{\overline{A}} + \overline{\overline{B}} + \overline{\overline{C}}) \cdot (\overline{\overline{A}} + \overline{\overline{B}} + \overline{\overline{C}}) \cdot (\overline{\overline{A}} + \overline{\overline{B}} + \overline{\overline{C}})$$

$$= (A + B + C) \cdot (A + B + \overline{C}) \cdot (\overline{A} + B + C) \cdot (\overline{A} + B + \overline{C}) \cdot (\overline{A} + \overline{B} + \overline{C})$$

$$F = \overline{\overline{A \cdot B \cdot C}} \cdot \overline{\overline{A \cdot B \cdot C}} \cdot \overline{\overline{A \cdot B \cdot C}} \cdot \overline{\overline{A \cdot B \cdot C}} \cdot \overline{\overline{A \cdot B \cdot C}}$$

$$\overline{(X \cdot Y \cdot Z)} = \overline{X} + \overline{Y} + \overline{Z}$$

Implementation of Boolean Functions

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

$$\bar{F} = (\bar{A}\bar{B}\bar{C}) + (\bar{A}\bar{B}C) + (\bar{A}B\bar{C}) + (\bar{A}BC) + (ABC)$$

- Itemising combinations of values of A, B, and C that ensure **NONE** of the input combinations that produces 0 is true

Product of Sums (POS)

► From DeMorgan's theorem

$$\text{DM1 } \overline{(X + Y + Z)} = \bar{X} \cdot \bar{Y} \cdot \bar{Z} \quad \text{DM2 } \overline{(X \cdot Y \cdot Z)} = \bar{X} + \bar{Y} + \bar{Z}$$

- Rewrite

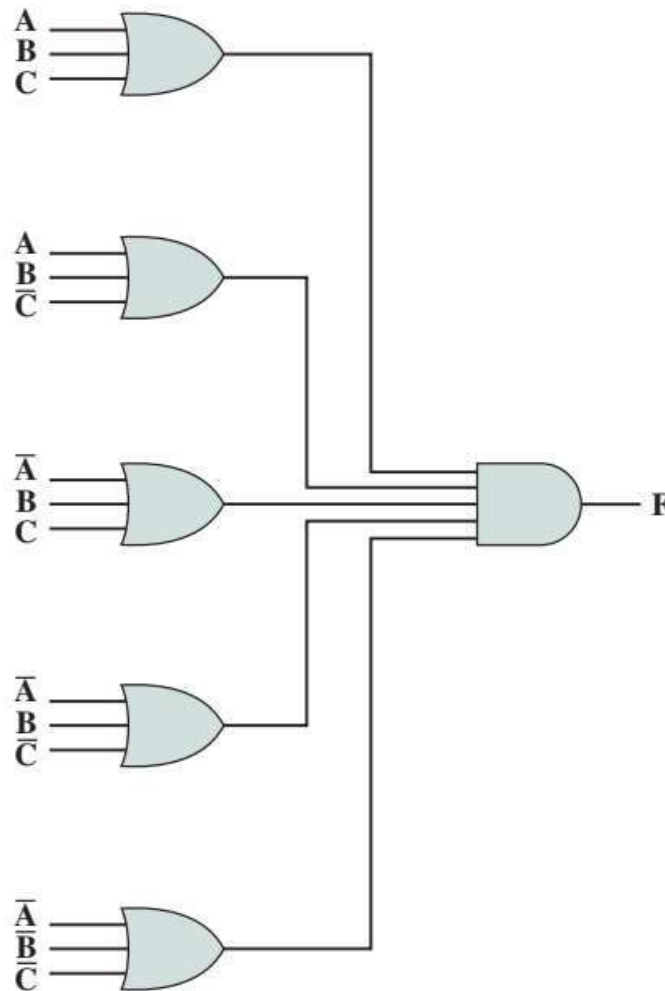
$$\bar{F} = (\bar{A}\bar{B}\bar{C}) + (\bar{A}\bar{B}C) + (\bar{A}B\bar{C}) + (\bar{A}BC) + (ABC)$$

$$F = \overline{(\bar{A}\bar{B}\bar{C}) + (\bar{A}\bar{B}C) + (\bar{A}B\bar{C}) + (\bar{A}BC) + (ABC)}$$

$$\text{DM1 } F = \overline{(\bar{A}\bar{B}\bar{C})} \cdot \overline{(\bar{A}\bar{B}C)} \cdot \overline{(\bar{A}B\bar{C})} \cdot \overline{(\bar{A}BC)} \cdot \overline{(ABC)}$$

$$\begin{aligned} \text{DM2 } F &= (\bar{\bar{A}} + \bar{\bar{B}} + \bar{\bar{C}}) \cdot (\bar{\bar{A}} + \bar{\bar{B}} + \bar{C}) \cdot (\bar{\bar{A}} + \bar{B} + \bar{\bar{C}}) \cdot (\bar{A} + \bar{\bar{B}} + \bar{C}) \cdot (\bar{A} + \bar{B} + \bar{\bar{C}}) \\ &= (A + B + C) \cdot (A + B + \bar{C}) \cdot (\bar{A} + B + C) \cdot (\bar{A} + B + \bar{C}) \cdot (\bar{A} + \bar{B} + \bar{C}) \end{aligned}$$

Product of Sums (POS)



$$F = (A+B+C) \cdot (A+B+\bar{C}) \cdot (\bar{A}+B+C) \cdot (\bar{A}+B+\bar{C}) \cdot (\bar{A}+\bar{B}+\bar{C})$$

Quick Activity #1

- Inspect each of these Boolean expressions, and determine whether each one is a sum of products (SOP) , or a product of sums (POS)

$$(B + \overline{C} + D)(\overline{A} + B)$$

$$A\overline{B}\overline{C} + \overline{A}BC$$

$$(X + \overline{Y} + \overline{Z})(\overline{Y} + Z)(\overline{X} + Y)$$

$$\overline{M}\overline{N}\overline{O} + MN\overline{O} + M\overline{N}O$$

$$(X + \overline{Y + Z})(\overline{Y + Z})$$

$$\overline{ABC} + A\overline{B}C$$

Quick Activity #2

- Write an SOP and POS expression for this truth table:

A	B	C	Output
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

SOP

A	B	C	Output
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

$$F = \bar{A}B\bar{C} + \bar{A}BC + ABC$$

POS

	A	B	C	Output	
	0	0	0	0	
	0	0	1	0	
	0	1	0	1	
	0	1	1	1	
	1	0	0	0	
	1	0	1	0	
	1	1	0	0	
	1	1	1	1	

$$\bar{F} = \bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}C + A\bar{B}\bar{C} + A\bar{B}C + AB\bar{C}$$

$$\bar{F} = \bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}C + A\bar{B}\bar{C} + A\bar{B}C + AB\bar{C}$$

$$F = \overline{\bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}C + A\bar{B}\bar{C} + A\bar{B}C + AB\bar{C}}$$

$$F = (\overline{\bar{A}\bar{B}\bar{C}}) \cdot (\overline{\bar{A}\bar{B}C}) \cdot (\overline{A\bar{B}\bar{C}}) \cdot (\overline{A\bar{B}C}) \cdot (\overline{AB\bar{C}})$$

$$F = (\bar{\bar{A}} + \bar{\bar{B}} + \bar{\bar{C}}) \cdot (\bar{\bar{A}} + \bar{\bar{B}} + \bar{C}) \cdot (\bar{A} + \bar{\bar{B}} + \bar{\bar{C}}) \cdot (\bar{A} + \bar{\bar{B}} + \bar{C}) \cdot (\bar{A} + \bar{B} + \bar{\bar{C}})$$

$$F = (A + B + C) \cdot (A + B + \bar{C}) \cdot (\bar{A} + B + C) \cdot (\bar{A} + B + \bar{C}) \cdot (\bar{A} + \bar{B} + C)$$

Sum of Products vs Product of Sums

- ▶ A Boolean function can be realized in either SOP or POS form.
- ▶ At this point, it would seem that the choice would depend on whether the truth table contains more 1s or 0s for the output function:
- ▶ The SOP has one term for each 1, and the POS has one term for each 0. However, there are other considerations:
 - ▶ It is often possible to derive a simpler Boolean expression from the truth table than either SOP or POS.
 - ▶ It may be preferable to implement the function with a single gate type (NAND or NOR)

Sum of Products vs Product of Sums

- ▶ The significance of the first point is that, with a simpler Boolean expression, fewer gates will be needed to implement the function.
- ▶ Three methods that can be used to achieve simplification are

Algebraic
simplification

Karnaugh maps

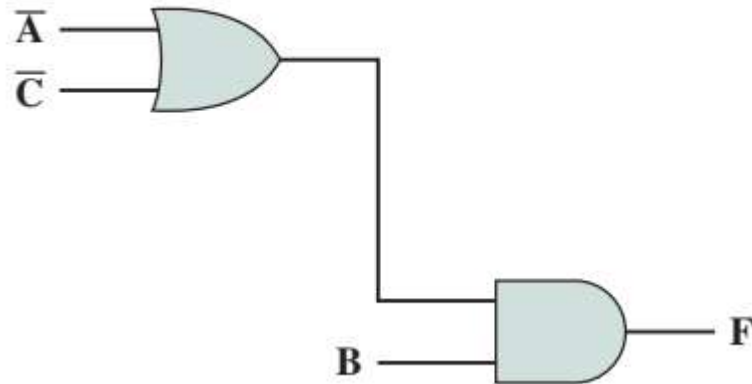
Quine-McCluskey
Tables

Algebraic simplification

$$F = \overline{A}B\overline{C} + \overline{A}BC + AB\overline{C}$$

$$F = \overline{A}B + B\overline{C}$$

$$F = B(\overline{A} + \overline{C})$$



AB \ CD		CD			
		00	01	11	10
AB	00	X ⁰	X ¹	0 ³	X ²
	01	1 ⁴	1 ⁵	1 ⁷	0 ⁶
	11	0 ¹²	1 ¹³	0 ¹⁵	1 ¹⁴
	10	1 ⁸	0 ⁹	1 ¹¹	1 ¹⁰

Karnaugh maps

Karnaugh Maps

- ▶ For purposes of simplification, the Karnaugh map is a convenient way of representing a Boolean function of a small number (**up to four**) of variables.
- ▶ The map is an array of 2^n squares, representing all possible combinations of values of n binary variables.
- ▶ It is essential for later purposes to list the combinations in the order **00, 01, 11, 10**.
- ▶ Because the squares corresponding to the combinations are to be used for recording information, the combinations are customarily written above the squares.

Karnaugh Maps

2 variables

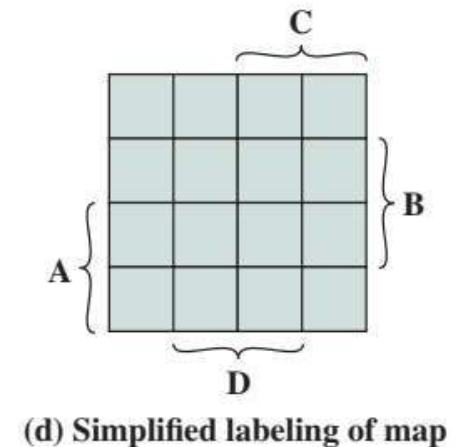
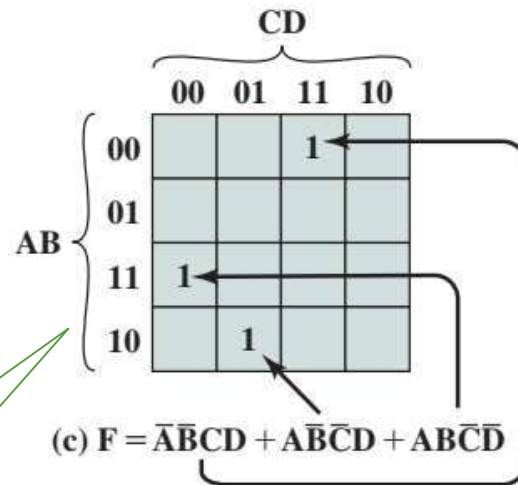
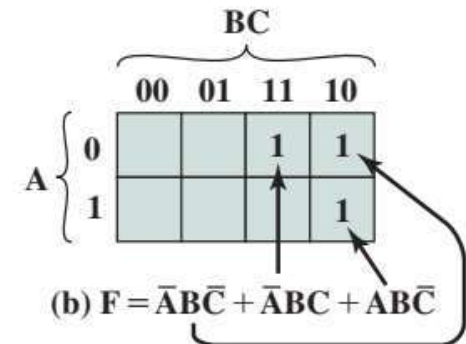
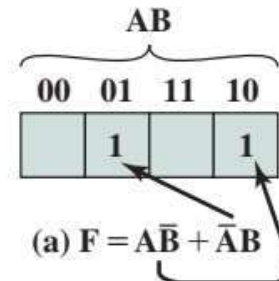
3 variables

- List of combination in order of 00, 01, 11, 10

► Differ in one digit when side-by-side

- Each square corresponds to unique product in **SOP form**
 - A 1 corresponding to the variable and a 0 to the NOT of that variable

4 variables



Constructing Karnaugh Maps

- ▶ To construct based on **truth table**
 - ▶ Easy
 - ▶ Fill in corresponding square if result is a 1

2 variables = 4 squares

3 variables = 8 squares

4 variables = 16 squares

Constructing Karnaugh Maps

- ▶ To construct from a **Boolean expression**
 - ▶ Need it in **canonical form**
 - ▶ Each term must contain each variable
 - ▶ we must first expand it into the full form of Equation
 - ▶ Then convert to the map

$$F = B(\overline{A} + \overline{C})$$



$$F = \overline{A}B\overline{C} + \overline{A}BC + AB\overline{C}$$

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

$$F = \overline{A}B\overline{C} + \overline{A}BC + AB\overline{C}$$

2
1
2

$$= \overline{A}B (\overline{C} + C) + B\overline{C} (\overline{A} + A)$$

$$= \overline{A}B (1) + B\overline{C} (1)$$

$$= \overline{A}B + B\overline{C}$$

$$F = \overline{A}B\overline{C} + \overline{A}BC + AB\overline{C}$$

2
1

$$F = \overline{A}B\overline{C} + \overline{A}BC + A\overline{B}\overline{C}$$

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

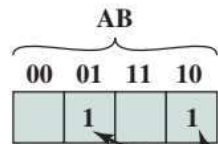
BC

		BC			
		00	01	11	10
A	0			1	1
	1				1

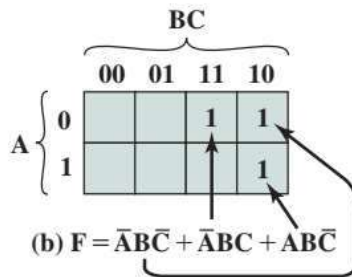
$$F = \overline{A}B\overline{C} + \overline{A}BC + A\overline{B}\overline{C}$$

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

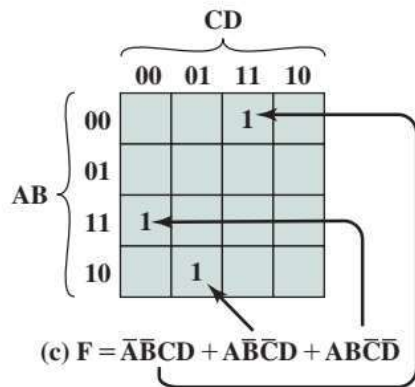
		C	
		┌───────────┐	
		0	1
AB	00		
	01	1	1
	11	1	
	10		



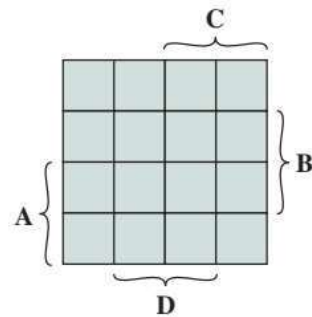
(a) $F = A\bar{B} + \bar{A}B$



(b) $F = \bar{A}\bar{B}\bar{C} + \bar{A}BC + ABC\bar{C}$



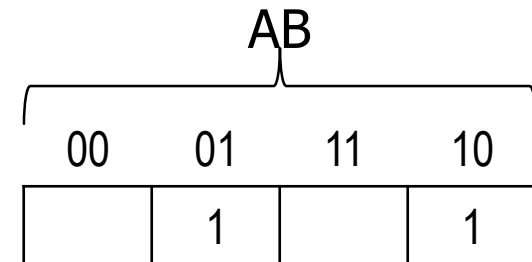
(c) $F = \bar{A}\bar{B}CD + \bar{A}B\bar{C}D + AB\bar{C}\bar{D}$

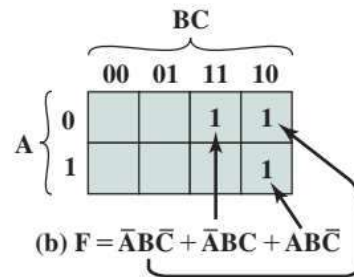
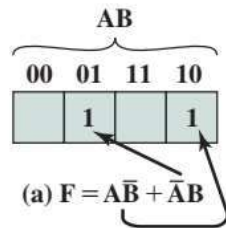


(d) Simplified labeling of map

$$F = A\bar{B} + \bar{A}B$$

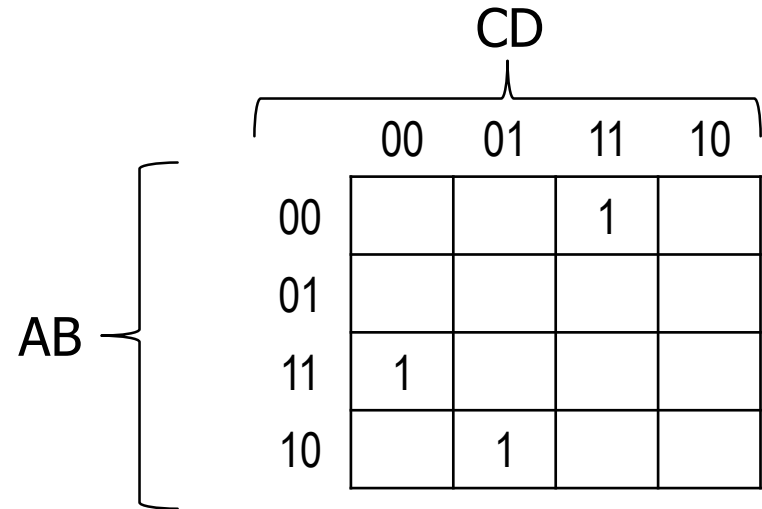
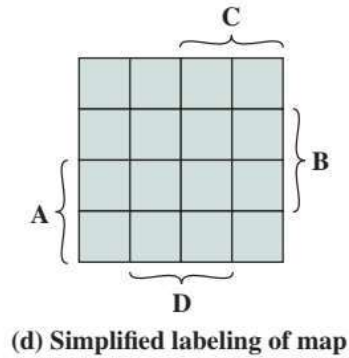
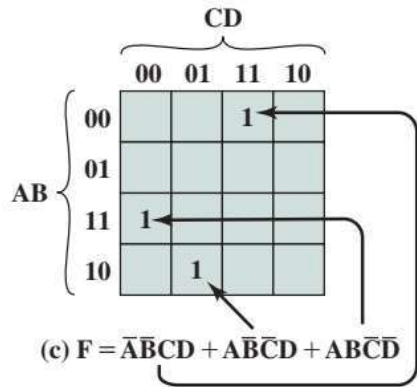
A	B	F
0	0	0
0 ($\bar{A}$)	1 (B)	1
1 (A)	0 ($\bar{B}$)	1
1	1	1

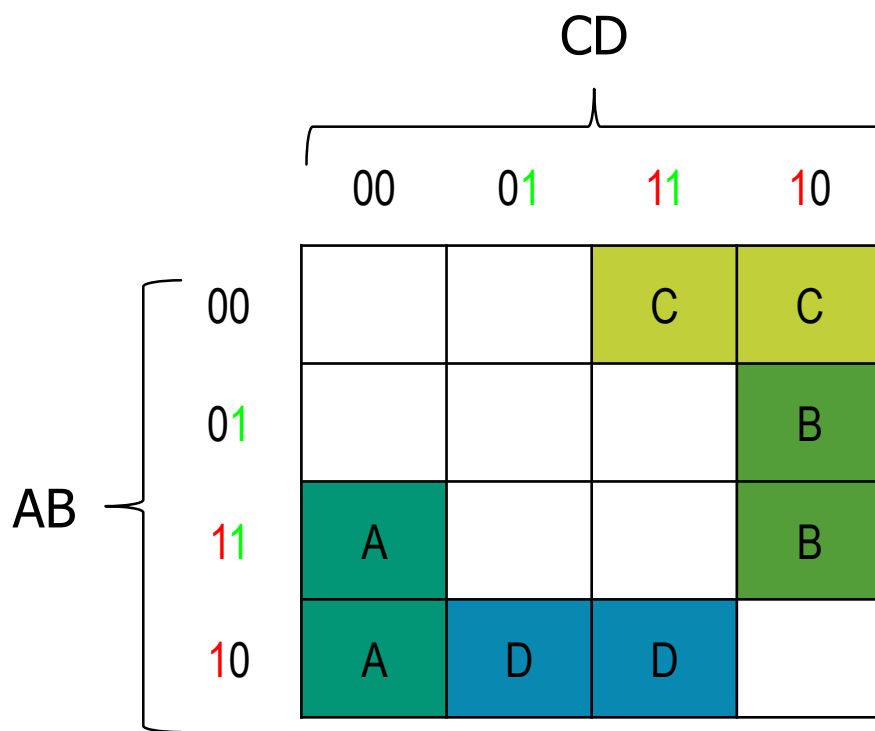
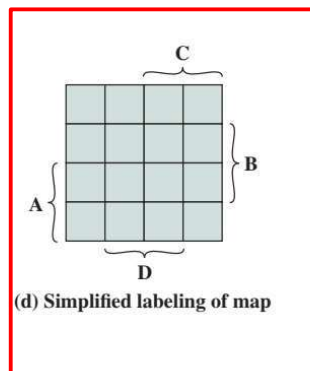
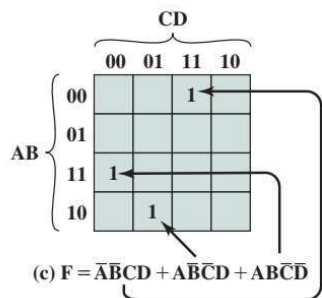
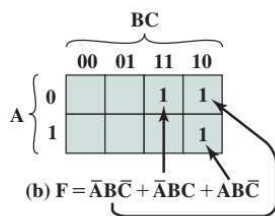
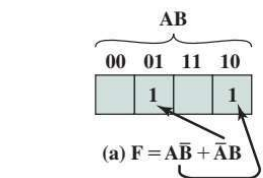




$$F = \bar{A}\bar{B}CD + \bar{A}B\bar{C}D + AB\bar{C}D$$

0011 1001 1100





A simplified table to show the true value for A, B, C and D

Use of Karnaugh Maps

► Algebraic

► Principle

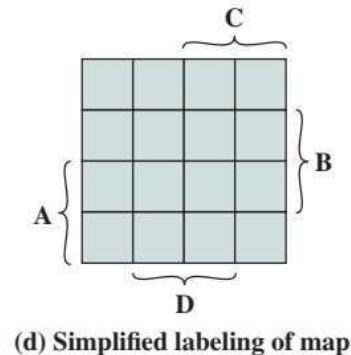
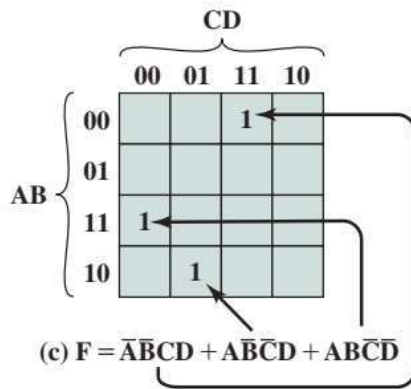
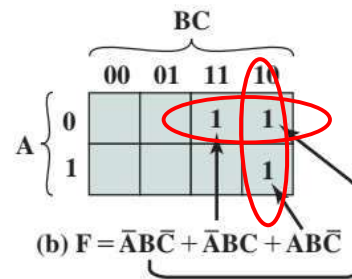
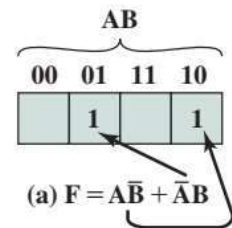
► Any t

va

► If **2**

ca

► Squa

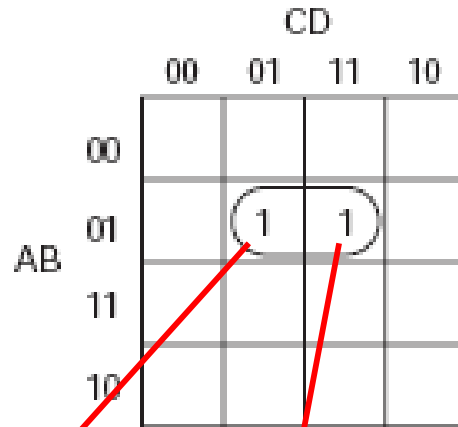


of 1s on the map

ily one

1e **2 terms**
at variable

Use of Karnaugh Maps



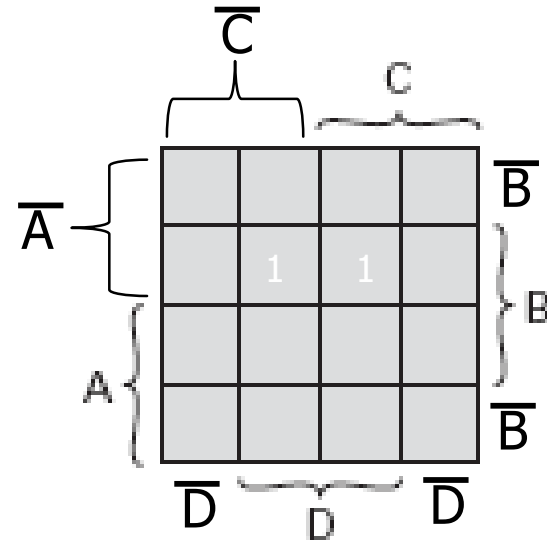
(a) $\bar{A}BD$

$$\begin{array}{ccc} \bar{A}\bar{B}\bar{C}D & + & \bar{A}B\bar{C}D = \bar{A}BD \\ 0101 & & 0111 \end{array}$$

$$= \bar{A}BD (\bar{C} + C)$$

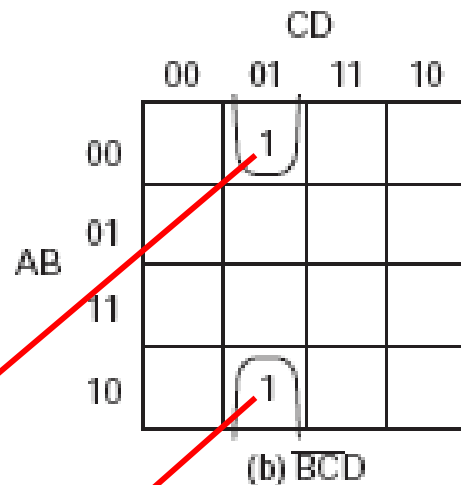
$$= \bar{A}BD (1)$$

$$= \bar{A}BD$$



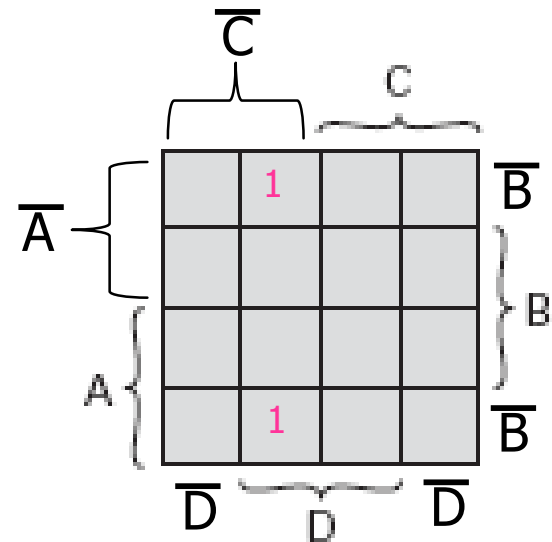
Use of Karnaugh Maps

- When wrapped around the edges



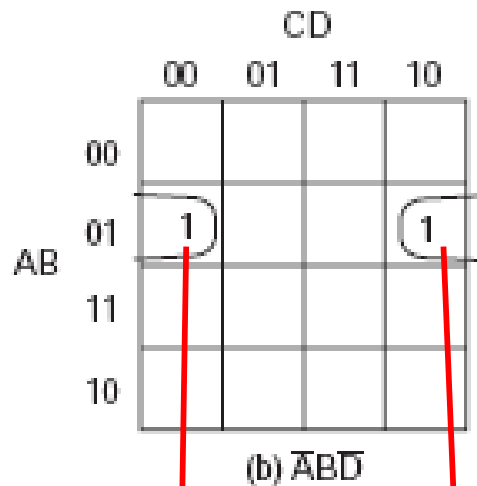
$$\begin{array}{c} \bar{A}\bar{B}\bar{C}D \\ 0001 \end{array} + \begin{array}{c} A\bar{B}\bar{C}D \\ 1001 \end{array} = \begin{array}{c} \bar{B}\bar{C}D \\ \text{---} \end{array}$$

A



Use of Karnaugh Maps

- When wrapped around the edges

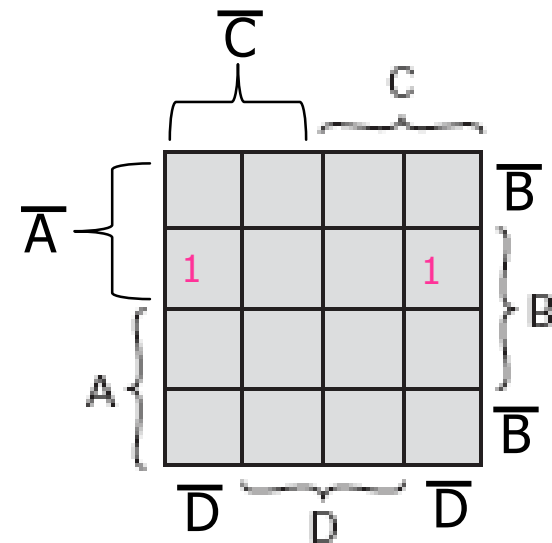


$$\bar{A}B\bar{C}\bar{D} + \bar{A}BC\bar{D} = \bar{A}B\bar{D}$$

0100

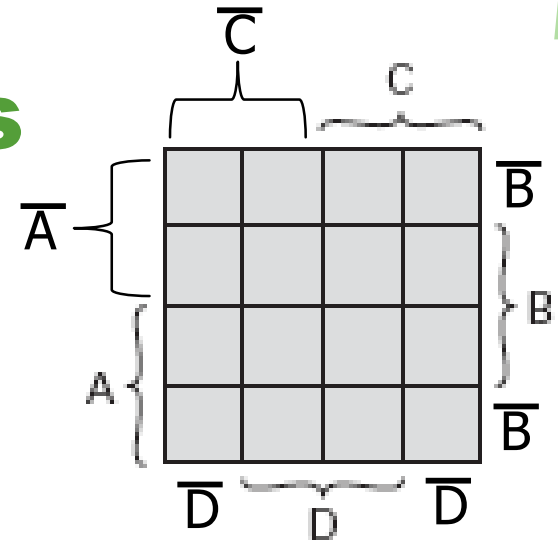
0110

$\bar{C}$



Use of Karnaugh Maps

- ▶ 2^n adjacent squares can be groups (2, 4, 8, etc.)
- ▶ Grouping of 4 squares eliminate 2 variables, with 8 – 3 variables



		CD			
		00	01	11	10
AB	00	1	1	1	1
	01				
	11				
	10				

(d) $\overline{A}\overline{B}$

		CD			
		00	01	11	10
AB	00				
	01	1	1		
	11	1	1		
	10				

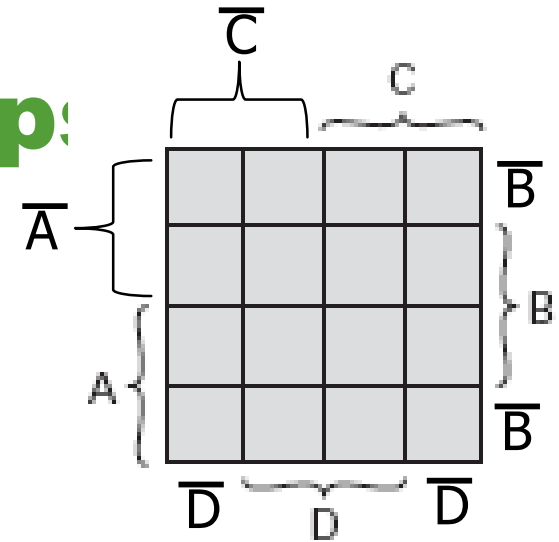
(e) $B\overline{C}$

		CD			
		00	01	11	10
AB	00				
	01	1			1
	11	1			1
	10				

(f) $B\overline{D}$

2 variables side by side – we drop 1 variables
 4 variables side by side – we drop 2 variables
 8 variables side by side – we drop 3 variables
 16 variables side by side – we drop all 4 variables

Use of Karnaugh Maps



		CD			
		00	01	11	10
AB	00	1	1	1	1
	01	1	1	1	1
	11				
	10				

(g) $\overline{A}$

		CD			
		00	01	11	10
AB	00	1			1
	01	1			1
	11	1			1
	10	1			1

(h) $\overline{D}$

		CD			
		00	01	11	10
AB	00			1	1
	01			1	1
	11			1	1
	10			1	1

(i) C

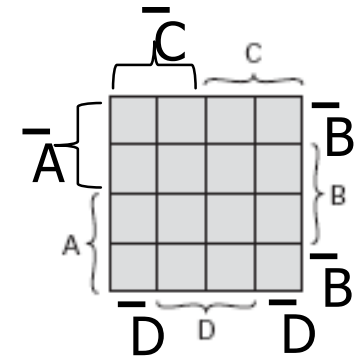
2 variables side by side – we drop 1 variables
 4 variables side by side – we drop 2 variables
 8 variables side by side – we drop 3 variables
 16 variables side by side – we drop all 4 variables

Rules for Simplification

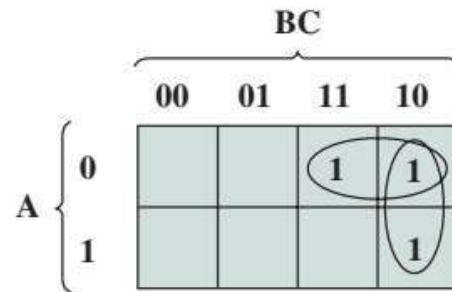
1. For squares with 1s, look for largest block of 2, 4 or 8 variables and circle.
2. Select additional blocks of marked square as large as possible and as few in number as possible. Include every marked square at least once.
3. Continue to draw loops around single marked squares, or pairs, or block of fours, eights to include all marked squares.
 - ▶ If any of isolated 1s remain after the groupings, group each of these individually
 - ▶ Any group that is completely overlapped by other group can be eliminated

Rules for Simplification

► Overlapping

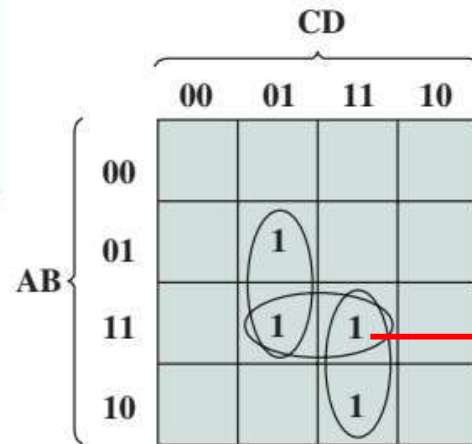


A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0



(a) $F = \bar{A}B + B\bar{C}$

Overlap in one cell



(b) $F = \bar{B}CD + ACD$

Eliminated

Rules for Simplification

- ▶ “Don’t care” condition marked as “d”
 - ▶ Certain combinations of values never occur, hence corresponding output never occur. It simplifies the expression since the value can be either 1 or 0
 - ▶ E.g. incrementer

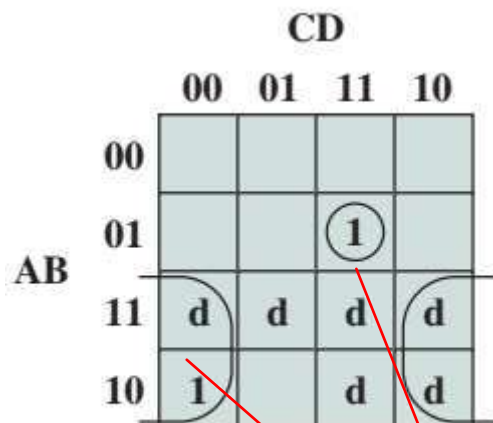
in packed decimal

Number	Input				Number	Output			
	A	B	C	D		W	X	Y	Z
0	0	0	0	0	1	0	0	0	1
1	0	0	0	1	2	0	0	1	0
2	0	0	1	0	3	0	0	1	1
3	0	0	1	1	4	0	1	0	0
4	0	1	0	0	5	0	1	0	1
5	0	1	0	1	6	0	1	1	0
6	0	1	1	0	7	0	1	1	1
7	0	1	1	1	8	1	0	0	0
8	1	0	0	0	9	1	0	0	1
9	1	0	0	1	0	0	0	0	0
Don't care condition	1	0	1	0		d	d	d	d
	1	0	1	1		d	d	d	d
	1	1	0	0		d	d	d	d
	1	1	0	1		d	d	d	d
	1	1	1	0		d	d	d	d
	1	1	1	1		d	d	d	d

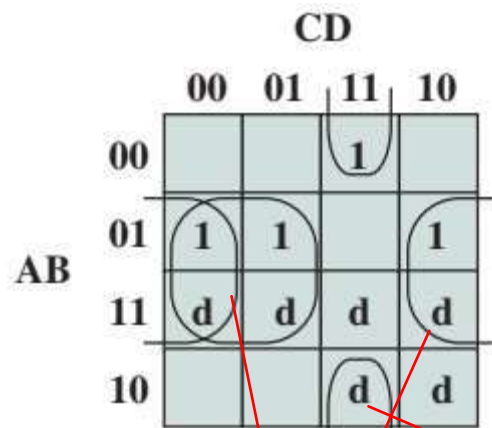
Don't care cannot form their own group

Example

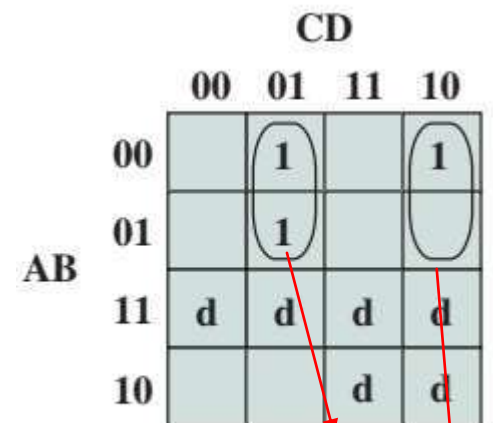
Number	Input				Number	Output			
	A	B	C	D		W	X	Y	Z
0	0	0	0	0	1	0	0	0	1
1	0	0	0	1	2	0	0	1	0
2	0	0	1	0	3	0	0	1	1
3	0	0	1	1	4	0	1	0	0
4	0	1	0	0	5	0	1	0	1
5	0	1	0	1	6	0	1	1	0
6	0	1	1	0	7	0	1	1	1
7	0	1	1	1	8	1	0	0	0
8	1	0	0	0	9	1	0	0	1
9	1	0	0	1	0	0	0	0	0
Don't care condition						d	d	d	d
						d	d	d	d
						d	d	d	d
						d	d	d	d
						d	d	d	d
						d	d	d	d



(a) $W = \bar{A}\bar{D} + \bar{A}BCD$



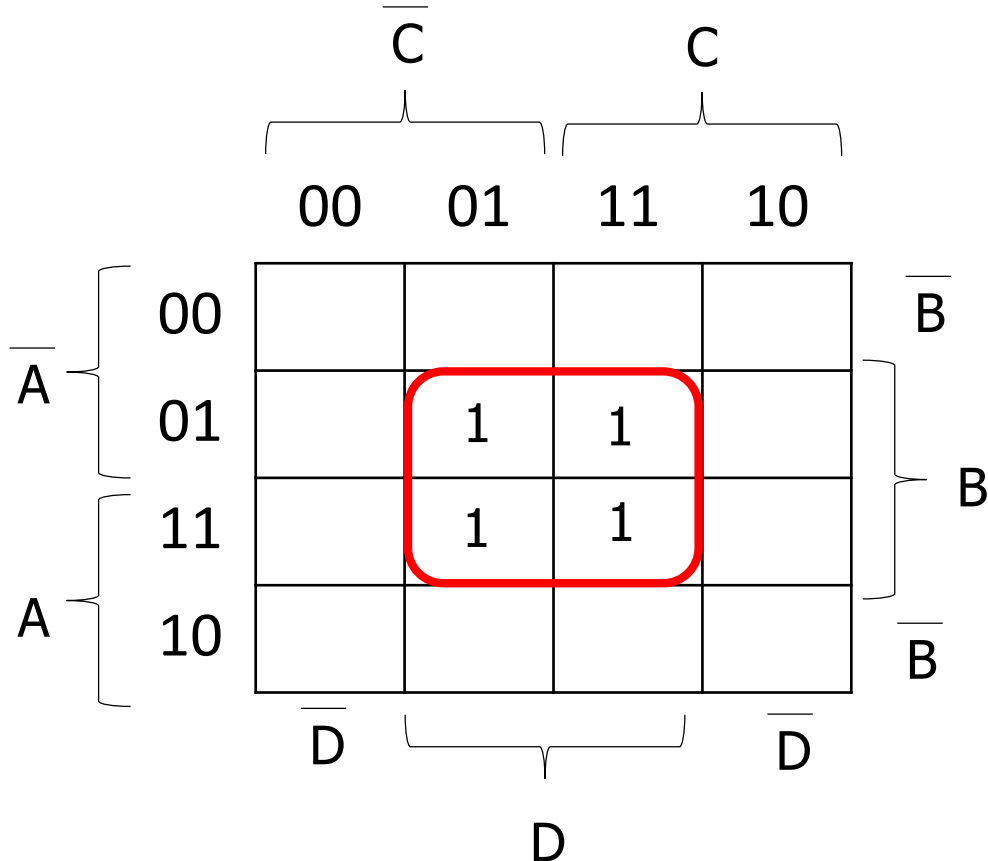
(b) $X = \bar{B}\bar{D} + BC + \bar{B}CD$



(c) $Y = \bar{A}\bar{C}D + \bar{A}C\bar{D}$

- Use Karnaugh map to simplify

$$F = \overline{A}\overline{B}\overline{C}D + \overline{A}BCD + A\overline{B}\overline{C}D + ABCD$$



$$F = BD$$

A	B	C	X
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

3 variables = 8 square

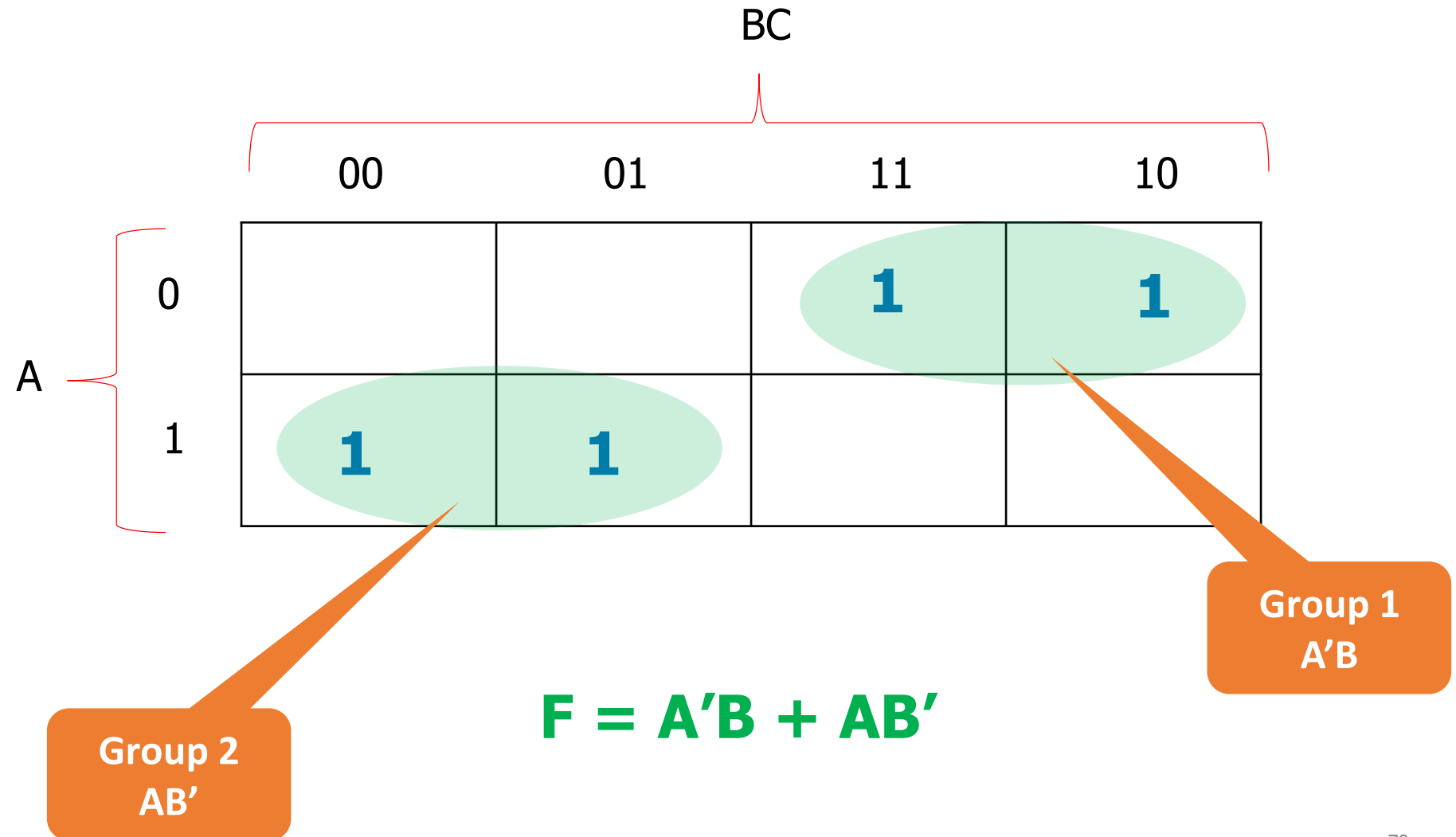
Labelling sequence = 00 01 11 10

Variable Labelling = A / BC

		BC			
		00	01	11	10
A	0				
	1				

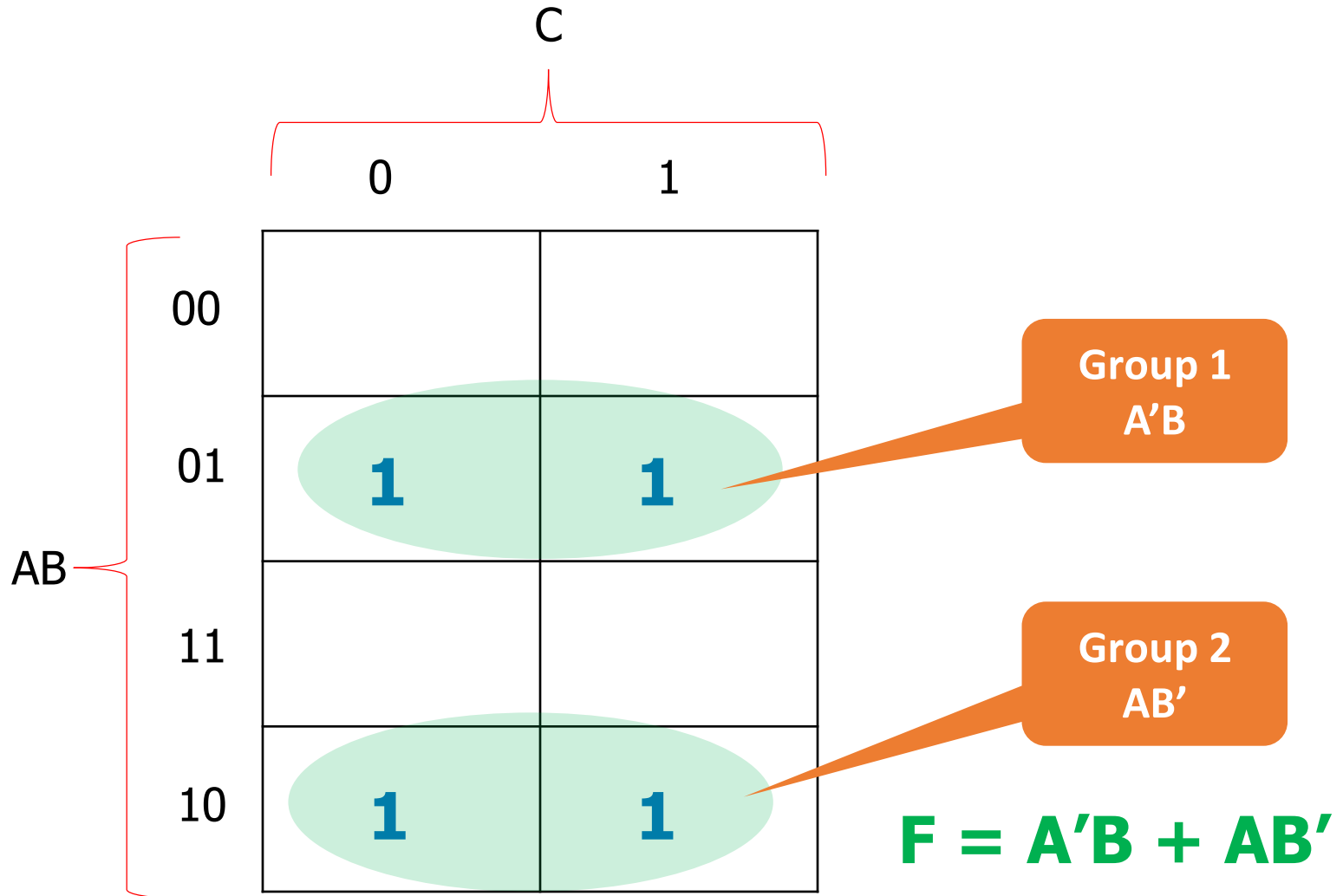
$$A'BC' + A'BC + AB'C' + AB'C$$

$$010 + 011 + 100 + 101$$



$$A'BC' + A'BC + AB'C' + AB'C$$

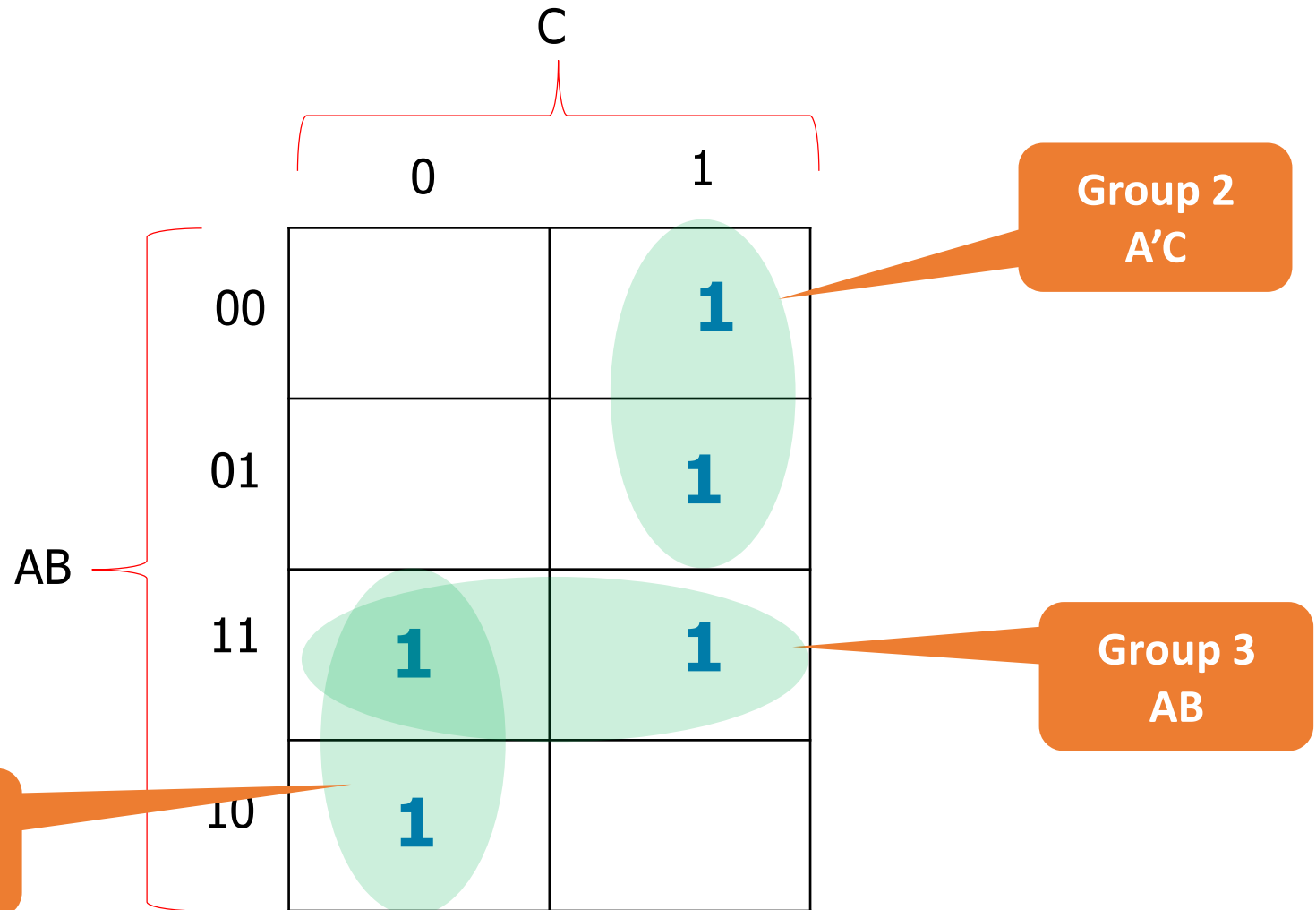
$$010 + 011 + 100 + 101$$



A	B	C	X
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

$$A'B'C + A'BC + AB'C' + ABC' + ABC$$

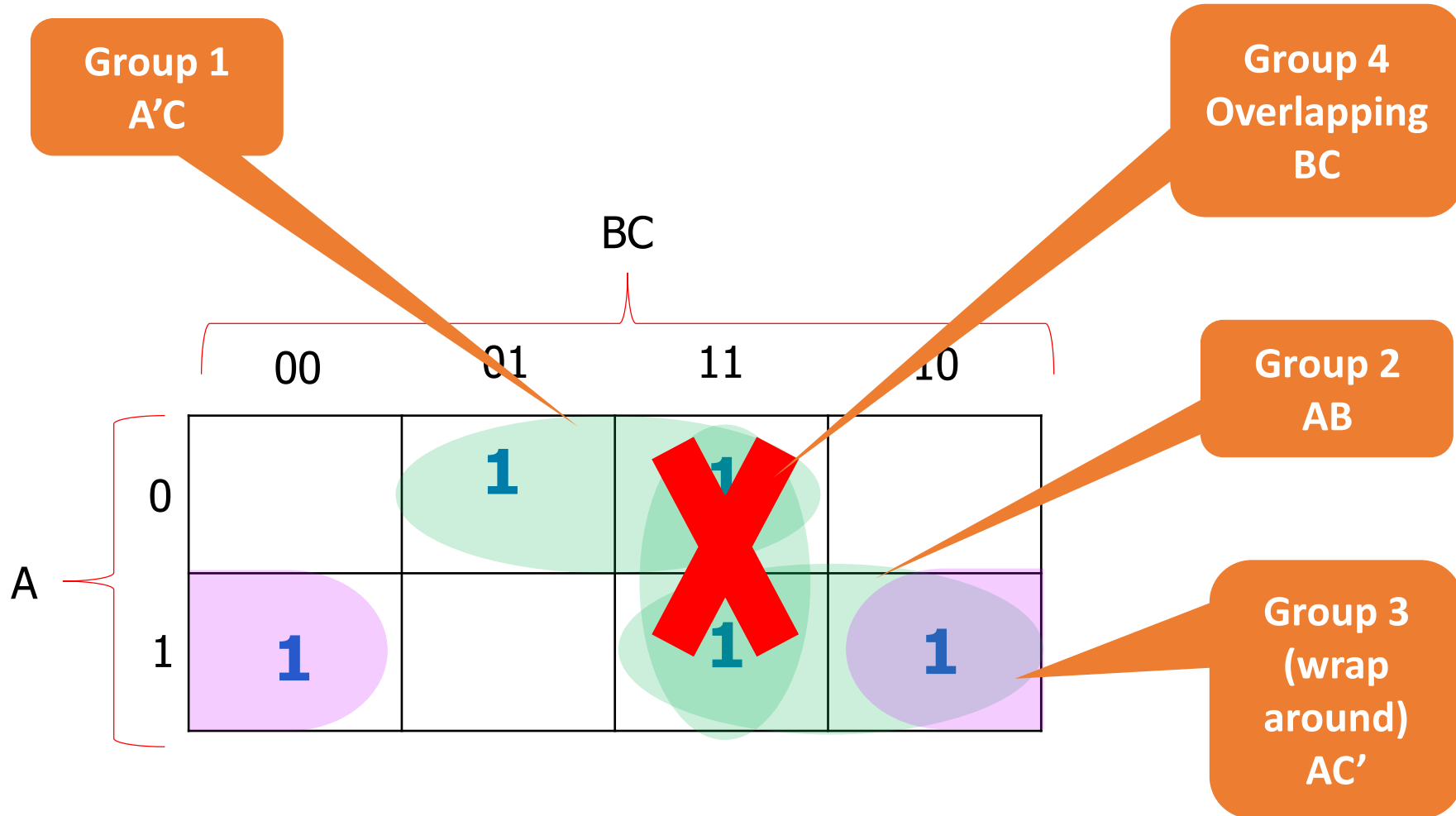
$$001 + 011 + 100 + 110 + 111$$



$$F = AC' + A'C + AB$$

$$A'B'C + A'BC + AB'C' + ABC' + ABC$$

$$001 + 011 + 100 + 110 + 111$$

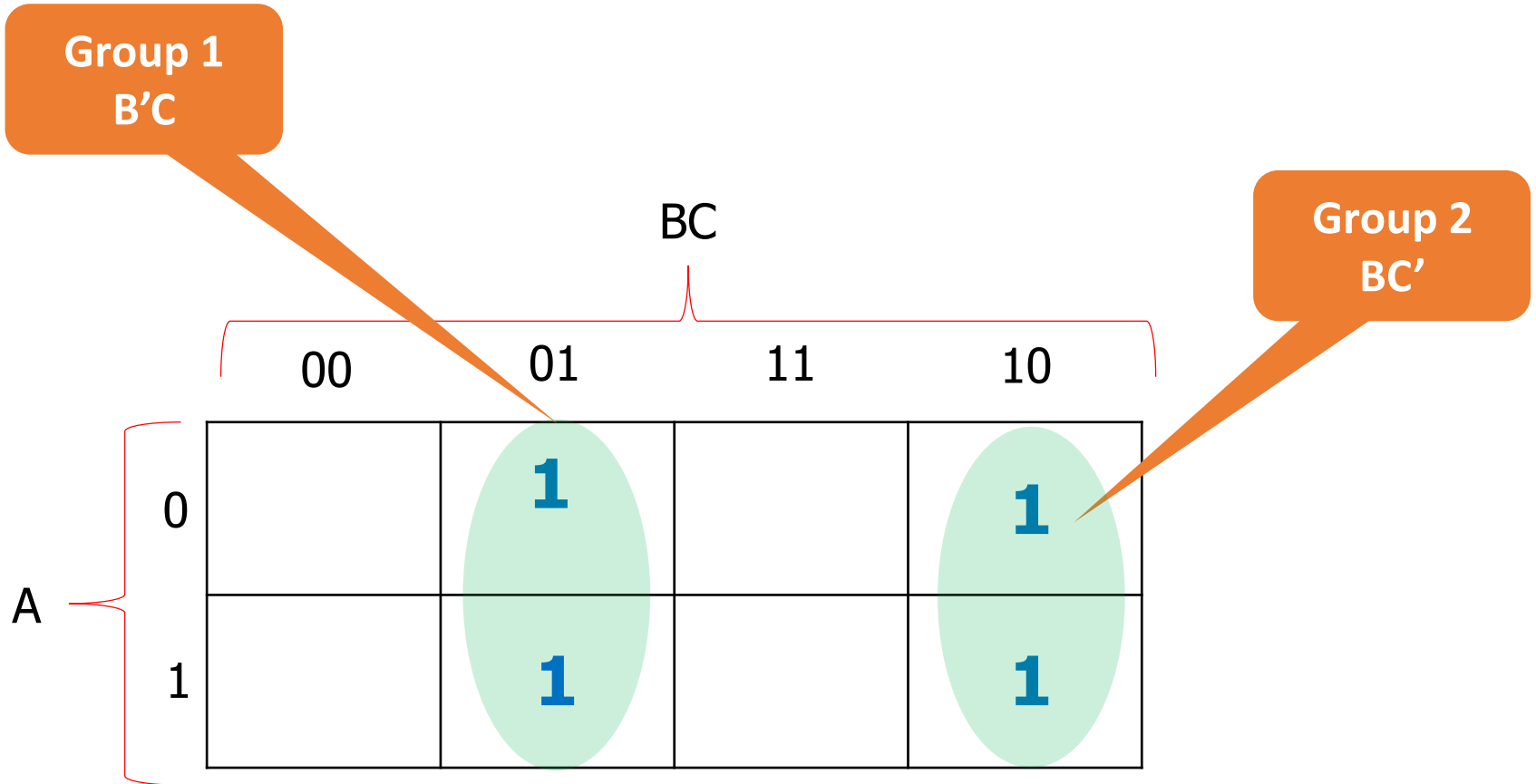


$$F = A'C + AB + AC'$$

A	B	C	X
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

$$A'B'C + A'BC' + AB'C + ABC'$$

$$001 + 010 + 101 + 110$$



$$F = B'C + BC'$$

$$A'B'C + A'BC' + AB'C + ABC'$$
$$001 + 010 + 101 + 110$$

Group 1
 BC'

		C	
		0	1
AB	00		1
	01	1	
	11	1	
	10		1

Group 2
 $B'C$

$$F = BC' + B'C$$

Quick Activity #3

- Simplify the following Boolean expression using the Karnaugh Maps method:

$$A'B'C' + A'B'C + AB'C' + AB'C$$

$$A'B'C + A'BC' + ABC' + AB'C'$$

$$A'B'C' + A'B'C + AB'C' + AB'C$$

$$000 + 001 + 100 + 101$$

		BC			
		00	01	11	10
A	0	1	1		
	1	1	1		

$$F = B'$$

$$A'B'C' + A'B'C + AB'C' + AB'C$$

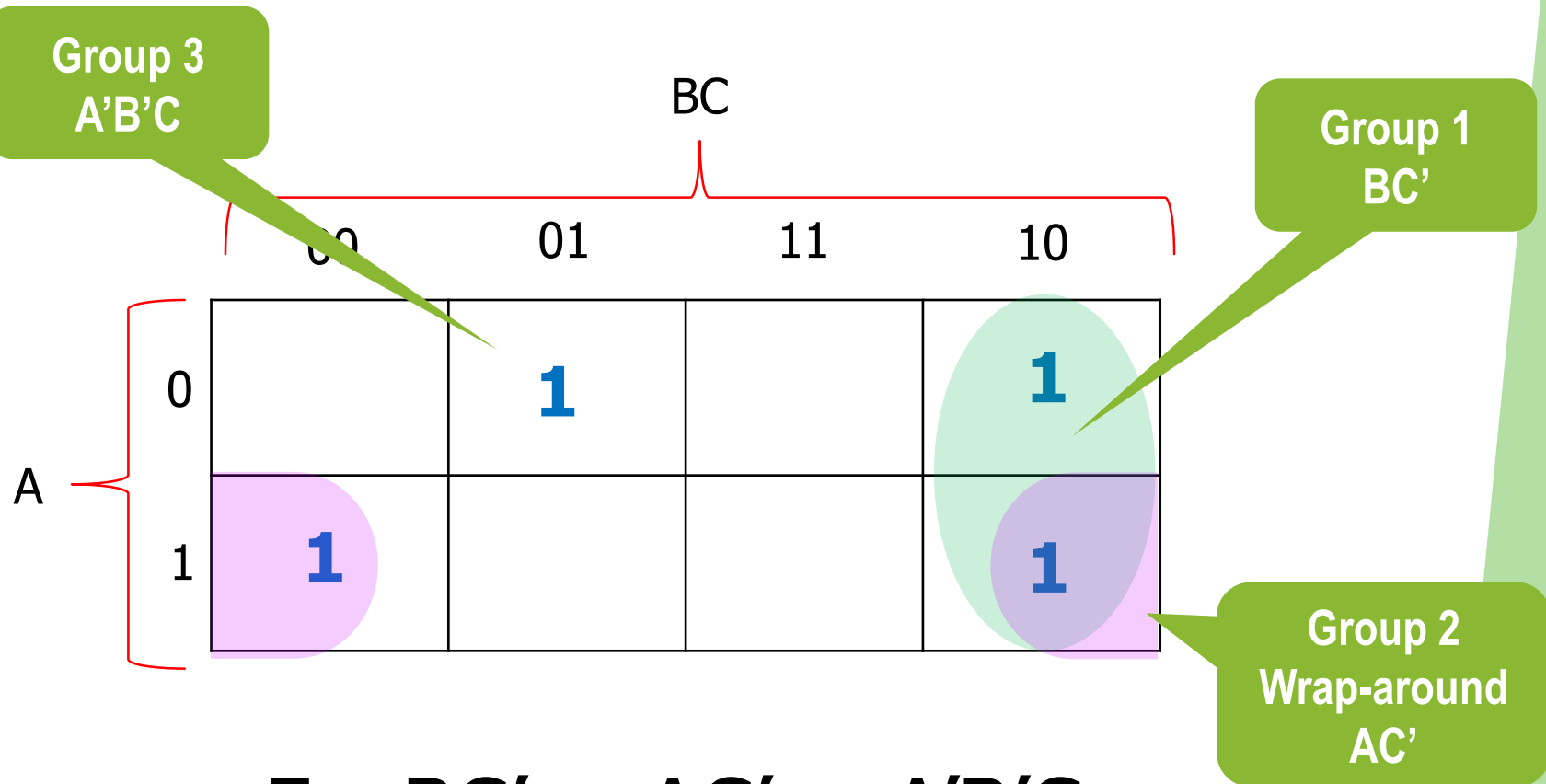
$$000 + 001 + 100 + 101$$

		C	
		0	1
AB	00	1	1
	01		
	11		
	10	1	1

$$F = B'$$

$$A'B'C + A'BC' + ABC' + AB'C'$$

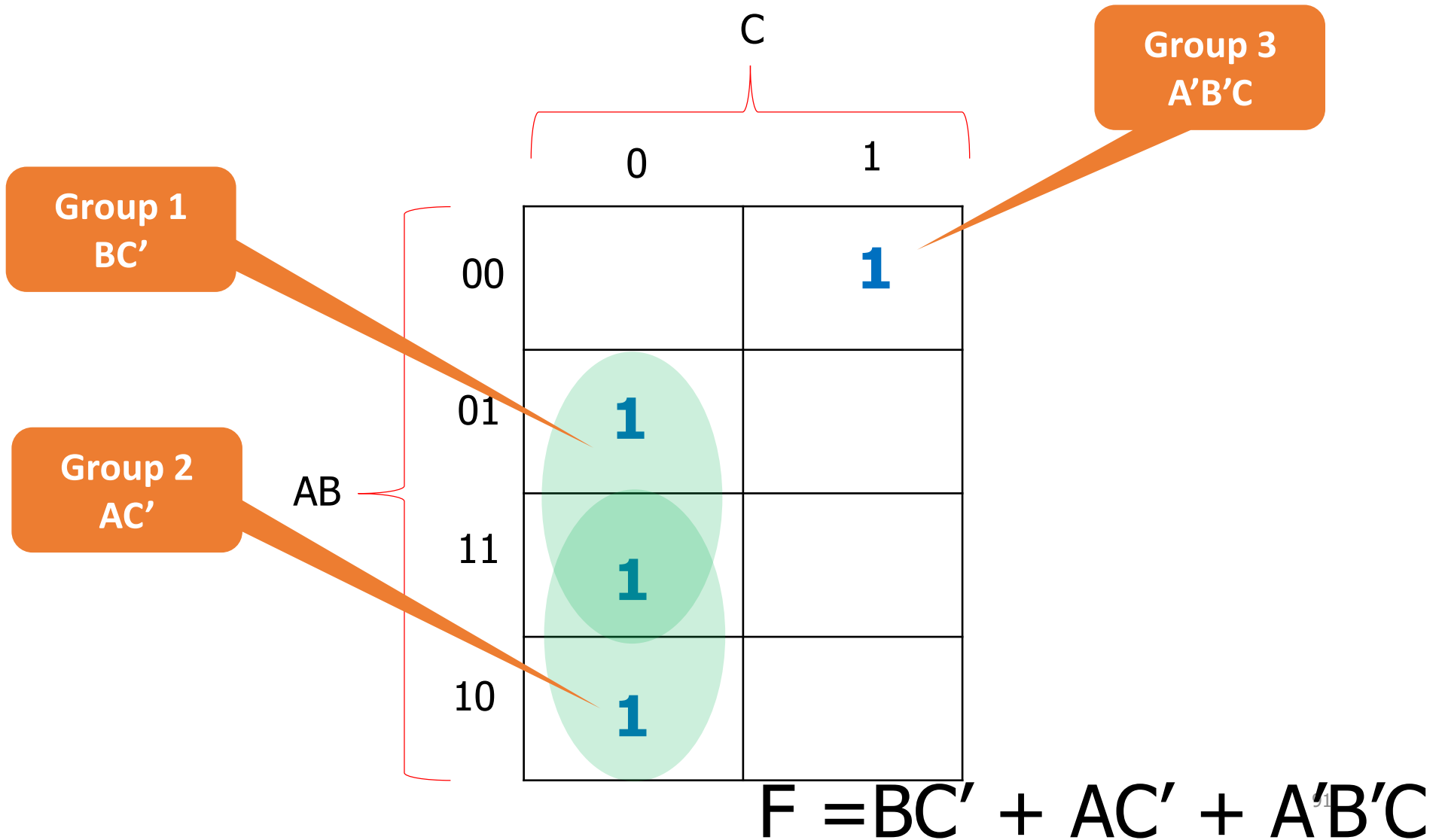
$$001 + 010 + 110 + 100$$



$$F = BC' + AC' + A'B'C$$

$$A'B'C + A'BC' + ABC' + AB'C'$$

$$001 + 010 + 110 + 100$$

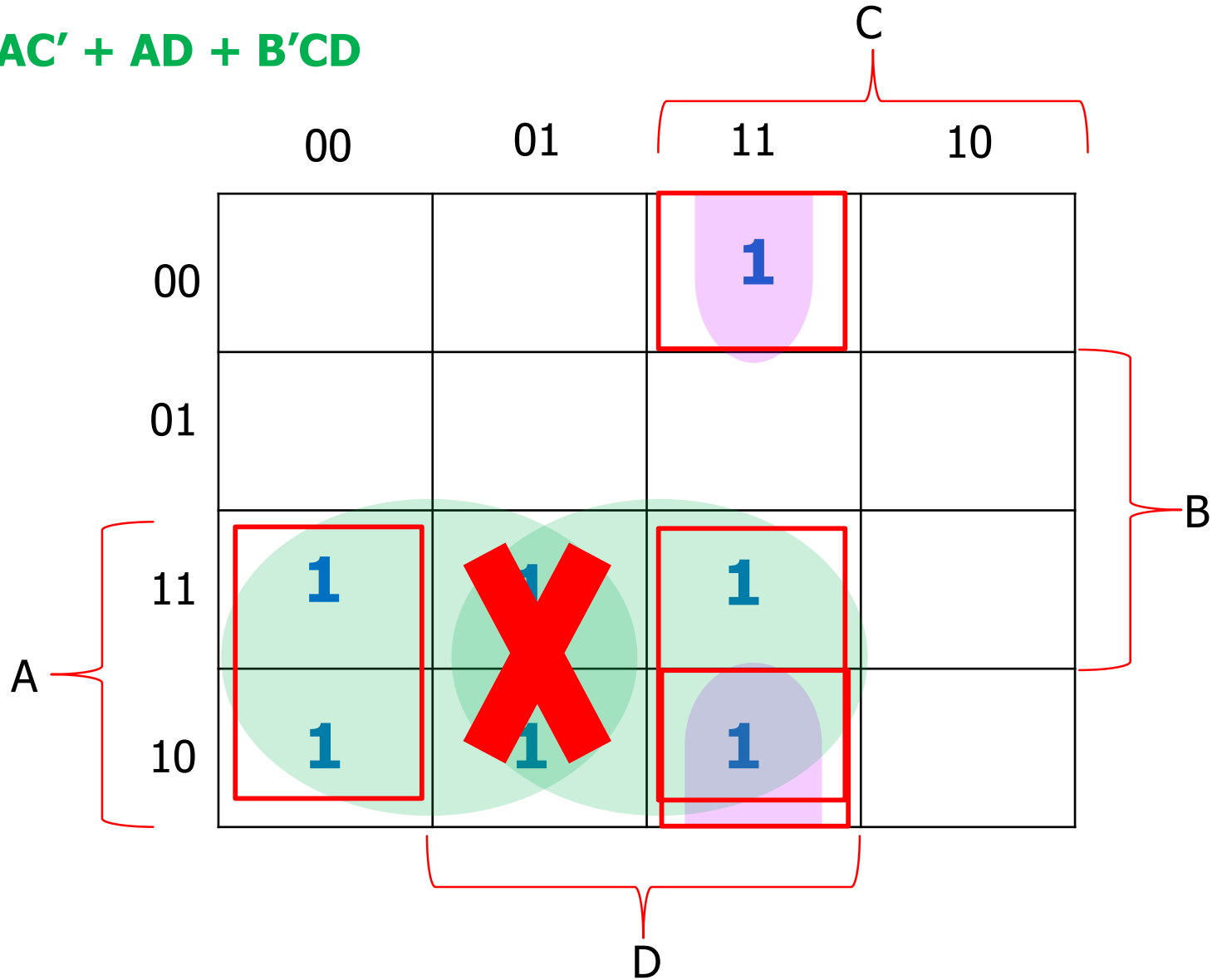


A	B	C	D	output
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	1
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1

$$A'B'CD + AB'C'D' + AB'C'D + AB'CD + ABC'D' + ABC'D + ABCD$$

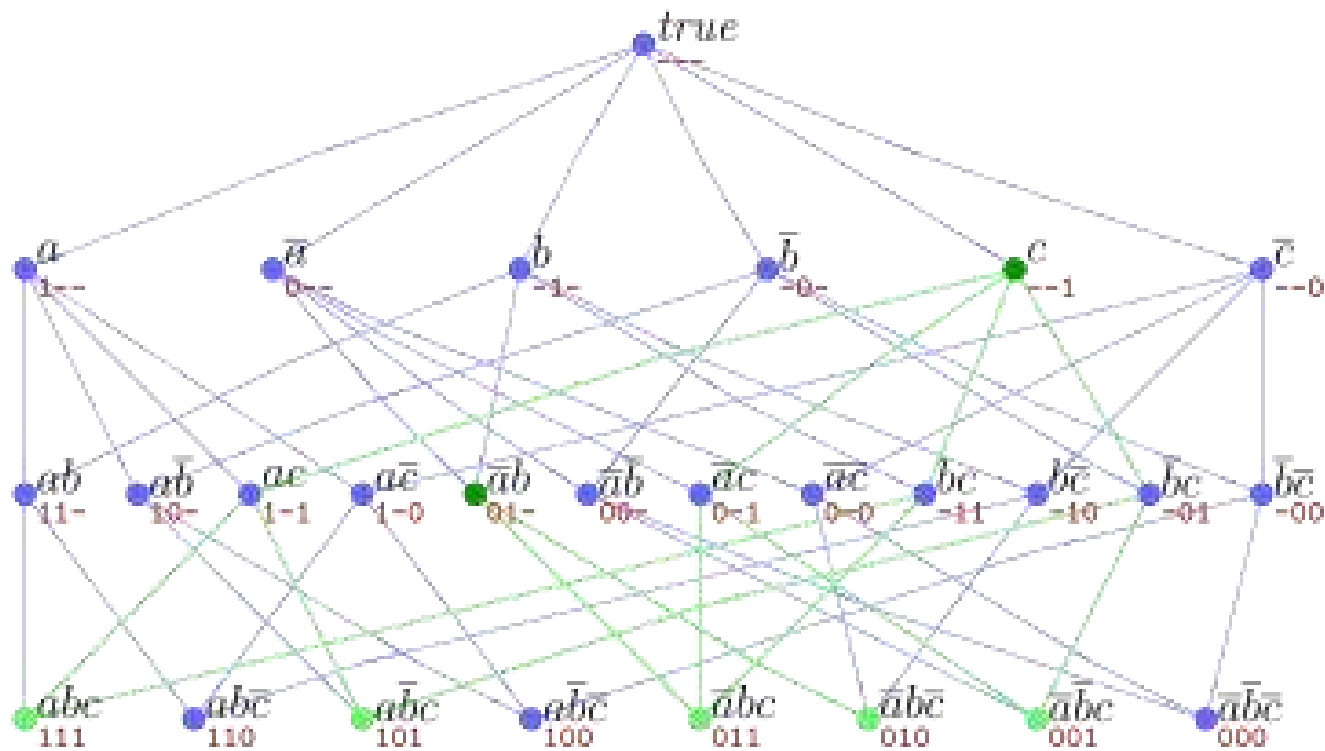
$$0011 + 1000 + 1001 + 1011 + 1100 + 1101 + 1111$$

$$F = AC' + AD + B'CD$$



Karnaugh Map

- ▶ Karnaugh map > 4 variables gets **cumbersome**
 - ▶ 5 variables: two 16 x 16 maps with one map on top of each other in **3D**
 - ▶ 6 variables: **4D**



Quine-McKluskey Method

Quine-McKluskey Method

- ▶ Quine-McKluskey method suitable for programming
 - ▶ The basic principle is the same, i.e. eliminating variables that complement each other

Quine-McKluskey Method

- Construct table in which each row corresponds to one of the product terms
 - Terms are grouped according to number of complemented variables

$$ABCD + AB\bar{C}D + AB\bar{C}\bar{D} + A\bar{B}CD + \bar{A}BCD + \bar{A}BC\bar{D} + \bar{A}\bar{B}CD + \bar{A}\bar{B}\bar{C}D$$

Product Term		Index	A				
G1	$\bar{A}\bar{B}\bar{C}D$	1	0	1. List the term based on the number of NOT (Larger to smaller)			
	$\bar{A}B\bar{C}D$	5	0				
G2	$\bar{A}BC\bar{D}$	6	0	1	1	0	✓
	$AB\bar{C}\bar{D}$	12	1	1	0	0	✓
	$\bar{A}BCD$	7	0	1	1	1	✓
G3	$A\bar{B}CD$	11	1	0	1	1	✓
	$AB\bar{C}D$	13	1	1	0	1	✓
G4	$ABCD$	15	1	1	1	1	✓

Quine-McKluskey Method

- Next step to find **all pairs of terms that differ in only one variable**
 - Start with first group and comparing each term of first group with every term of second group
 - We compare group 1 and 2, 2 and 3 & 3 and 4.

Product Term	Index	A	B	C	D	
$\overline{A} \overline{B} \overline{C} D$	1	0	0	0	1	✓
$\overline{A} B \overline{C} D$	5	0	1	0	1	✓
$\overline{A} B C \overline{D}$	6	0	1	1	0	✓
$A \overline{B} \overline{C} \overline{D}$	12	1	1	0	0	✓
$\overline{A} B C D$	7	0	1	1	1	✓
$A \overline{B} C D$	11	1	0	1	1	✓
$A B \overline{C} D$	13	1	1	0	1	✓
$A B C D$	15	1	1	1	1	✓

Quine-McKluskey Method

Product Term	Index	A	B	C	D	
$\bar{A}\bar{B}\bar{C}D$	1	0	0	0	1	✓
$\bar{A}B\bar{C}D$	5	0	1	0	1	✓
$\bar{A}BC\bar{D}$	6	0	1	1	0	✓
$AB\bar{C}\bar{D}$	12	1	1	0	0	✓
$\bar{A}BCD$	7	0	1	1	1	✓
$A\bar{B}CD$	11	1	0	1	1	✓
$AB\bar{C}D$	13	1	1	0	1	✓
$ABCD$	15	1	1	1	1	✓

$\bar{A}\bar{C}D$

$B\bar{C}D$
 $\bar{A}BD$
 $\bar{A}BC$
 $AB\bar{C}$

BCD
 ACD
 ABD

Quine-McKluskey Method

Product Term	Index	A	B	C	D	
$\bar{A}\bar{B}\bar{C}D$	1	0	0	0	1	✓
$\bar{A}B\bar{C}D$	5	0	1	0	1	✓
$\bar{A}BC\bar{D}$	6	0	1	1	0	✓
$AB\bar{C}\bar{D}$	12	1	1	0	0	✓
$\bar{A}BCD$	7	0	1	1	1	✓
$A\bar{B}CD$	11	1	0	1	1	✓
$AB\bar{C}D$	13	1	1	0	1	✓
$ABCD$	15	1	1	1	1	✓

$B\bar{C}D$

$\bar{A}BD$

$\bar{A}BC$

$AB\bar{C}$

$\bar{A}B\bar{C}D$

$\bar{A}B\bar{C}\bar{D}$

$\bar{A}BC\bar{D}$

$AB\bar{C}\bar{D}$

$AB\bar{C}D$

$\bar{A}BCD$

$\bar{A}BCD$

$AB\bar{C}D$

← A and Not A – differ in 1 variable

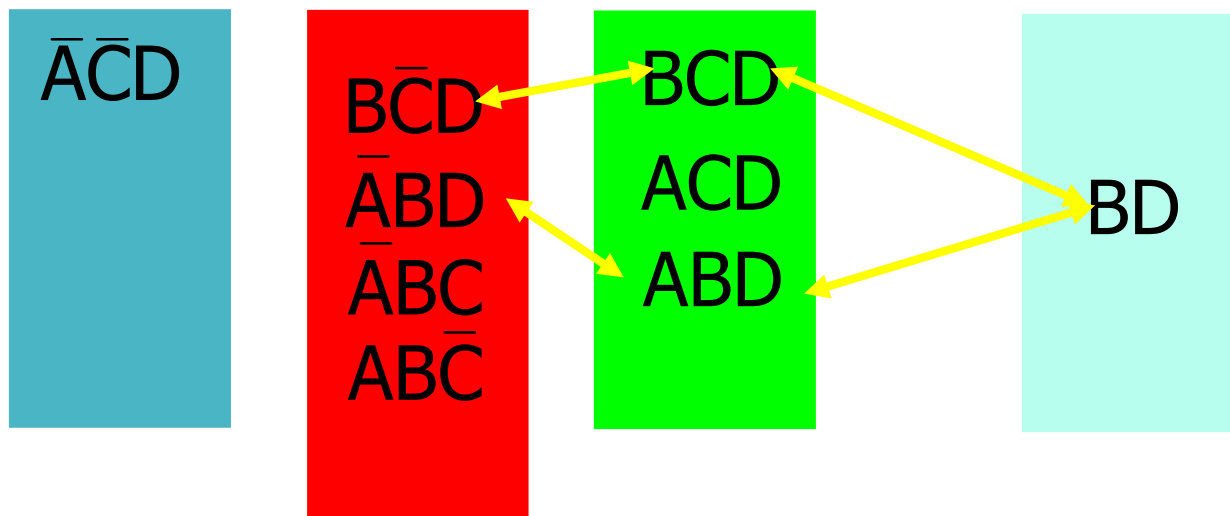
← Not C and C – differ in 1 variable

← Not D and D – differ in 1 variable

← Not D and D – differ in 1 variable

Quine-McKluskey Method

- ▶ New table also organised into groups
- ▶ Processed in the same way as first table where you will find the different pairs term



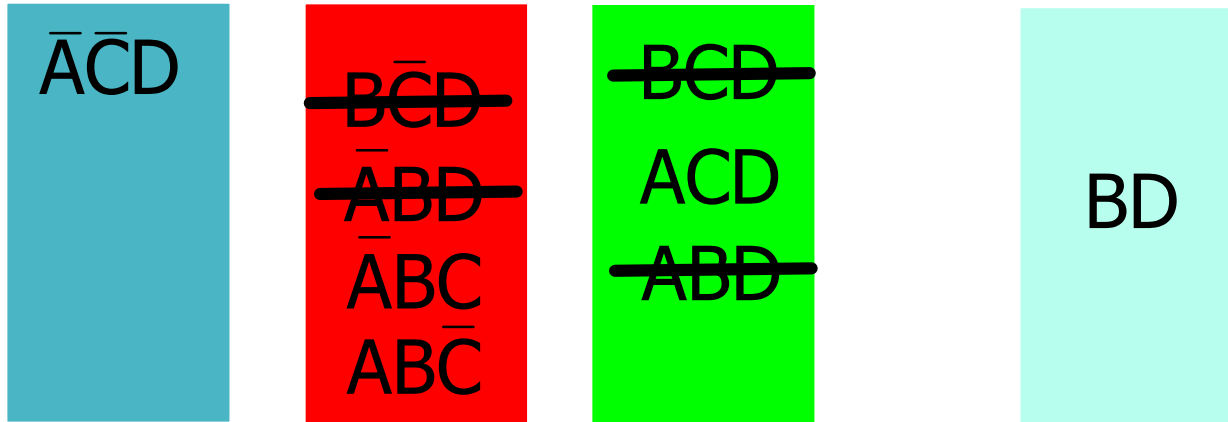
- The third table contains only one term: BD

Quine-McKluskey Method

- ▶ Terms that have not been eliminated are used to construct a matrix
 - ▶ Row: term that have not been eliminated
 - ▶ Column: Term in original expression
 - ▶ X: row and column “compatible”
 - ▶ $\bigcirc$ X X that is alone in column
 - ▶ $\boxed{X}$ X in any row with a $\bigcirc$ X
- ▶ If every column has either $\bigcirc$ X or $\boxed{X}$ then done
- ▶ Row elements whose Xs have been marked constitute the minimal expression

$$ABCD + AB\bar{C}D + AB\bar{C}\bar{D} + A\bar{B}CD + \bar{A}BCD + \bar{A}BC\bar{D} + \bar{A}\bar{B}CD + \bar{A}\bar{B}\bar{C}D$$

Quine-McKluskey Method



	$ABCD$	$AB\bar{C}D$	$AB\bar{C}\bar{D}$	$A\bar{B}CD$	$\bar{A}BCD$	$\bar{A}BC\bar{D}$	$\bar{A}\bar{B}CD$	$\bar{A}\bar{B}\bar{C}D$
BD	X	X			X		X	
$\bar{A}\bar{C}D$							X	⊗
$\bar{A}BC$					X	⊗		
$AB\bar{C}$		X	⊗					
ACD	X			⊗				

$$ABCD + AB\bar{C}D + AB\bar{C}\bar{D} + A\bar{B}CD + \bar{A}BCD + \bar{A}BC\bar{D} + \bar{A}\bar{B}CD + \bar{A}\bar{B}\bar{C}D$$

	$ABCD$	$AB\bar{C}D$	$AB\bar{C}\bar{D}$	$A\bar{B}CD$	$\bar{A}BCD$	$\bar{A}BC\bar{D}$	$\bar{A}\bar{B}CD$	$\bar{A}\bar{B}\bar{C}D$
BD	X	X			X		X	
$\bar{A}\bar{C}D$							X	⊗
$\bar{A}BC$					X	⊗		
$AB\bar{C}$		X	⊗					
ACD	X			⊗				

$\bar{A}\bar{C}D$

~~$B\bar{C}D$~~
 ~~$\bar{A}BD$~~
 $\bar{A}BC$
 $AB\bar{C}$

~~BCD~~
 ACD
 ~~ABD~~

BD

Row elements whose Xs have been marked constitute the minimal expression

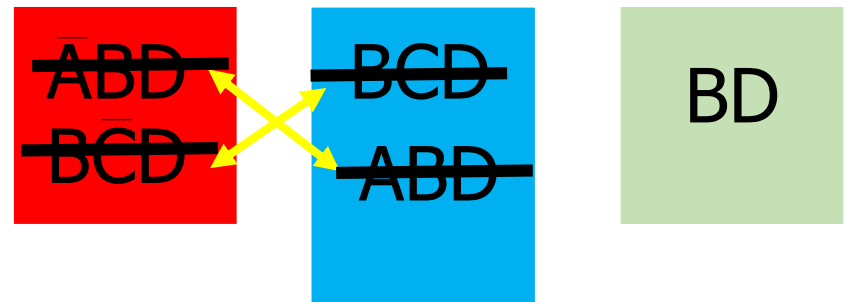
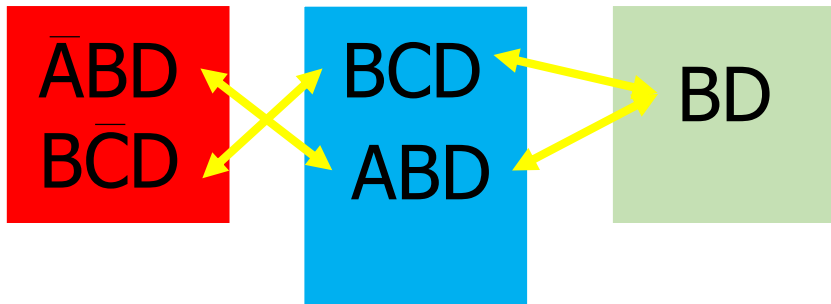
$$\bar{A}\bar{C}D + \bar{A}BC + AB\bar{C} + ACD$$

- Use Quine-McKluskey Method to simplify

$$F = \bar{A}\bar{B}\bar{C}D + \bar{A}BCD + AB\bar{C}D + ABCD$$

	A	B	C	D
$\bar{A}\bar{B}\bar{C}D$	0	1	0	1
$\bar{A}BCD$	0	1	1	1
$AB\bar{C}D$	1	1	0	1
$ABCD$	1	1	1	1

$$F = \bar{A}\bar{B}\bar{C}D + \bar{A}BCD + AB\bar{C}D + ABCD$$



$$F = \bar{A}\bar{B}\bar{C}D + \bar{A}BCD + A\bar{B}\bar{C}D + ABCD$$

	$\bar{A}\bar{B}\bar{C}D$	$\bar{A}BCD$	$A\bar{B}\bar{C}D$	$ABCD$
BD	X	X	X	X

$$F = BD$$

Quick Activity #4

- ▶ Discuss the difference between Karnaugh map and Quine McCluskey methods.
- ▶ Next, simplify the following Boolean expression using both Karnaugh map and Quine McCluskey method

$$\bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}\bar{C}D + \bar{A}\bar{B}C\bar{D} + \bar{A}B\bar{C}\bar{D} + A\bar{B}\bar{C}\bar{D} + ABC\bar{D}$$

NAND and NOR Implementations

- ▶ Another consideration in the implementation of Boolean functions concerns the types of gates used.
- ▶ It is sometimes desirable to implement a Boolean function solely with **NAND** gates or solely with **NOR** gates.
- ▶ Although this may not be the minimum-gate implementation, it has the advantage of regularity, which can simplify the manufacturing process.

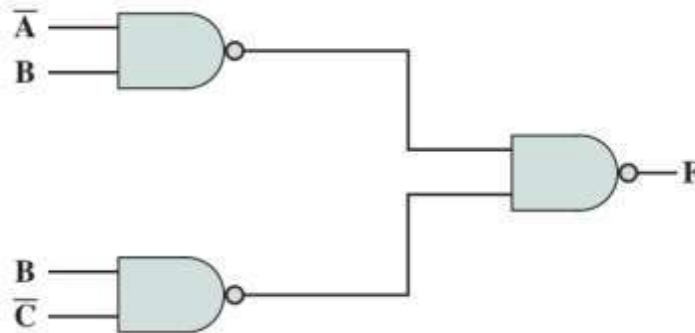
DeMorgan's Theorems	$\overline{A \cdot B} = \overline{A} + \overline{B}$	$\overline{A + B} = \overline{A} \cdot \overline{B}$
---------------------	--	--

$$F = B(\overline{A} + \overline{C})$$

$$F = B(\overline{A} + \overline{C}) = \overline{\overline{B(\overline{A} + \overline{C})}}$$

- Apply DeMorgan's theorem

$$F = \overline{\overline{AB}} \cdot \overline{\overline{BC}}$$



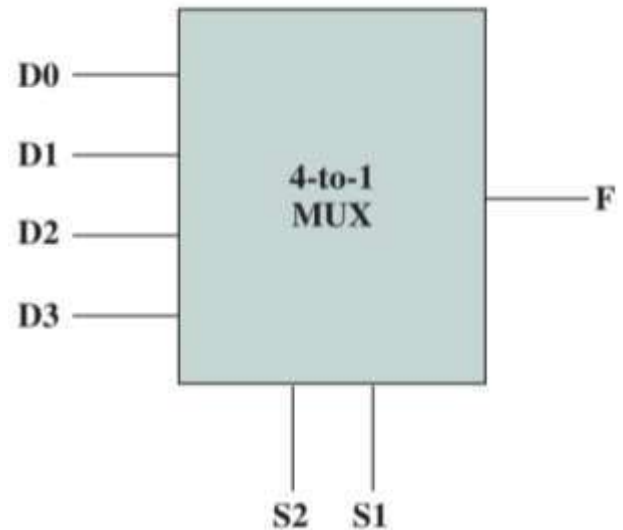
Outline

- ▶ Boolean Algebra
- ▶ Gates
- ▶ Combinational Circuits
 - ▶ Implementation of Boolean Functions
 - ▶ Multiplexers
 - ▶ Decoders
 - ▶ Read-Only Memory
 - ▶ Adders

Multiplexers

- ▶ The multiplexer connects **multiple inputs** to a single output.
- ▶ At any time, one of the inputs is selected to be passed to the output.
- ▶ One of inputs is **selected** to be passed to output
- ▶ An example of 4-to-1 multiplexer
 - ▶ 4 inputs (D0, D1, D2, D3)
 - ▶ 2 selector lines (S1, S2)

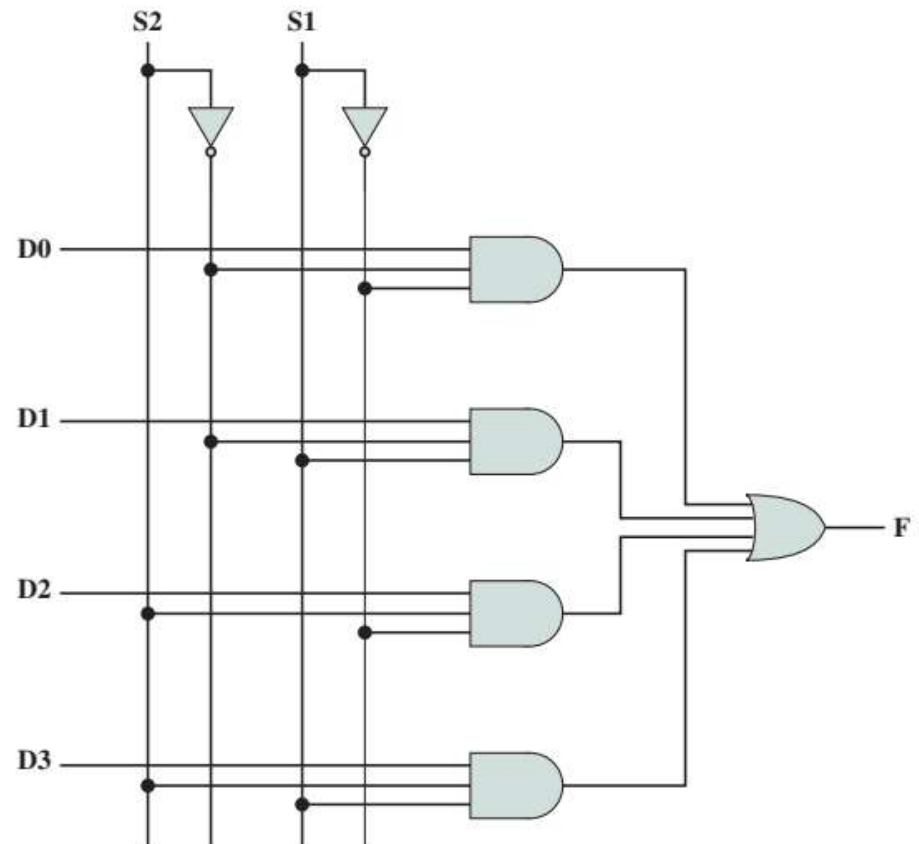
S2	S1	F
0	0	D0
0	1	D1
1	0	D2
1	1	D3



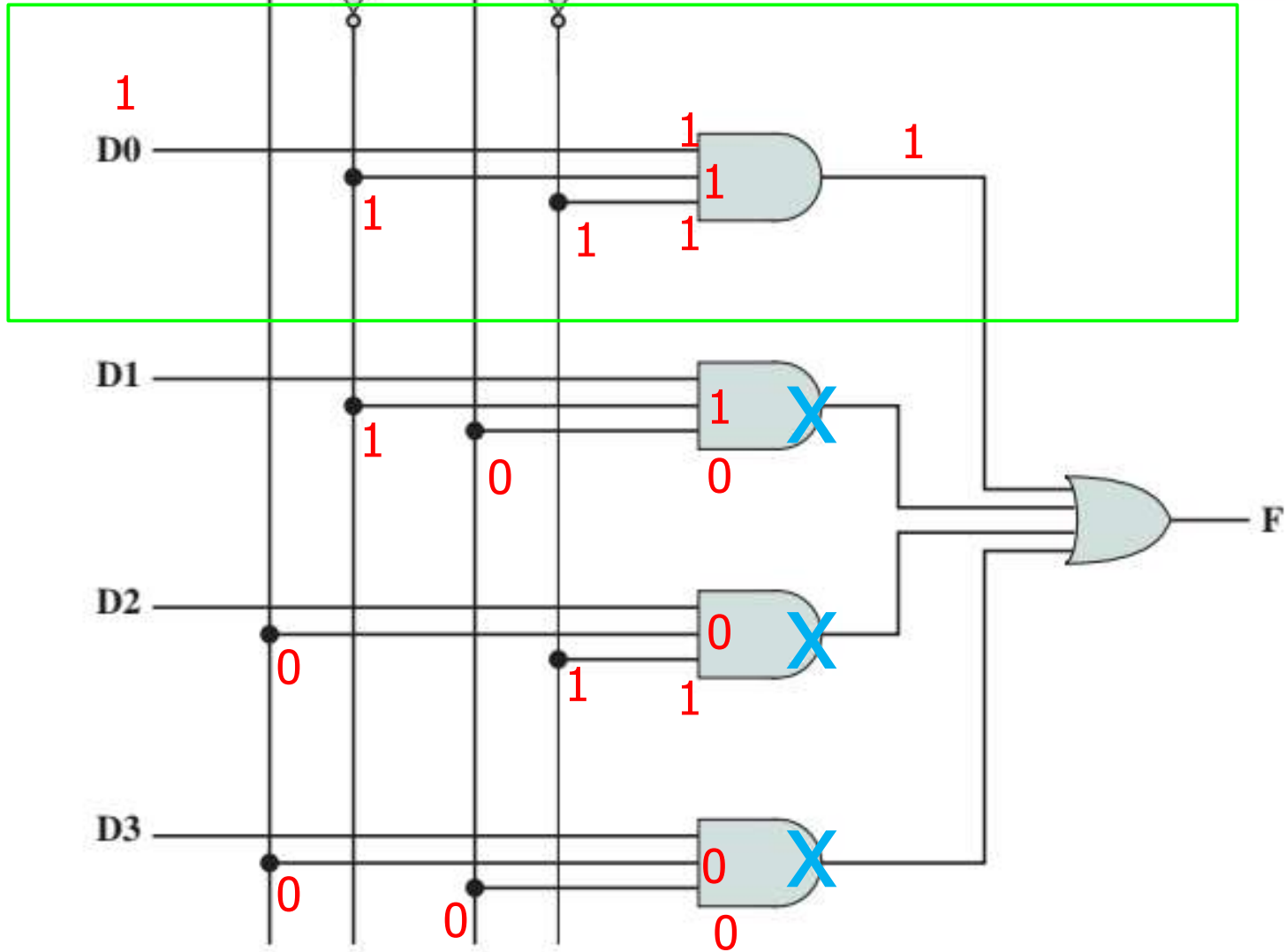
Multiplexers

► Implementation using AND, OR, and NOT gates

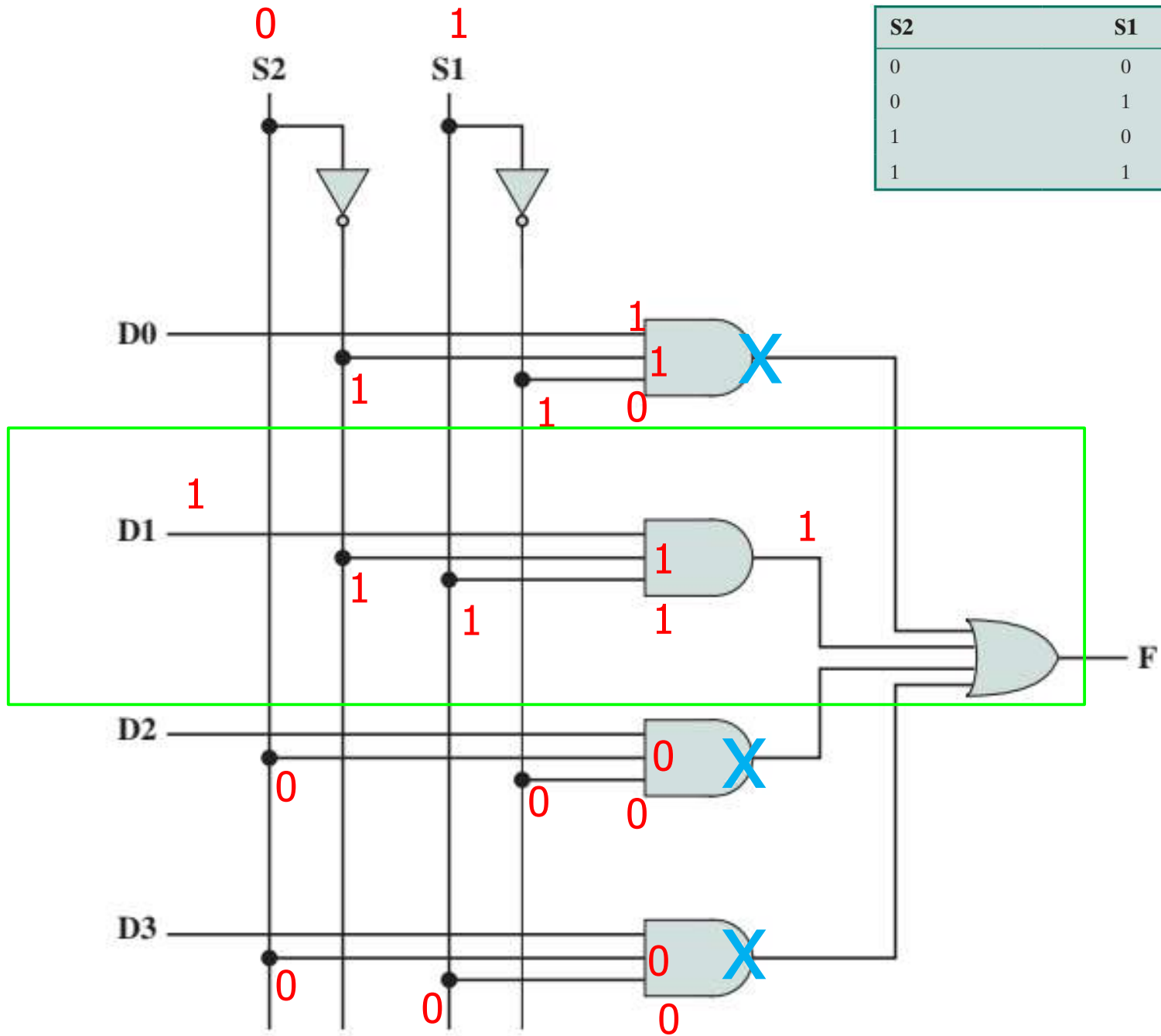
- For any combination of the inputs, **three AND gates will result in a zero**, the **fourth will output the value of the selected lines** (either 0 or 1)
- This can extend to MUX 8-to-1, 16-to-1



S2	S1	F
0	0	D0
0	1	D1
1	0	D2
1	1	D3

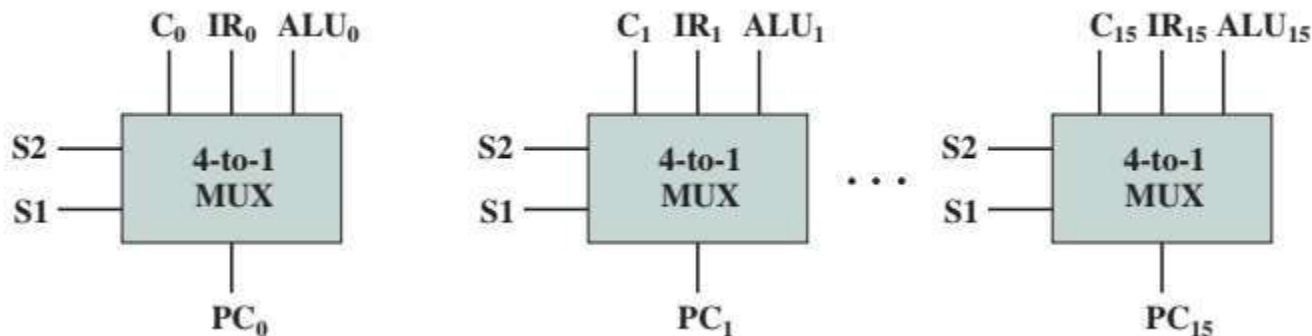


S2	S1	F
0	0	D0
0	1	D1
1	0	D2
1	1	D3

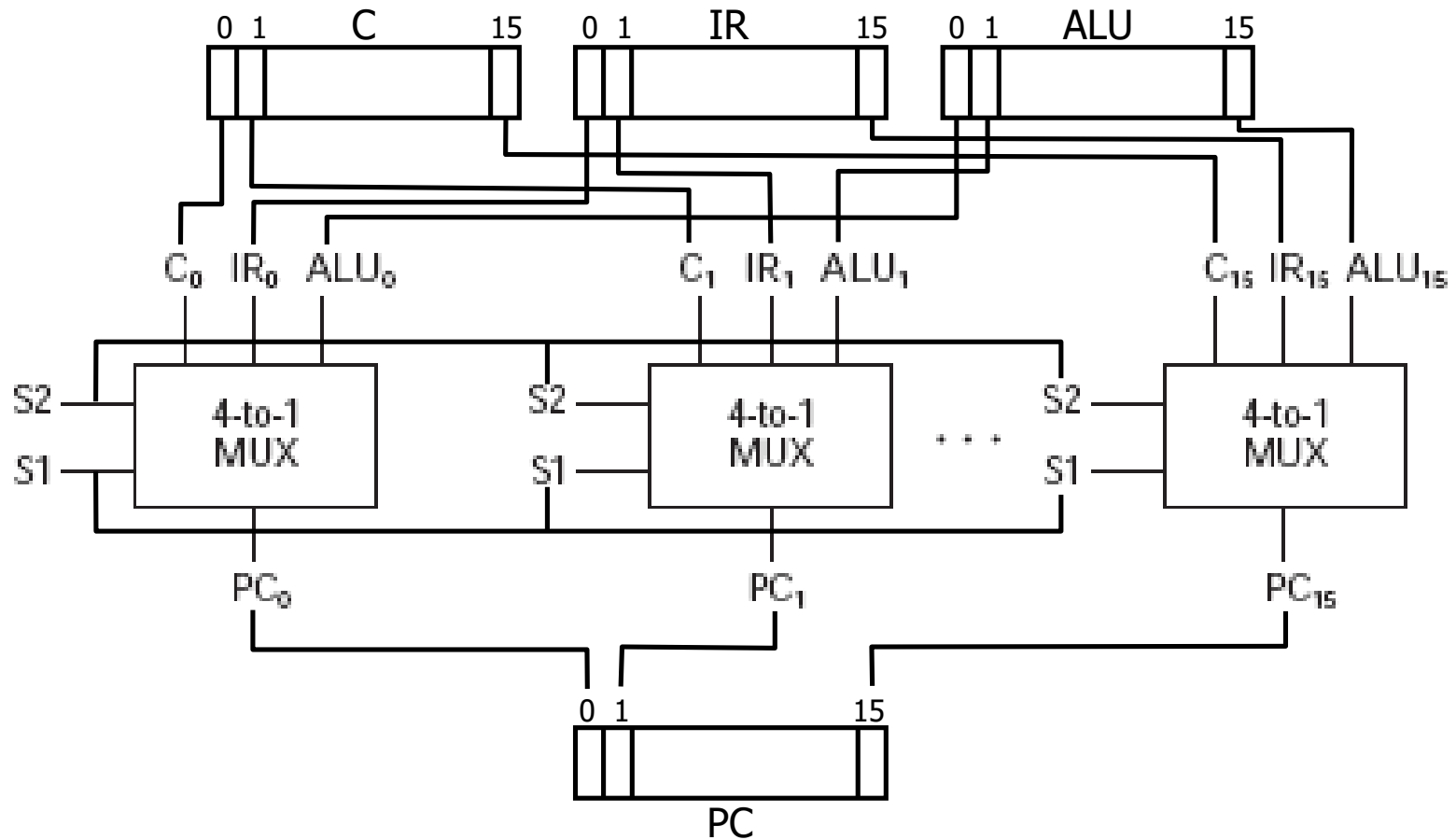


Multiplexers

- ▶ Used to control signal and data **routing**
- ▶ Example: address to be loaded in the Program Counter may come from different sources:
 - ▶ a binary counter if the PC is to be incremented for the next instruction
 - ▶ instruction register (direct branch) if a branch instruction using a direct address has just been executed
 - ▶ output of the ALU (indirect branching) if the branch instruction specifies the address using a displacement mode.

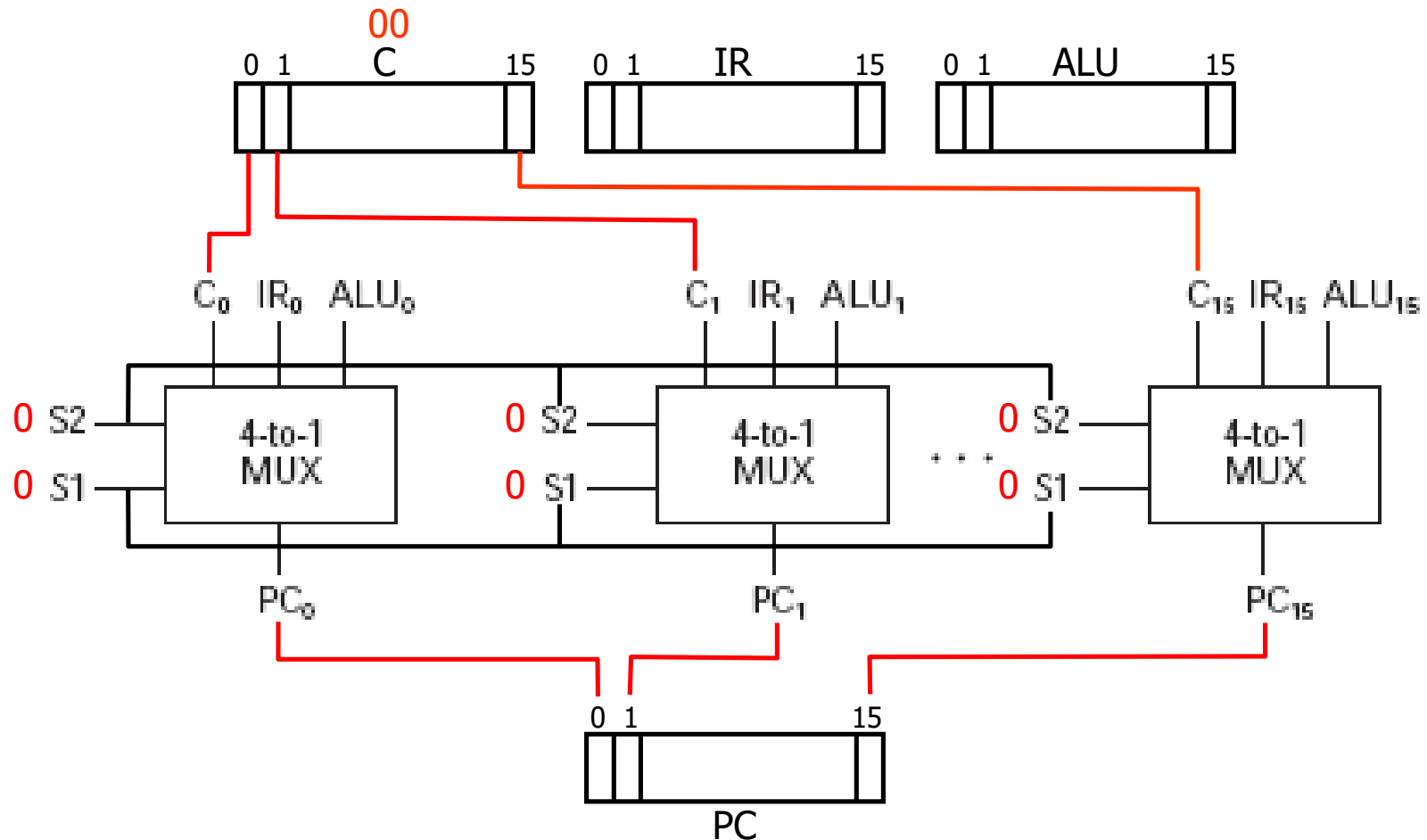


Multiplexers



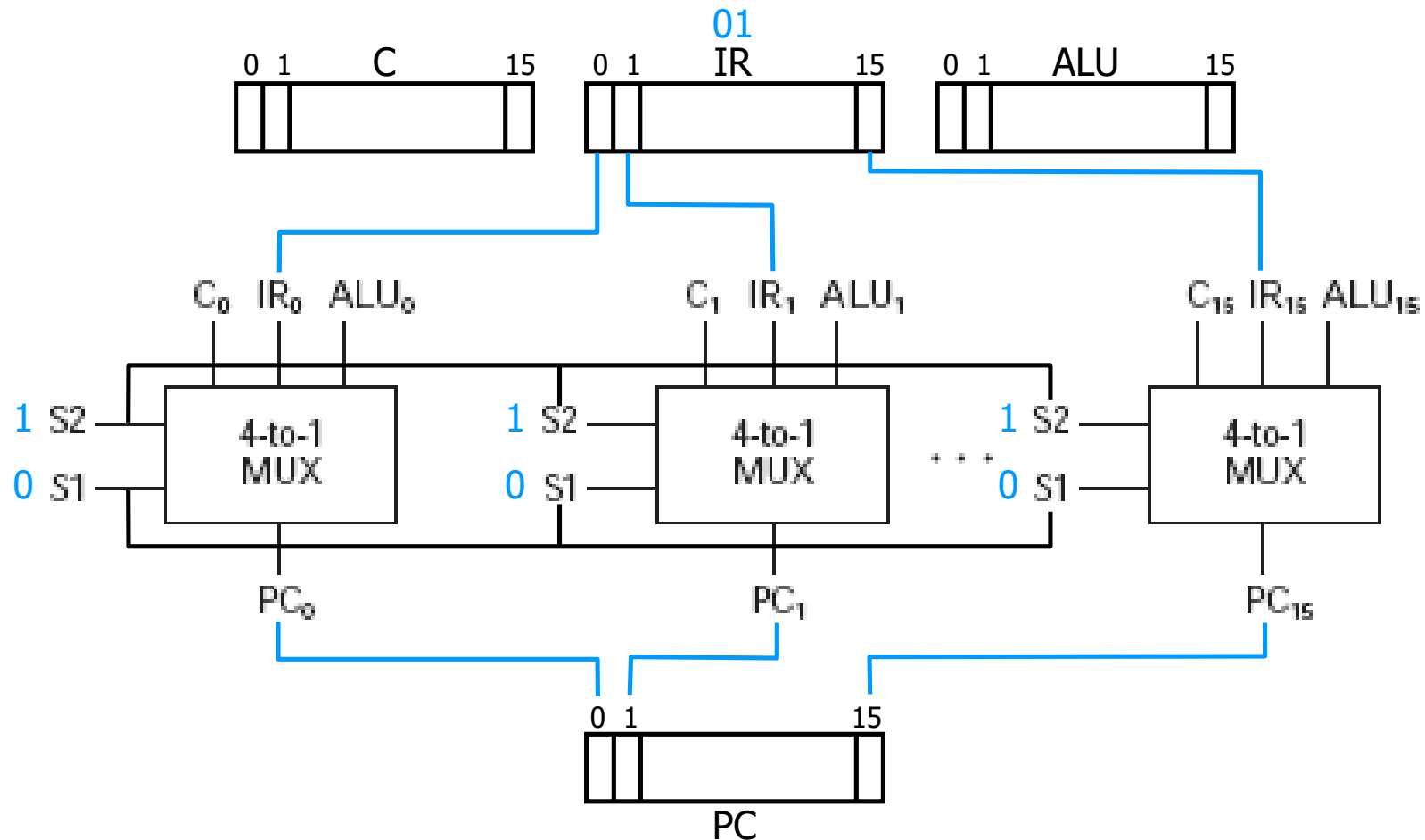
Multiplexers

S2	S1	F
0	0	D0
0	1	D1
1	0	D2
1	1	D3



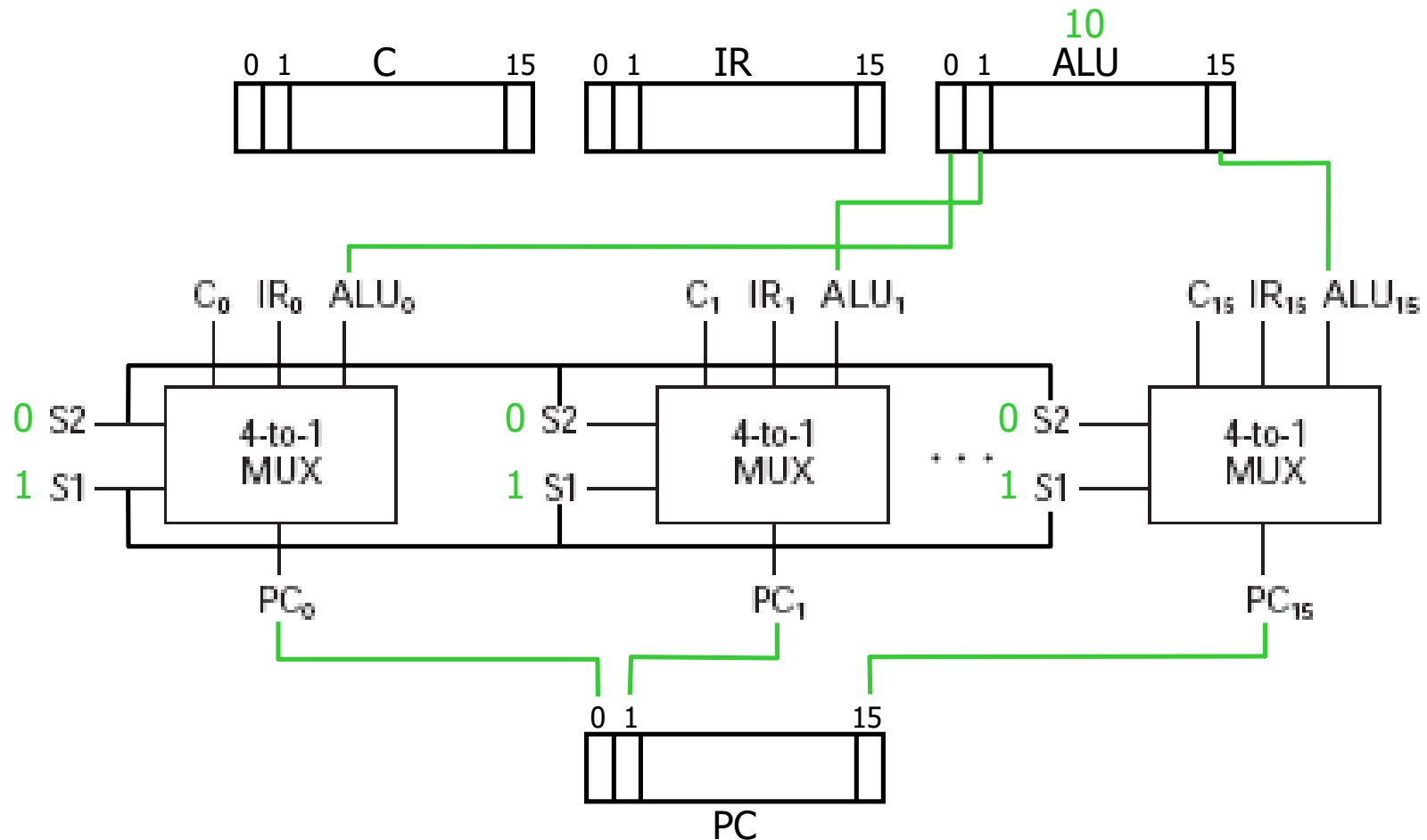
Multiplexers

S2	S1	F
0	0	D0
0	1	D1
1	0	D2
1	1	D3



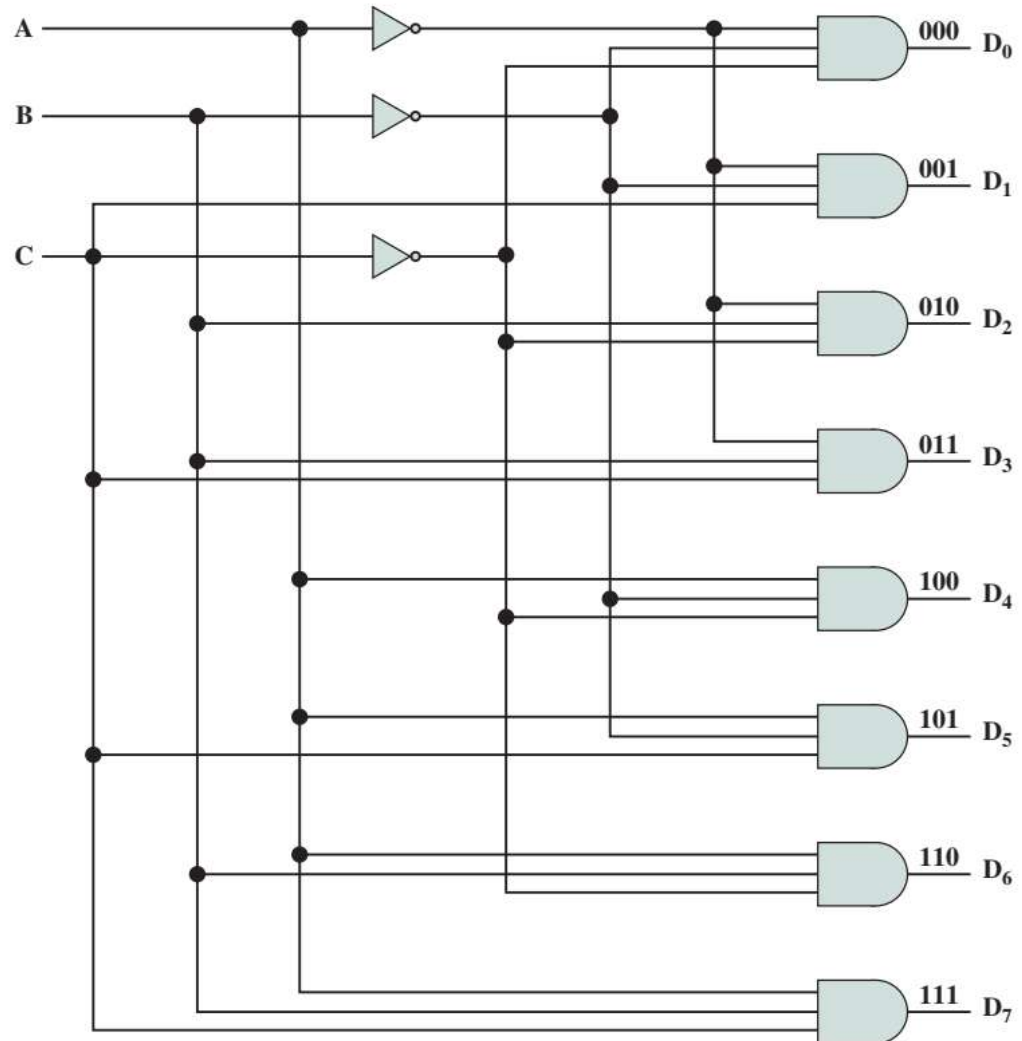
Multiplexers

S2	S1	F
0	0	D0
0	1	D1
1	0	D2
1	1	D3



Decoders

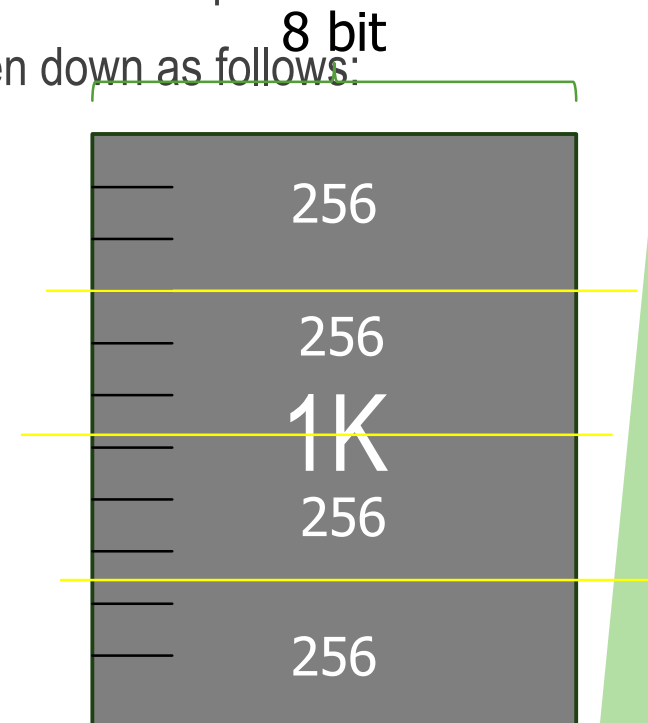
- ▶ A combinational circuit with **a number of output lines**
 - ▶ Only one asserted at any one time
 - ▶ Depends on pattern of input lines
- ▶ **n input lines** to a maximum of **2^n output lines**
- ▶ E.g. 3 inputs, 8 outputs



Decoders

- ▶ Decoders find many uses in digital computers.
- ▶ Example: address decoding
 - ▶ Build a 1K-byte memory using **four** 256 x 8-bit RAM chips
 - ▶ A single unified address space can be broken down as follows:

	Address	Chip
0 - 255	0000–00FF	0
256 - 511	0100–01FF	1
512 - 767	0200–02FF	2
768 - 1024	0300–03FF	3



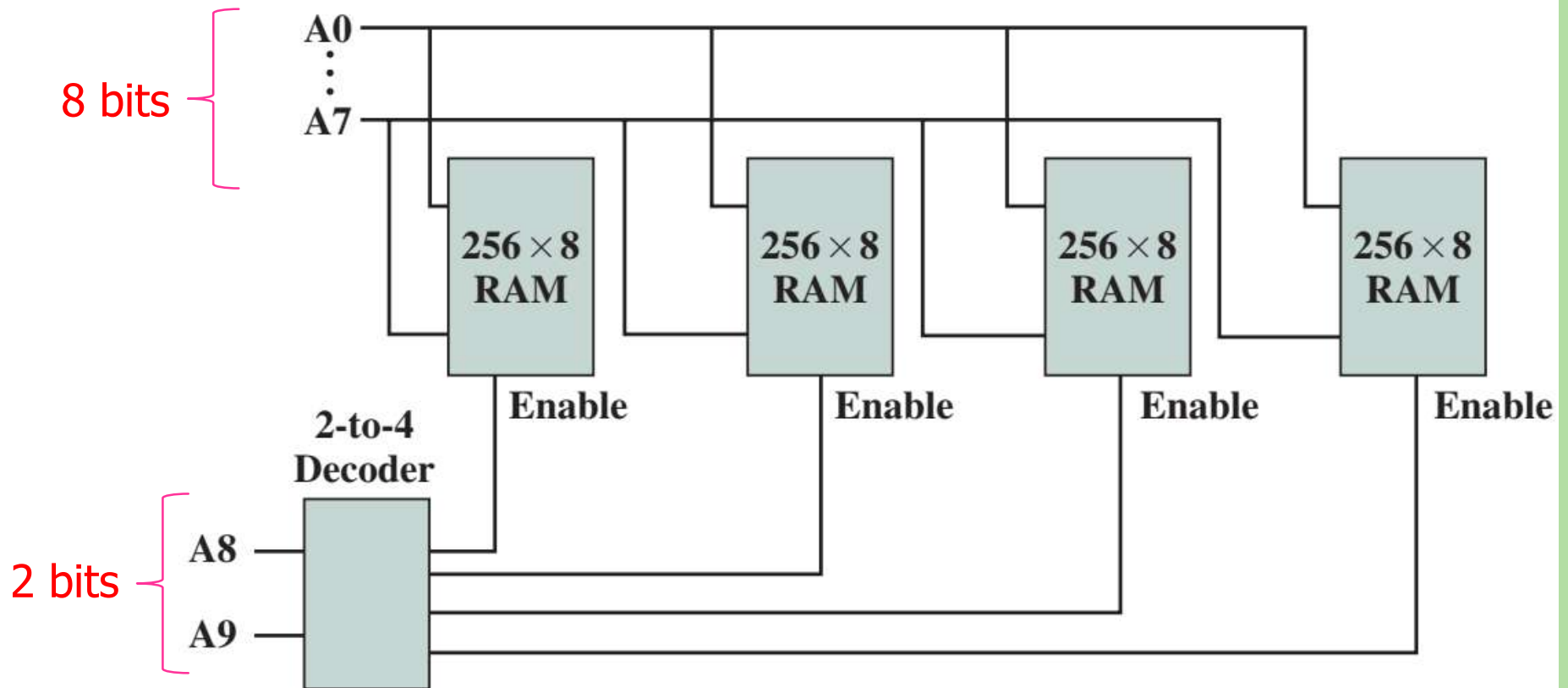
Decoders

- ▶ Example: address decoding
 - ▶ Each chip requires 8 address lines (to choose one of the 2^8 = 256 words)
 - ▶ Higher-order 2 bits used to select one of the four RAM chips
 - ▶ 2-to-4 decoder needed
 - ▶ Total $8+2=10$ bits needed to address 1KB

8 bits – to address each location in the chips

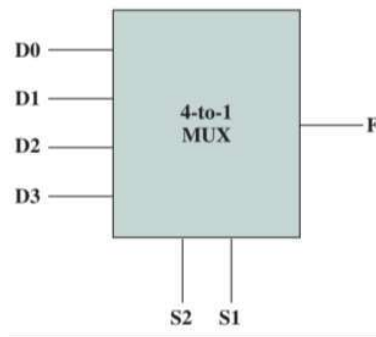
2 bits – to use to select which chips will be utilized

Decoders



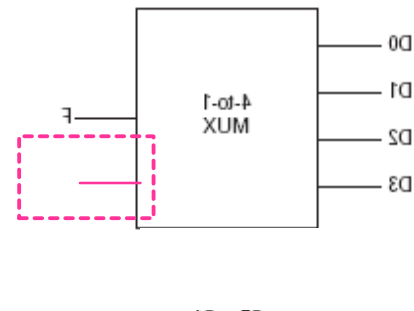
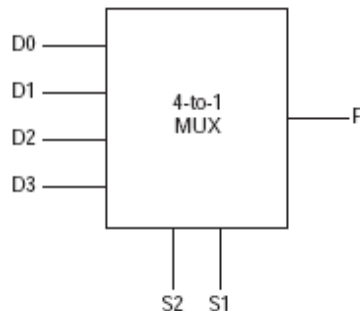
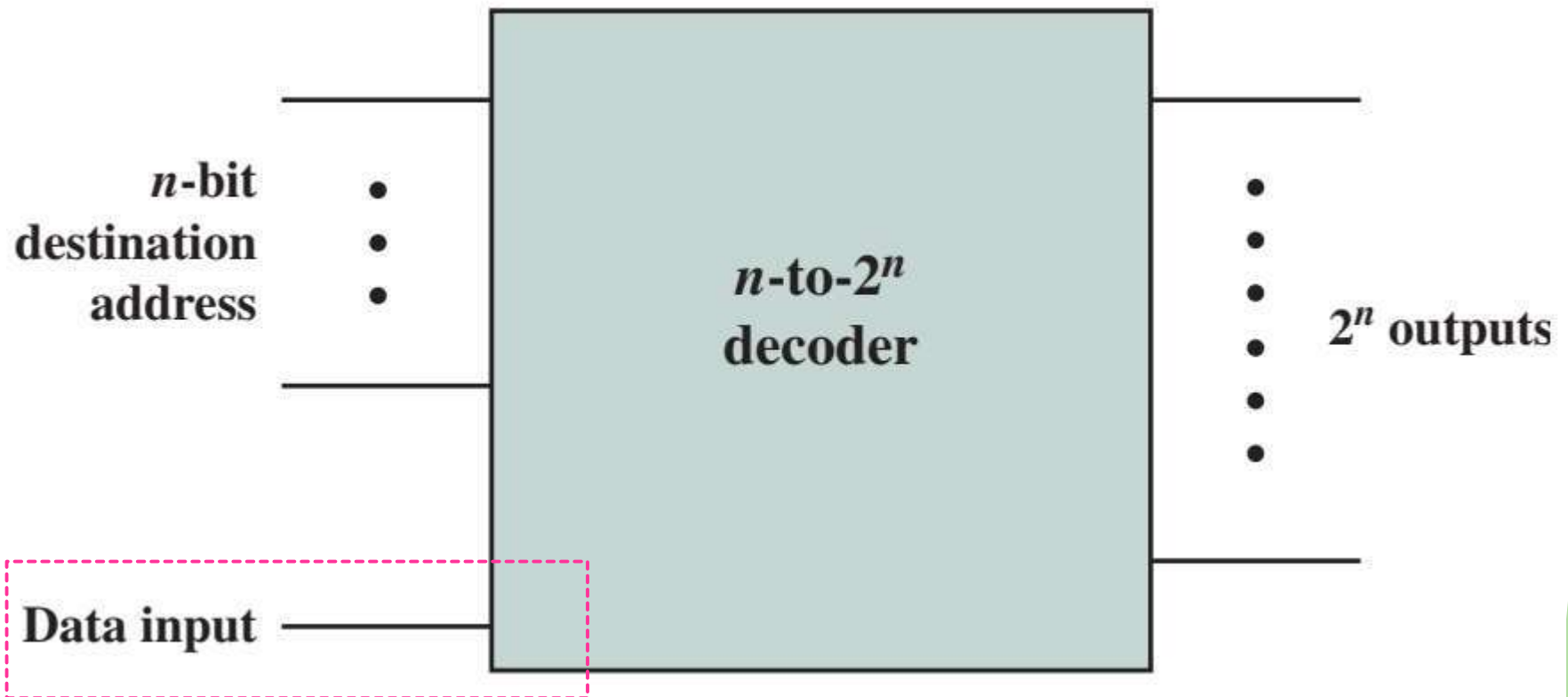
Decoders

- ▶ With additional input line, a decoder can act as a **demultiplexer**
 - ▶ connects **single input** to **one of several outputs**
- ▶ n inputs decoded to produce one of 2^n outputs
- ▶ All 2^n outputs lines are ANDed with a data input line
- ▶ Thus, the **n inputs** act as an address to **select a particular output line**, and the value on the data input line is routed to that output line



Multiplexer is many to one

Decoders



Read-Only Memory (ROM)

- ▶ Combinational circuits are often referred to as “memoryless” circuits, because their output depends only on their current input and no history of prior inputs is retained.
- ▶ Combinational circuits do not have the capability to store information
- ▶ Although memory requires “storing” capability, this is not true for ROM

Read-Only Memory (ROM)

- ▶ ROM performs **only read operation**, the info in ROM is permanent with each address keeping the same input data. This is achieved by combinational circuit.

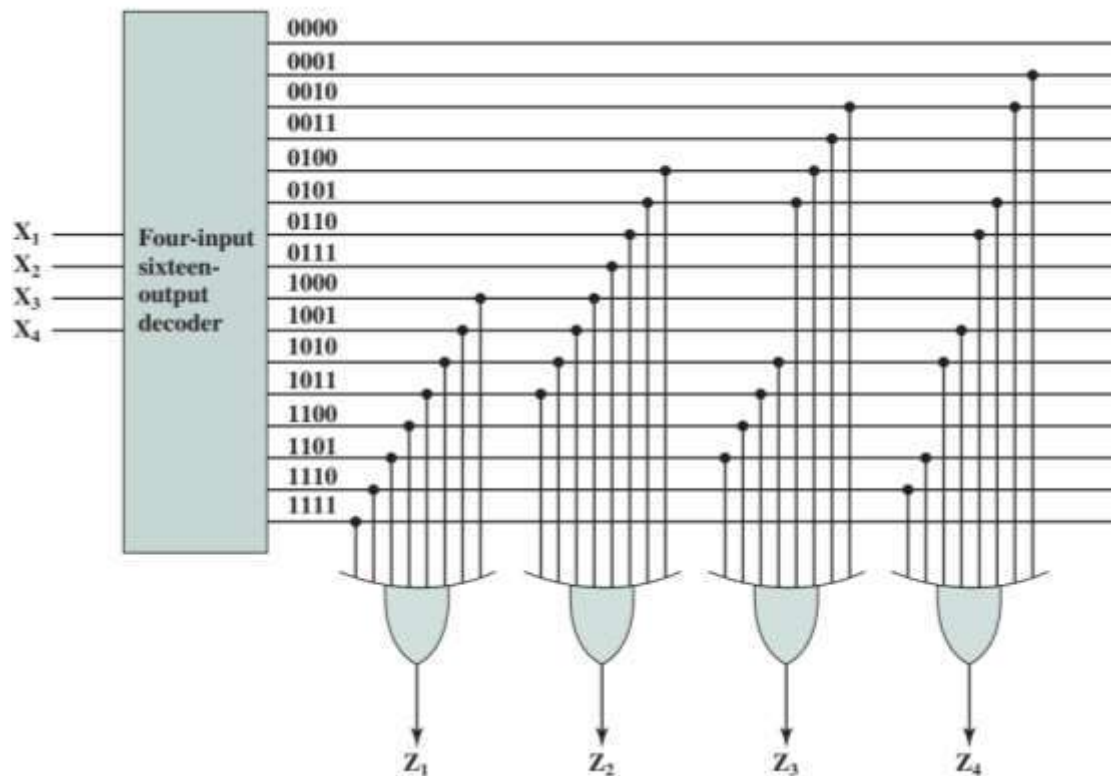
Read-Only Memory (ROM)

- ▶ Can be implemented using a **decoder** and **OR** gates
- ▶ E.g. truth table with 4 inputs and 4 outputs

Input				Output			
X_1	X_2	X_3	X_4	Z_1	Z_2	Z_3	Z_4
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	0
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0
1	1	0	0	1	0	1	0
1	1	0	1	1	0	1	1
1	1	1	0	1	0	0	1
1	1	1	1	1	0	0	0

Read-Only Memory (ROM)

- ▶ 64-bit ROM with 16 words of 4 bits each
 - ▶ 4 inputs specify and address
 - ▶ 4 outputs specify the contents of the location



Adders

- ▶ So far, we have seen how gates implement functions to **route** signals, decode, and store
- ▶ Gates can be used for **arithmetic**
- ▶ Addition
 - ▶ A single bit addition: two 1-bit inputs, a sum with a carry bit

0	0	1	1
+ 0	+ 1	+ 0	+ 1
0	1	1	10

Adders

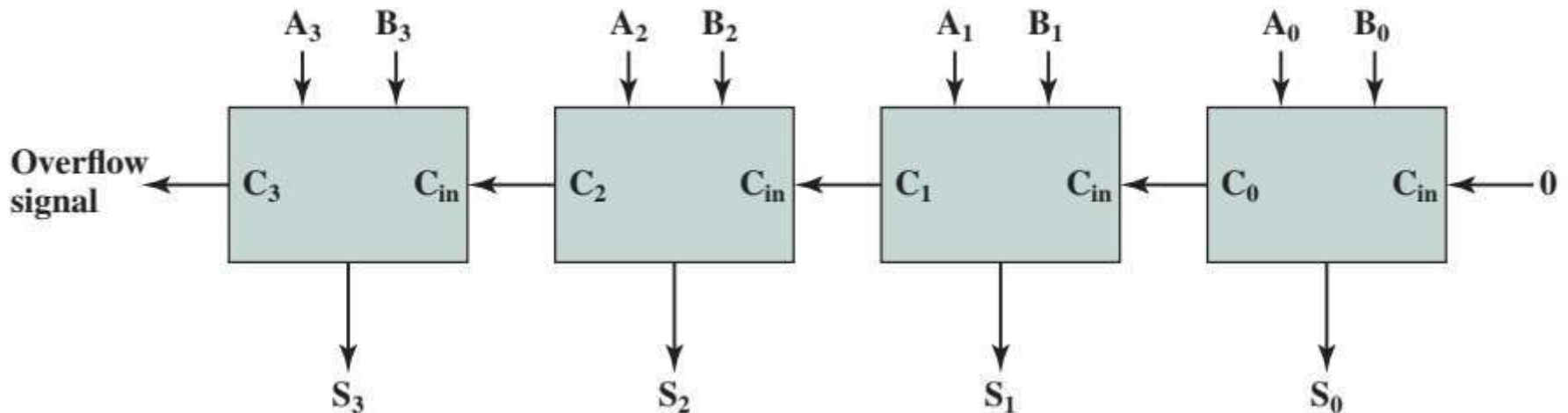
- Logic for **adding two input** bits to produce **1-bit sum** and a **carry bit**
 - Simple for single pair of bits

(a) Single-Bit Addition			
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

0A	0A	1A	1A
+ 0B	+ 1B	+ 0B	+ 1B
Carry 00Sum	Carry 01Sum	Carry 01Sum	Carry 10Sum

Adders

- ▶ Adding two n -bit numbers require n sets adders
 - ▶ Carry from one adder is provided as **input** to the next adder
 - ▶ Each adder need 3 inputs
 - ▶ A carry from the lower-order adder apart from the 2 usual inputs



Adder

(b) Addition with Carry Input					
C _{in}	A	B	Sum	C _{out}	
0	0	0	0	0	
0	0	1	1	0	
0	1	0	1	0	
*	0	1	0	1	
	1	0	0	1	
*	1	0	1	0	
*	1	1	0	1	
	1	1	1	1	

$$\text{Sum} = \bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}\bar{C} + A\bar{B}C + A\bar{B}C + ABC$$

$$\text{Carry} = A\bar{B}\bar{C} + \bar{A}B\bar{C} + A\bar{B}C + ABC$$

$$\text{Carry} = A\bar{B}\bar{C} + \bar{A}B\bar{C} + A\bar{B}C + ABC + ABC + ABC$$

$$\text{Carry} = AB(\bar{C}+C) + AC(\bar{B}+B) + BC(\bar{A}+A)$$

$$\text{Carry} = AB + AC + BC$$

$$\text{Sum} = \bar{A}B\bar{C} + A\bar{B}\bar{C} + \bar{A}\bar{B}C + ABC \quad (\text{X simplified})$$

$$\text{Carry} = \underline{AB\bar{C}} + \bar{A}BC + \underline{A\bar{B}C} + ABC$$

$$= AB(\bar{C} + C) + \bar{A}BC + A\bar{B}C$$

$$= \underline{AB} + \bar{A}BC + \underline{A\bar{B}C} \quad (\text{based on algebra simplification})$$

$$= AC(B + \bar{B}) = \underline{AC}$$

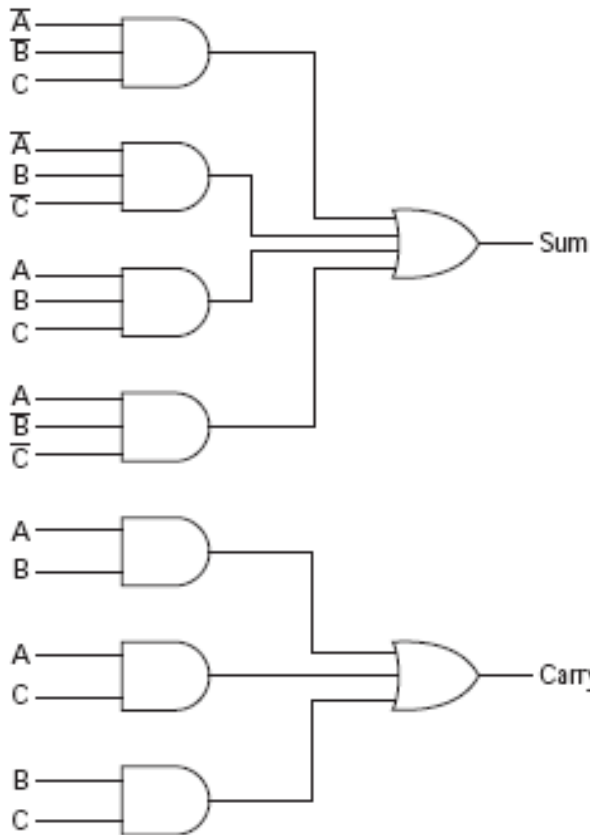
$$= AB + \bar{A}BC + AC$$

$$= \underline{AB} + AC + \underline{\bar{A}BC}$$

$$= BC(A + \bar{A}) = \underline{BC}$$

$$= AB + AC + BC$$

Adders



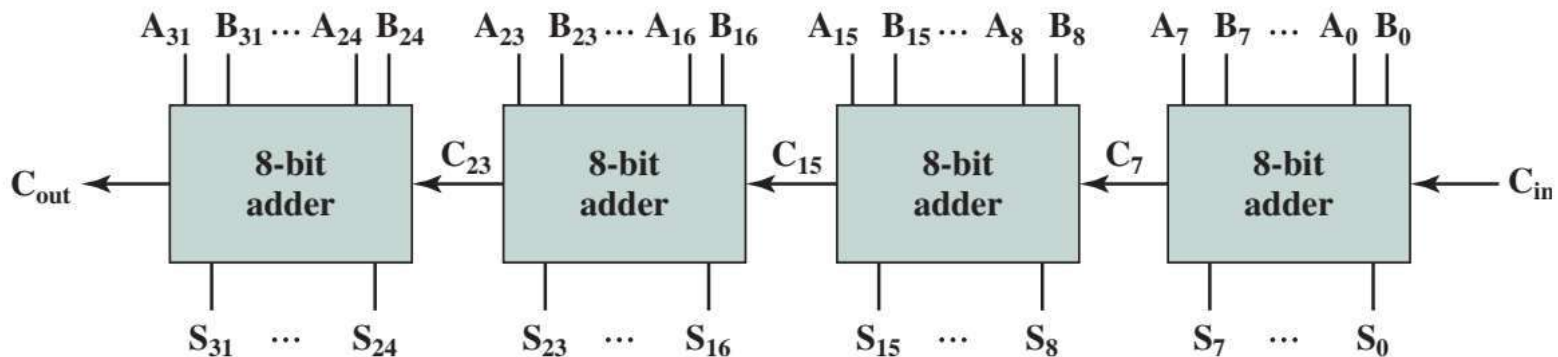
$$\text{Sum} = \bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}C + \bar{A}B\bar{C} + ABC$$

$$\text{Carry} = AB + AC + BC$$

Adders

► Multiple-bit adder

- Output of each adder depends on **carry from previous adder**
- **Increasing delay** from least significant to most significant bit
- Each single-bit adder has **gate delay** which accumulates
- Delay can be high for very large adders



Adders

- ▶ If carry could be determined without going through all the stages, each single-bit adder could function **independently**
 - ▶ Delay would not accumulate
 - ▶ Carry lookahead
- ▶ In 4-bit adder:

$$\begin{array}{r} C_0 \\ A_0 \\ B_0 \\ \hline S_0 \end{array}$$

$$\begin{array}{r} C_1 \quad C_0 \\ A_1 \quad A_0 \\ B_1 \quad B_0 \\ \hline S_1 \quad S_0 \end{array}$$

$$\begin{array}{r} C_2 \quad C_1 \quad C_0 \\ A_2 \quad A_1 \quad A_0 \\ B_2 \quad B_1 \quad B_0 \\ \hline S_2 \quad S_1 \quad S_0 \end{array}$$

$$C_0 = A_0 B_0$$

$$C_1 = A_1 B_1 + A_1 \overline{A_0} B_0 + B_1 \overline{A_0} B_0$$

$$C_2 = A_2 B_2 + A_2 \overline{A_1} B_1 + A_2 \overline{A_1} \overline{A_0} B_0 + A_2 \overline{B_1} \overline{A_0} B_0 + B_2 \overline{A_1} B_1 + B_2 \overline{A_1} \overline{A_0} B_0 + B_2 \overline{B_1} \overline{A_0} B_0$$

A person is sitting on a grassy bank next to a body of water. In the background, there are trees and a building. A bicycle is parked on the grass to the right of the person. The scene is captured in a low-angle shot, emphasizing the grass in the foreground.

Reflection...

From the lecture today,
I learnt...

Link on eLearn / Chat

Attendance

Computer Organisation

Topic 4

Digital Logic (Part 2)

Outline

- Sequential Circuits
 - Flip-Flops
 - Registers
 - Counters
- Programmable Logic Devices
 - Programmable Logic Array
 - Field-Programmable Gate Array

Introduction

- Combinational circuits
 - Implement **functions** of a computer
 - Outputs depend entirely on the present input
 - **No memory** (except for ROM)

Introduction

- **Storage** elements
 - Devices that are capable of storing binary information (which defines the state of the sequential circuit)

Sequential Circuit

- Current output depends on the current input and the current state of the circuit
- Consists of a combinational circuit to which storage elements are connected to form a **feedback path**

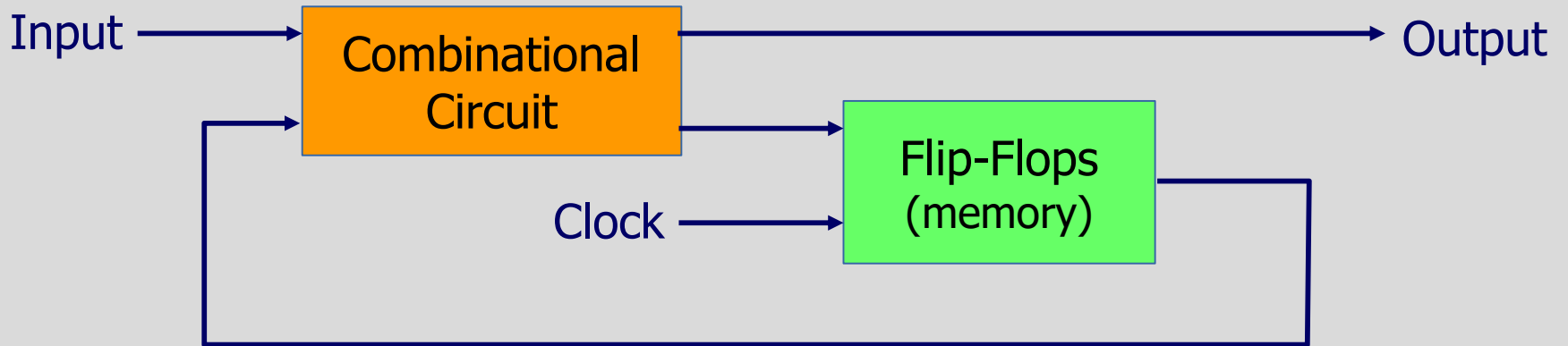
Flip-Flops

- **Simplest** sequential circuit
- It has the capability to **"store"** **information** (1-bit memory)

Flip-Flops

- A **bistable** device
 - Exists in one of two states
 - In the absence of the input remains in that state
- With **two outputs**, Q & $\overline{Q}$, which are the complement of each other

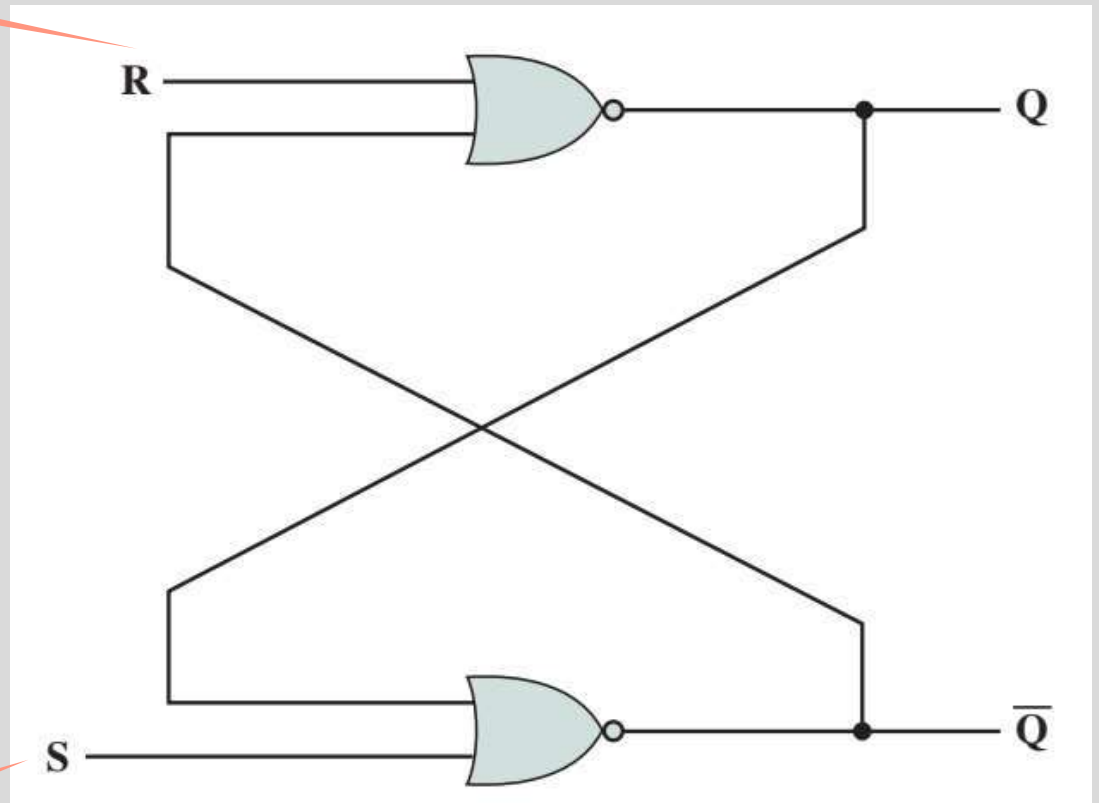
Flip-Flops



S-R Latch

- 2 inputs
 - S (Set) and R (Reset)
- 2 outputs
 - Q and $\bar{Q}$
- 2 NOR gates connected in a feedback manner

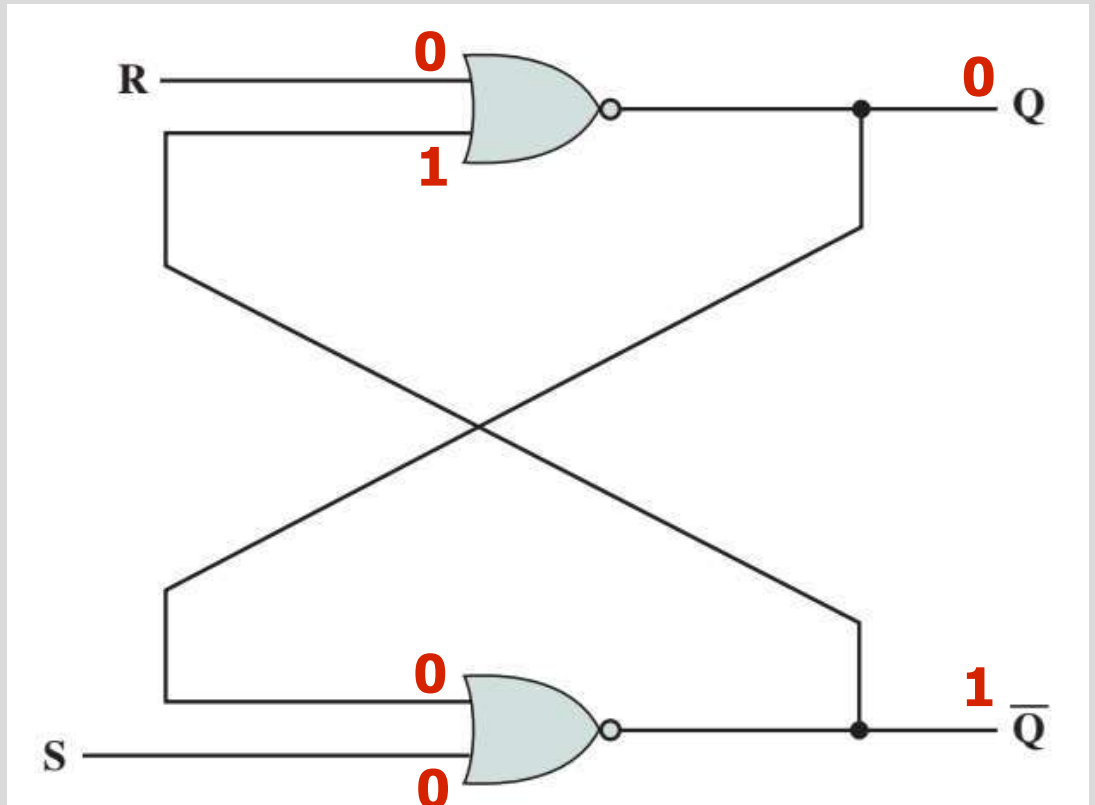
Reset = 0



Set = 1

S-R Latch

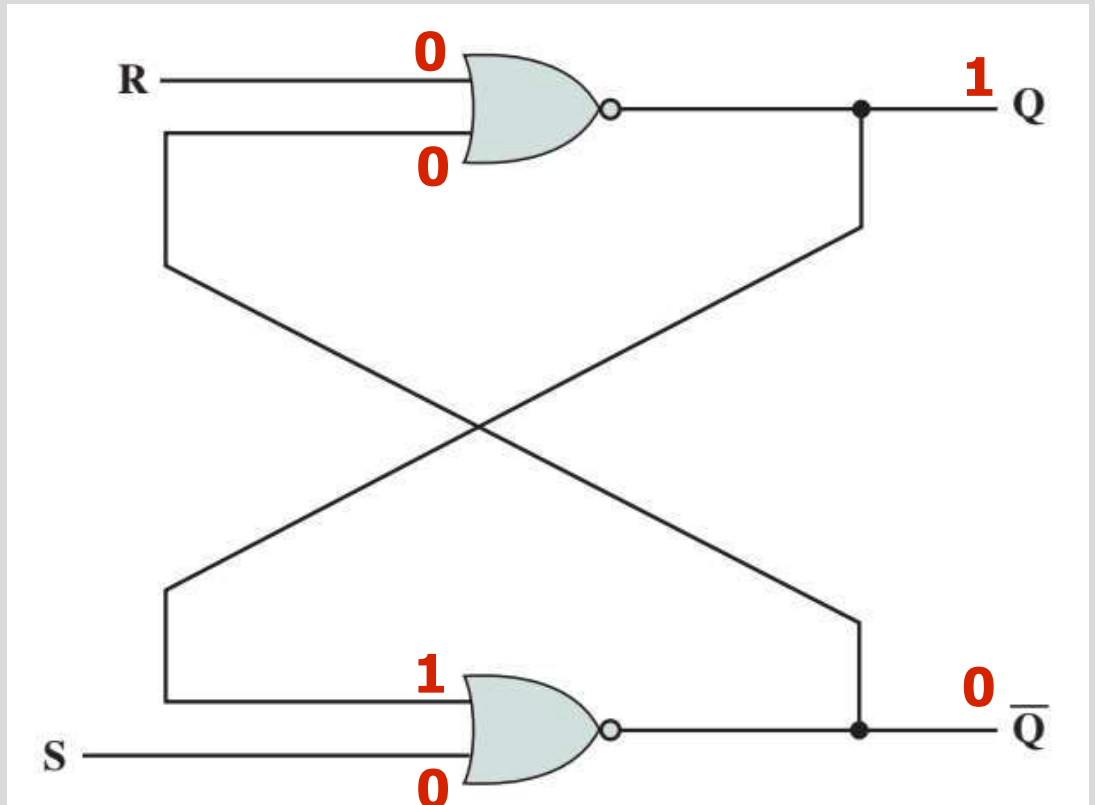
- $S = 0, R = 0, Q = 0$
 - Inputs to lower NOR gate are $Q = 0, S = 0$



Flip-flops remains stable in this state

S-R Latch

- $S = 0, R = 0, Q = 1$
 - Inputs to lower NOR gate are $Q = 1, S = 0$



Flip-flops remains stable in this state

Setting the S-R Latch

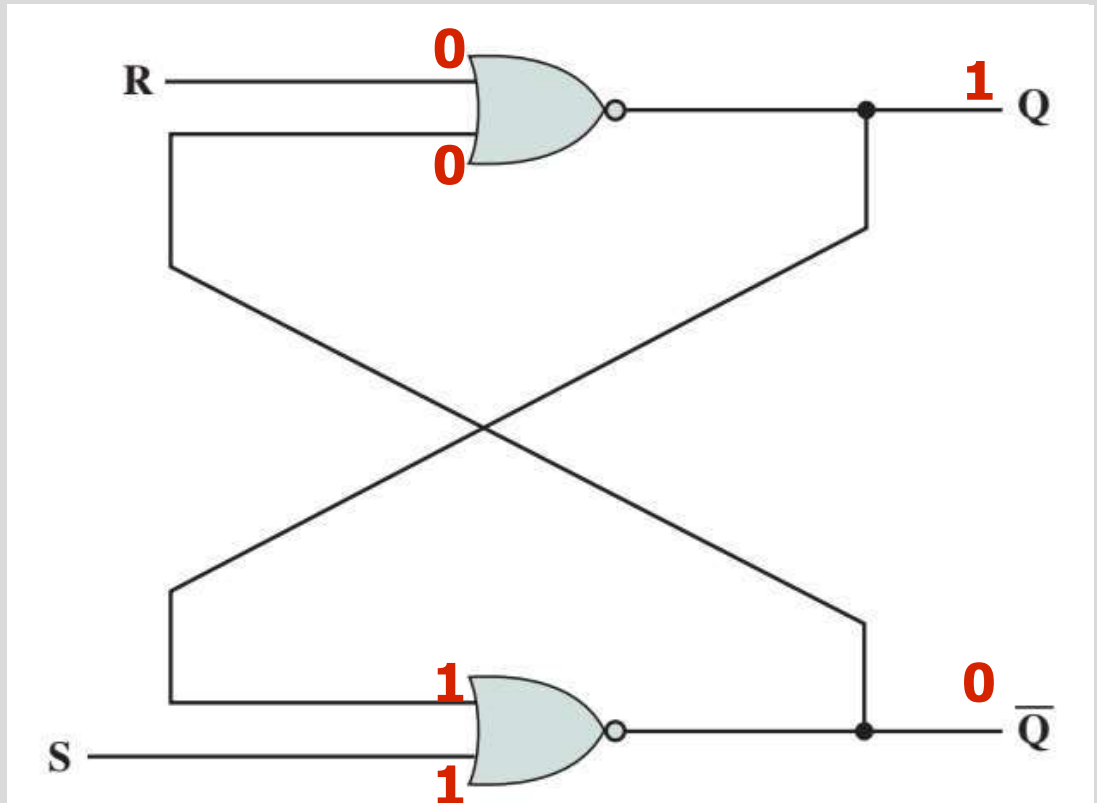
S-R Latch

Let's say initially

- $S=0, R=0, Q=0, \bar{Q}=1$

Then $S=1$

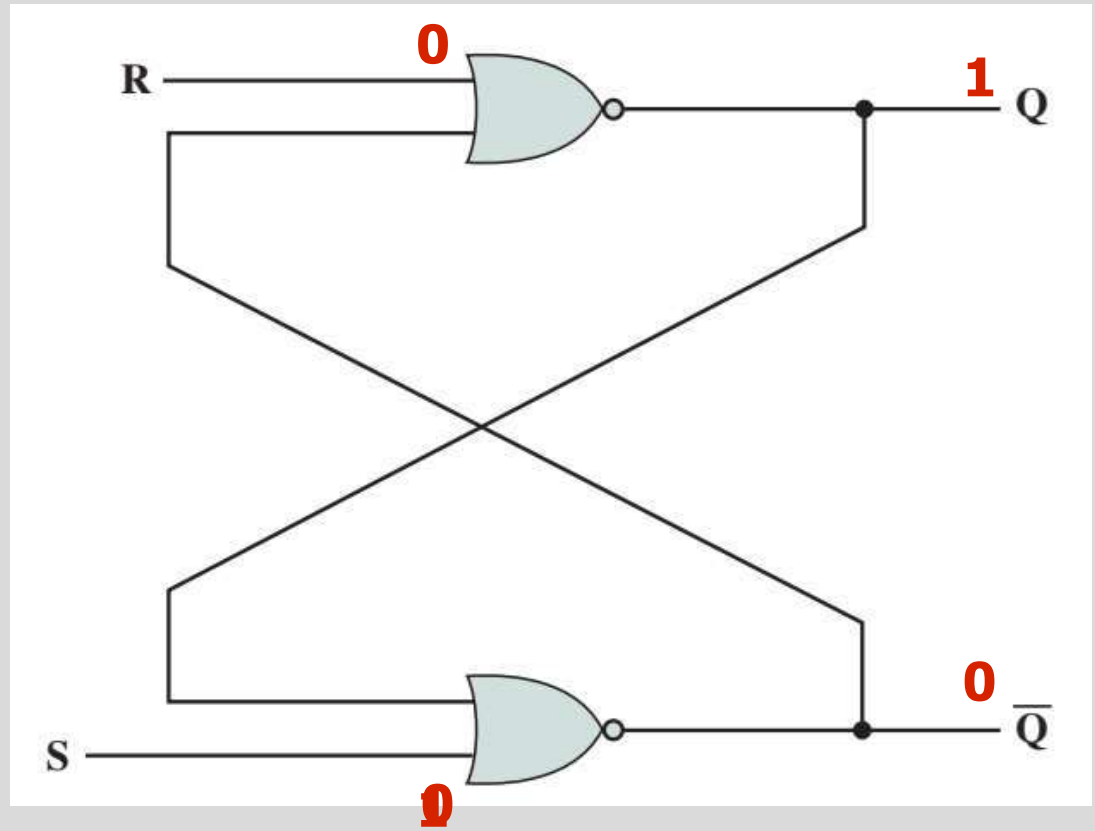
- ✓ Inputs to lower NOR gate are $S=1, Q=0$
- ✓ After Δt , output of lower NOR gate will be $\bar{Q}=0$
- ✓ Inputs to the upper NOR gate become $R=0, \bar{Q}=0$
- ✓ After another Δt , output $Q=1$



S-R Latch

After that, if $S=0$

- Inputs to lower NOR gate are $S=0$, $Q=1$
- After Δt , output of lower NOR gate will be $Q=0$
- Inputs to the upper NOR gate become $R=0$, $Q=0$
- After another Δt , output $Q=1$



Reset the S-R Latch

S-R Latch

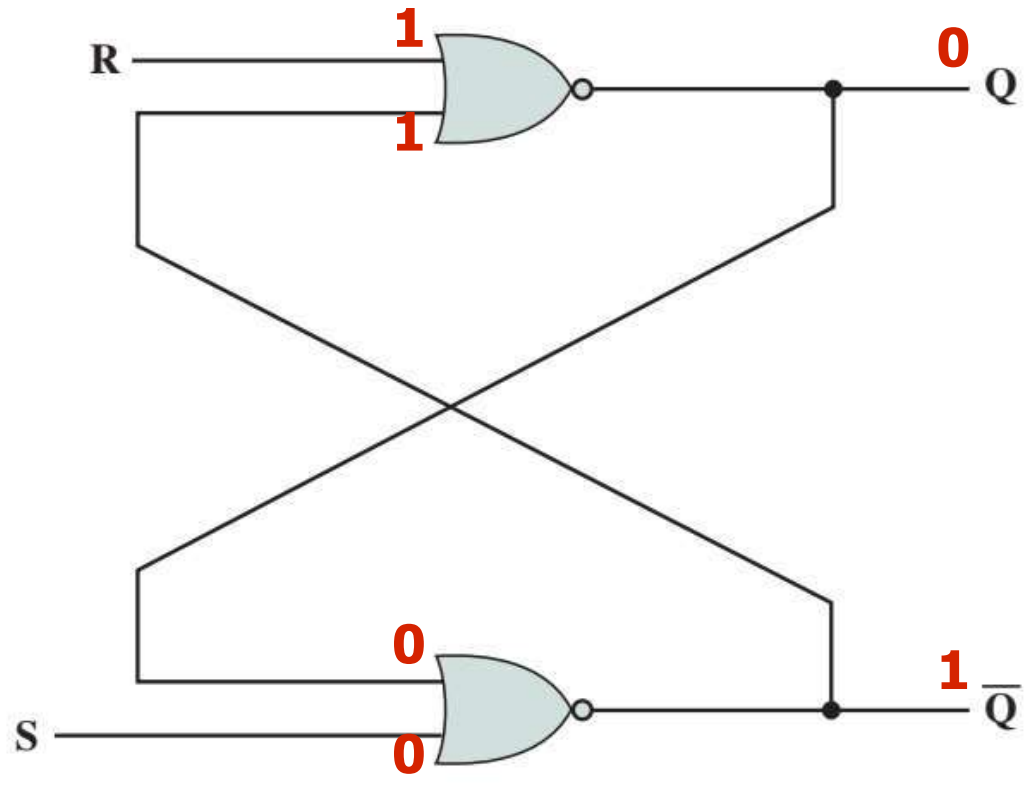
Initially $S=0$, $R=0$, $Q=1$, $\bar{Q}=0$

$R=1$

- Inputs to upper NOR gate are $R=1$, $Q=0$
- After Δt , output of upper NOR gate will be $Q=0$
- Inputs to the lower NOR gate become $S=0$, $Q=0$
- After another Δt , output $\bar{Q}=1$

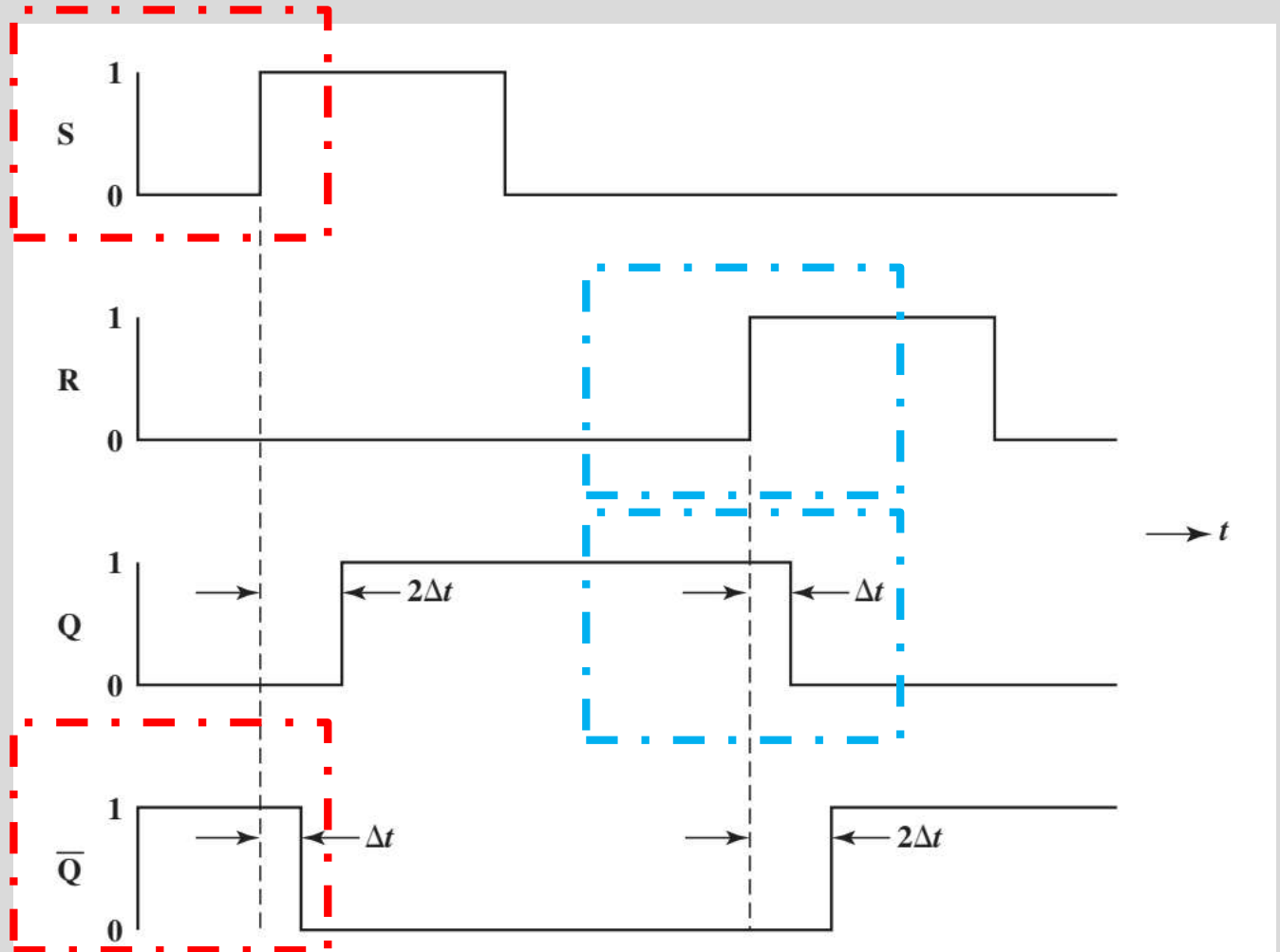
• After that, if $R=0$

- $Q=0$, $\bar{Q}=1$



S-R Latch

NOR S-R Latch Timing Diagram



S-R Latch

- Summary
 - S writes a 1 to Q
 - R writes a 0 to Q
 - S=1 and R=1 at the same time are not allowed
 - Inconsistent output

(a) Characteristic Table		
Current Inputs	Current State	Next State
SR	Q_n	Q_{n+1}
00	0	0
00	1	1
01	0	0
01	1	0
10	0	1
10	1	1
11	0	—
11	1	—

(b) Simplified Characteristic Table		
S	R	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	—

S-R Latch

(c) Response to Series of Inputs

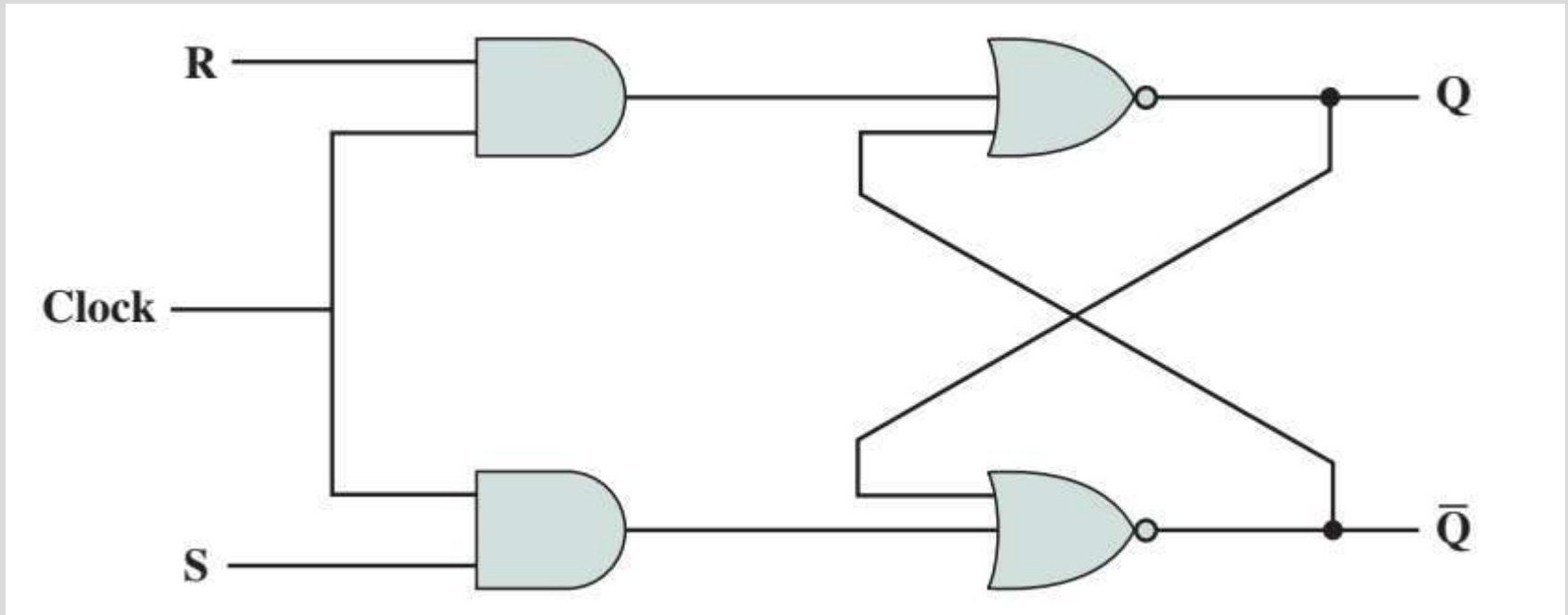
t	0	1	2	3	4	5	6	7	8	9
S	1	0	0	0	0	0	0	0	1	0
R	0	0	0	1	0	0	1	0	0	0
Q_{n+1}	1	1	1	0	0	0	0	0	1	1

Clocked S-R Flip-Flop

- S-R latch
 - Output changes in **response** to a change in input
 - Brief time **delay**
 - **Asynchronous**

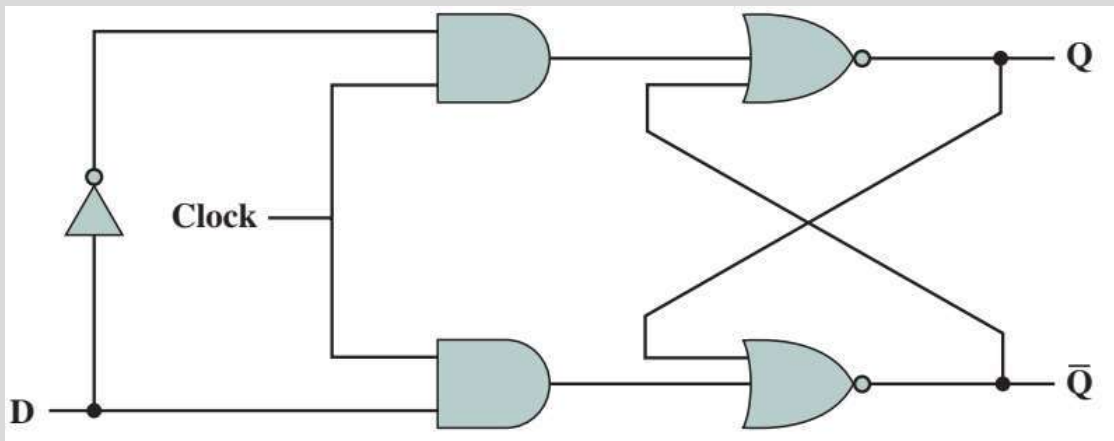
Clocked S-R Flip-Flop

- Events in digital computer are **synchronised** according to clock pulse
 - Clocked S-R flip-flop



D Flip-Flop

- A single input ensures that **$S=R=1$ never occurs**
 - Inverter ensures (non-clock) inputs to the AND gates are opposite
- Also called **data** flip-flop



D	Q_{n+1}
0	0
1	1

Reset

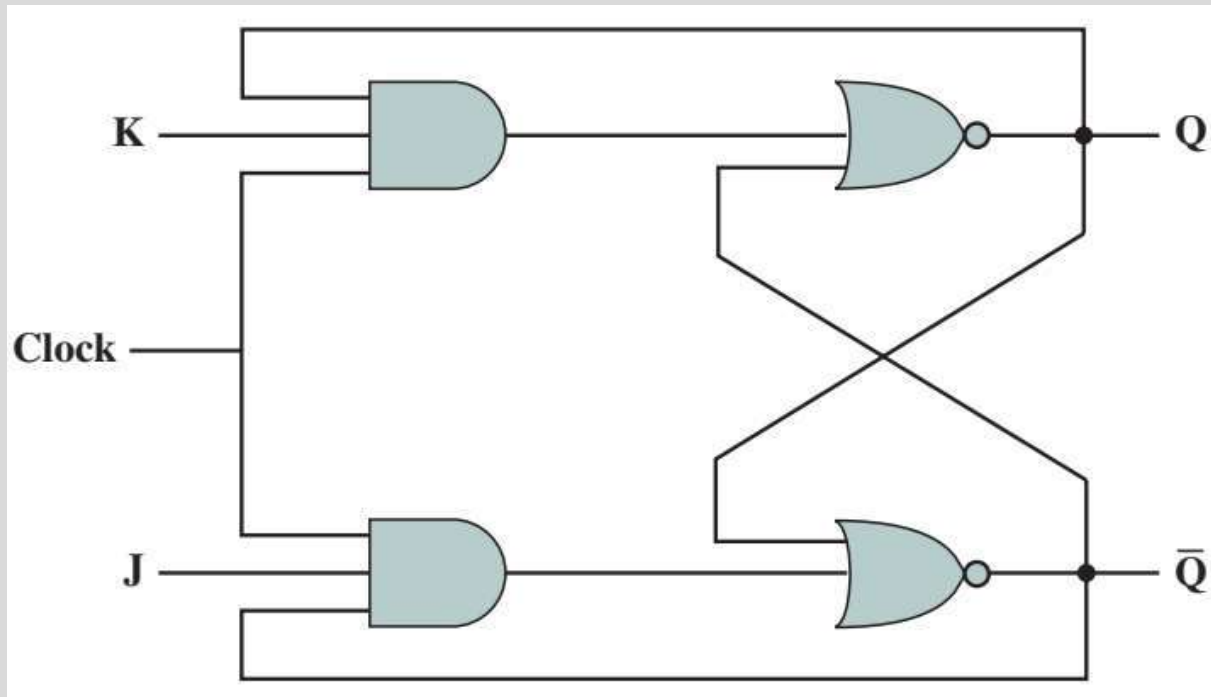
Set

J-K Flip-Flop

- Similar to S-R flip-flop
- **All combinations** of input values are **valid**

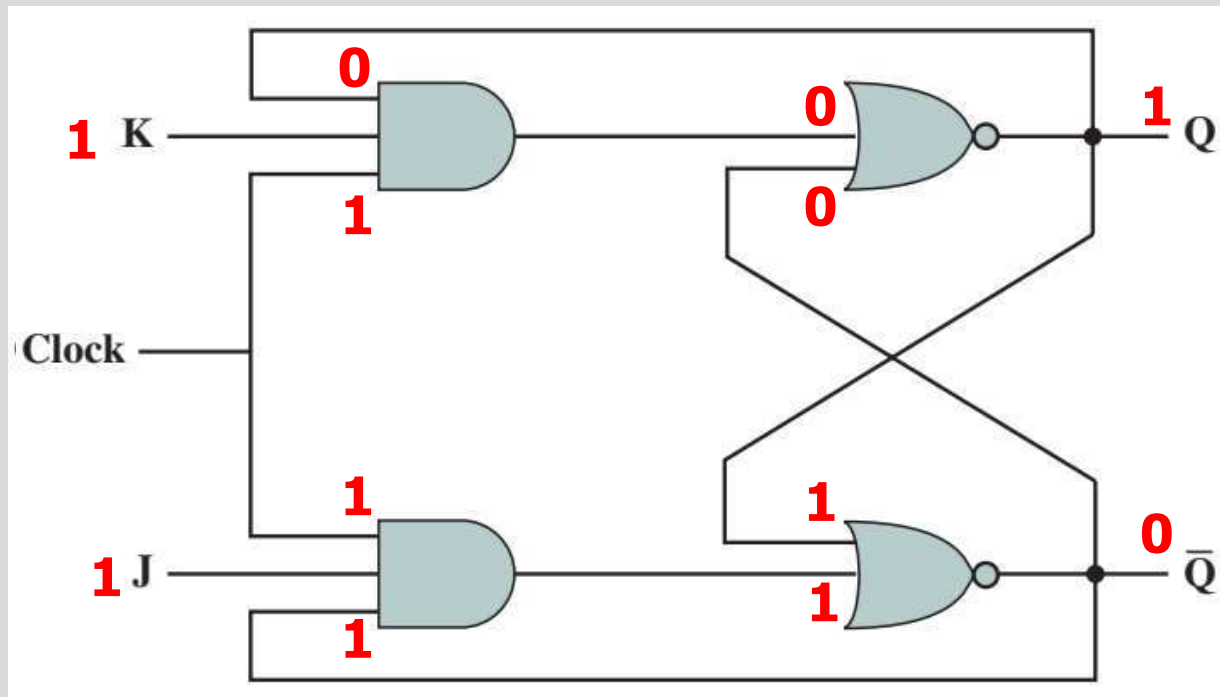
J-K Flip-Flop

- When $J=K=1$ it acts as a **toggle** function, the output is a **reverse** of the present state

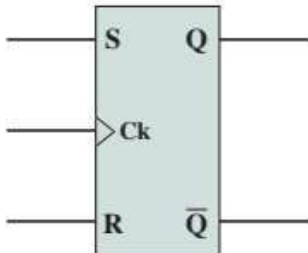
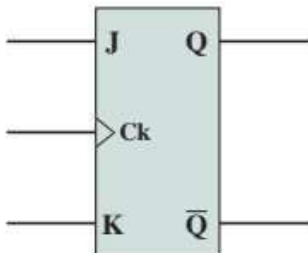
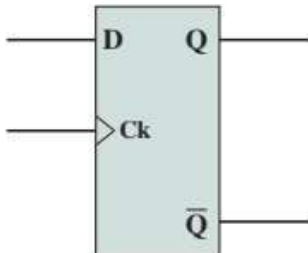


J-K Flip-Flop

- When $J=K=1$ it acts as a **toggle** function, the output is a **reverse** of the present state



S-R vs. J-K vs. D Flip-Flops

Name	Graphical Symbol	Truth Table															
S-R		<table><tr><th>S</th><th>R</th><th>Q_{n+1}</th></tr><tr><td>0</td><td>0</td><td>Q_n</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>-</td></tr></table>	S	R	Q_{n+1}	0	0	Q_n	0	1	0	1	0	1	1	1	-
S	R	Q_{n+1}															
0	0	Q_n															
0	1	0															
1	0	1															
1	1	-															
J-K		<table><tr><th>J</th><th>K</th><th>Q_{n+1}</th></tr><tr><td>0</td><td>0</td><td>Q_n</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>$\overline{Q_n}$</td></tr></table>	J	K	Q_{n+1}	0	0	Q_n	0	1	0	1	0	1	1	1	$\overline{Q_n}$
J	K	Q_{n+1}															
0	0	Q_n															
0	1	0															
1	0	1															
1	1	$\overline{Q_n}$															
D		<table><tr><th>D</th><th>Q_{n+1}</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	D	Q_{n+1}	0	0	1	1									
D	Q_{n+1}																
0	0																
1	1																

S = to set

R = to reset

S = R = 1, not ALLOWED

J = to set

K = to reset

S = R = 1, it will toggle the Q value

Input value will be channelled to output

Outline

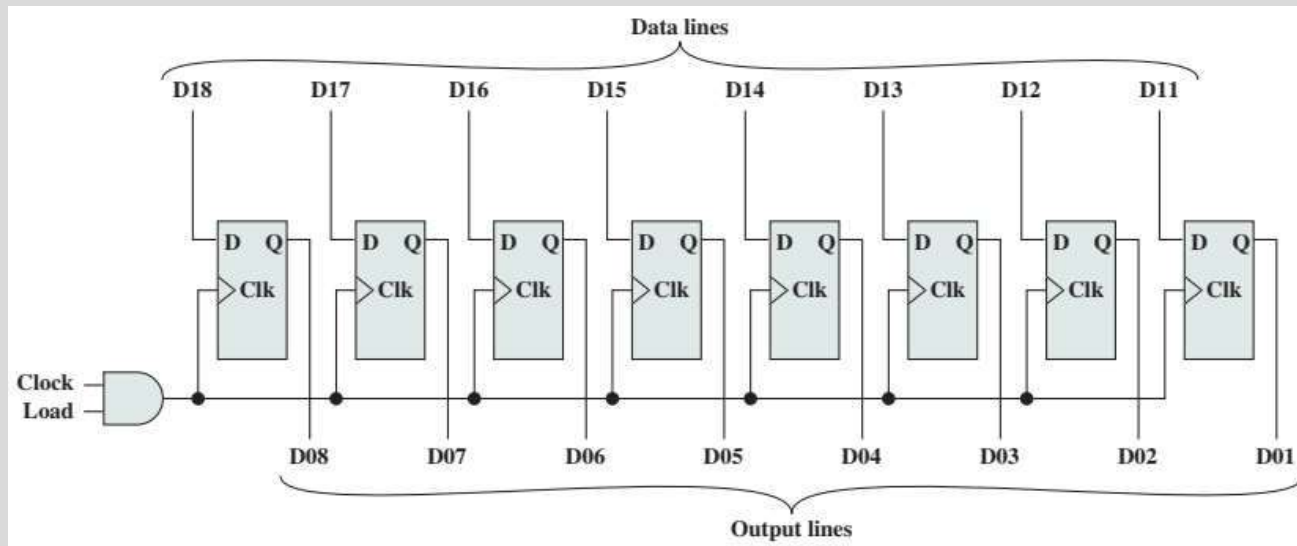
- Sequential Circuits
 - Flip-Flops
 - Registers
 - Counters
- Programmable Logic Devices
 - Programmable Logic Array
 - Field-Programmable Gate Array

Registers

- Digital circuit within CPU to **store** one or more bits of data
- Two types:
 - **Parallel** registers
 - **Shift** registers

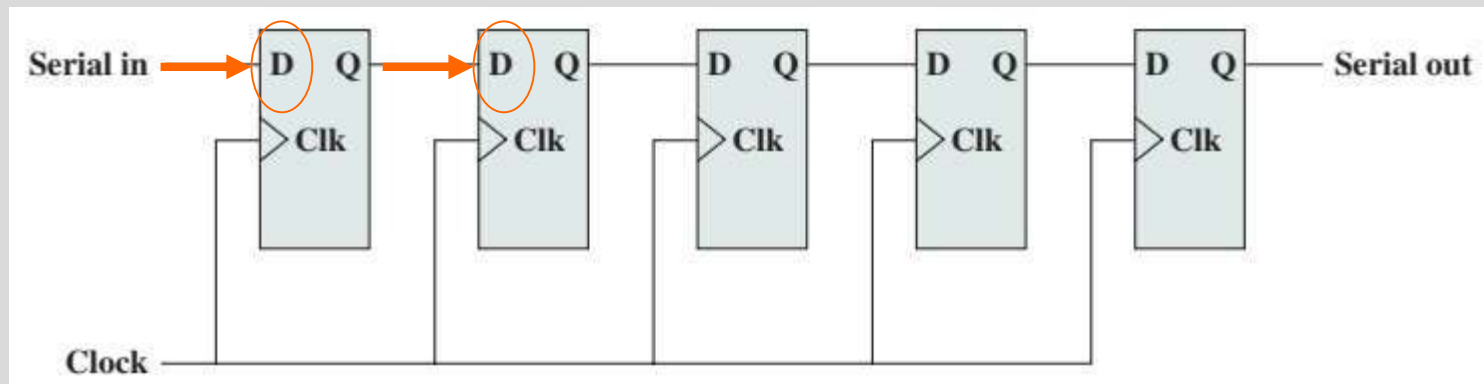
Parallel Registers

- Supports **simultaneous** reading or writing to a set of 1-bit memories
- Uses **D** flip-flops



Shift Registers

- Accepts/**transfers** information **serially**
 - Use (clocked) D flip-flops (or even S-R)
 - Input only to left-most flip-flop
 - Data are shifted one position at every clock pulse
 - Use to interface to serial I/O devices, perform logical shift/rotate functions (ALU), etc.



Counters

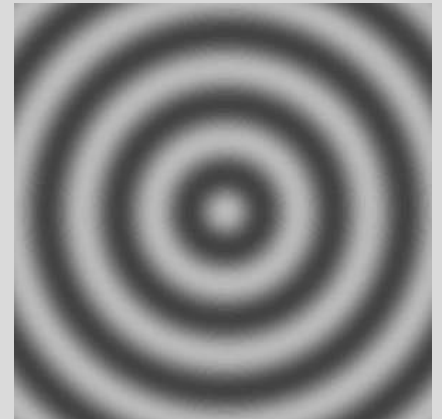
- Register which increment its value by modulo of its capacity
- n flip-flop has the capacity up to $2^n - 1$
 - Beyond maximum, is set to 0
- E.g. CPU's program counter

4 flip flops = $2^4 - 1 = 15$ counters

Ripple Counter

- **Slow**

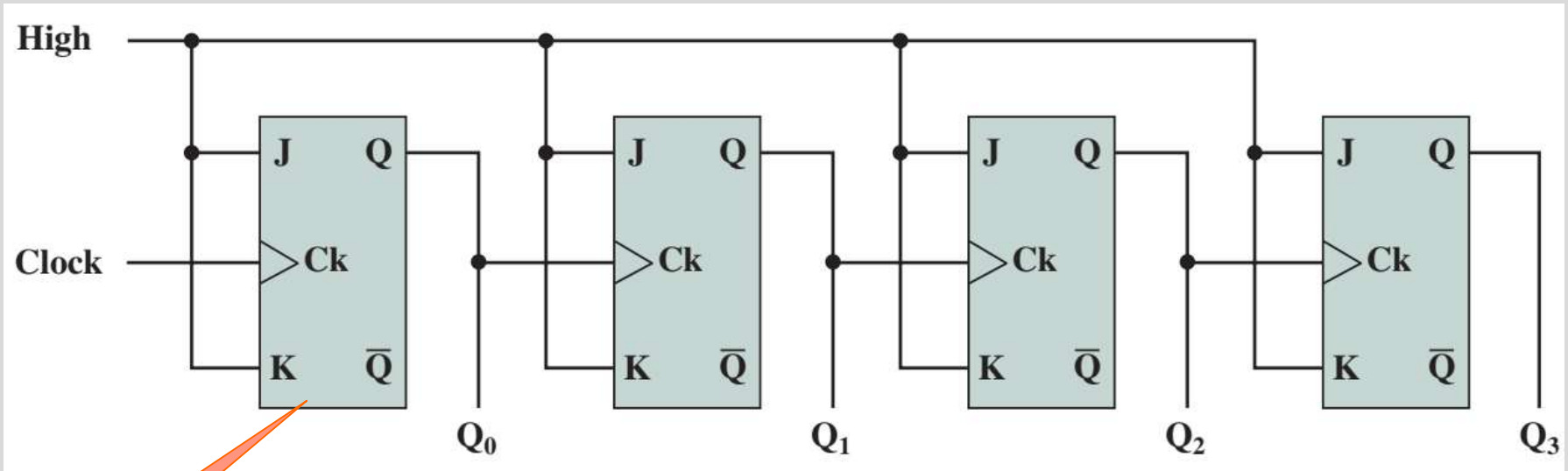
- Output of one flip-flop **triggers a change in the next** flip-flop (ripple effect)
- The counter is incremented at each clock pulse (0000, 0001, 0010, 0011, ... 1111, 0000)



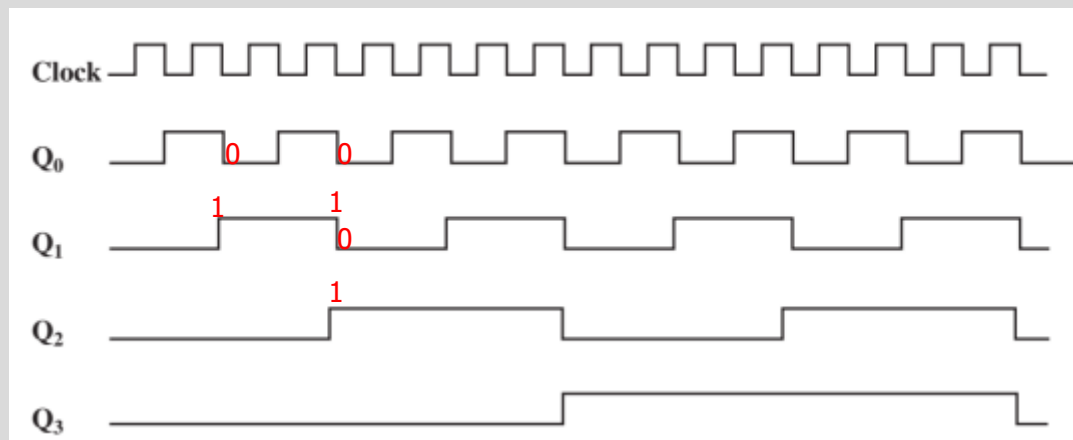
Ripple Counter

- **J and K inputs** to each flip-flop are **held at 1**
 - When there is clock pulse, output Q is **inverted** (0 to 1, 1 to 0)
- Q_0 is least significant bit

Ripple Counter

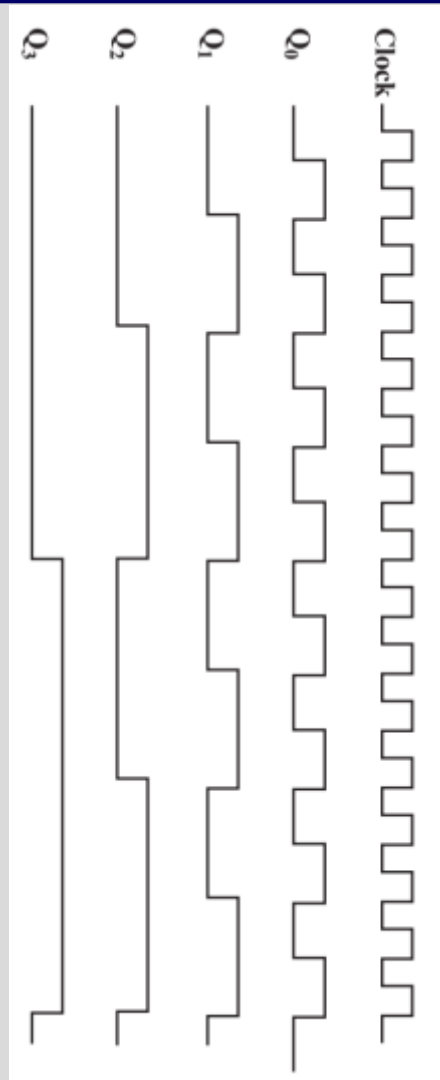


Will be
toggling
most often



When Q₀
becomes 0,
then only Q₁
becomes 1

Ripple Counter



Q_3	Q_2	Q_1	Q_0
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1
1	0	1	0
1	0	1	1
1	1	0	0
1	1	0	1
1	1	1	0
1	1	1	1

Q_0 toggle the most often

Synchronous Counter

- **Faster**
 - All flip-flops **change state at the same time**
 - CPUs use synchronous counters
- E.g. 3-bit counter using J-K flip-flop
 - 3 flip-flops (A, B, and C – A is least significant bit)
 - Construct truth table

Synchronous Counter

C	B	A	Jc	Kc	Jb	Kb	Ja	Ka
0	0	0	0	d	0	d	1	d
0	0	1	0	d	1	d	d	1
0	1	0	0	d	d	0	1	d
0	1	1	1	d	d	1	d	1
1	0	0	d	0	0	d	1	d
1	0	1	d	0	1	d	d	1
1	1	0	d	0	d	0	1	d
1	1	1	d	1	d	1	d	1

J	K	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	$\overline{Q}_{n+1}$

Q_n	J	K	Q_{n+1}
0	0	d	0
0	1	d	1
1	d	1	0
1	d	0	1

Synchronous Counter

MSB

LSB

S R

C	B	A	Jc	Kc	Jb	Kb	Ja	Ka
0	0	0	0	d	0	d	1	d
0	0	1	0	d	1	d	d	1
0	1	0	0	d	d	0	1	d
0	1	1	1	d	d	1	d	1
1	0	0	d	0	0	d	1	d
1	0	1	d	0	1	d	d	1
1	1	0	d	0	d	0	1	d
1	1	1	d	1	d	1	d	1

Set & Toggle

Toggle & Reset

J	K	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	$\overline{Q_n}$

Q_n	J	K	Q_{n+1}
0	0	d	0
0	1	d	1
1	d	1	0
1	d	0	1

Synchronous Counter

$J_c = BA$

		BA			
		00	01	11	10
0	C			1	
1		d	d	d	d

$K_c = BA$

		BA			
		00	01	11	10
0	C	d	d	d	d
1				1	

$J_b = A$

		BA			
		00	01	11	10
0	C		1	d	d
1			1	d	d

$K_b = A$

		BA			
		00	01	11	10
0	C	d	d	1	
1		d	d	1	

$J_a = 1$

		BA			
		00	01	11	10
0	C	1	d	d	1
1		1	d	d	1

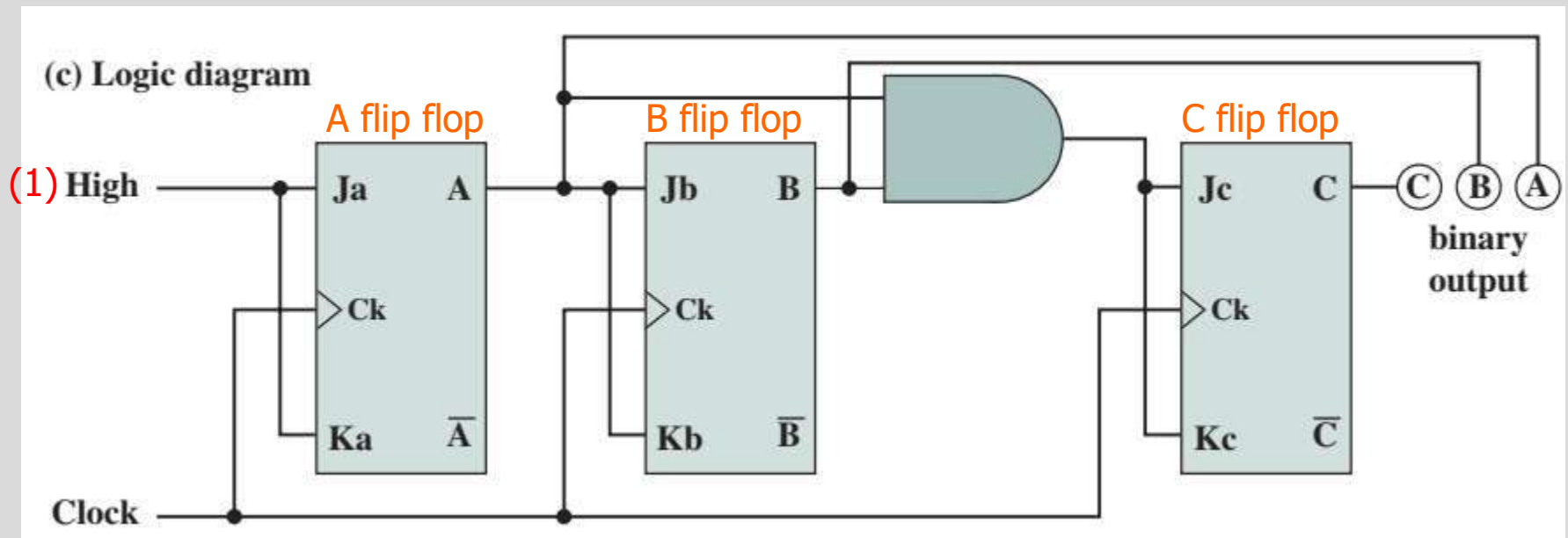
$K_a = 1$

		BA			
		00	01	11	10
0	C	d	1	1	d
1		d	1	1	d

C	B	A	Jc	Kc	Jb	Kb	Ja	Ka
0	0	0	0	d	0	d	1	d
0	0	1	0	d	1	d	d	1
0	1	0	0	d	d	0	1	d
0	1	1	1	d	d	1	d	1
1	0	0	d	0	0	d	1	d
1	0	1	d	0	1	d	d	1
1	1	0	d	0	d	0	1	d
1	1	1	d	1	d	1	d	1

Synchronous Counter

- $J_a = 1$
- $K_a = 1$
- $J_b = A$
- $K_b = A$
- $J_c = BA$
- $K_c = BA$



Outline

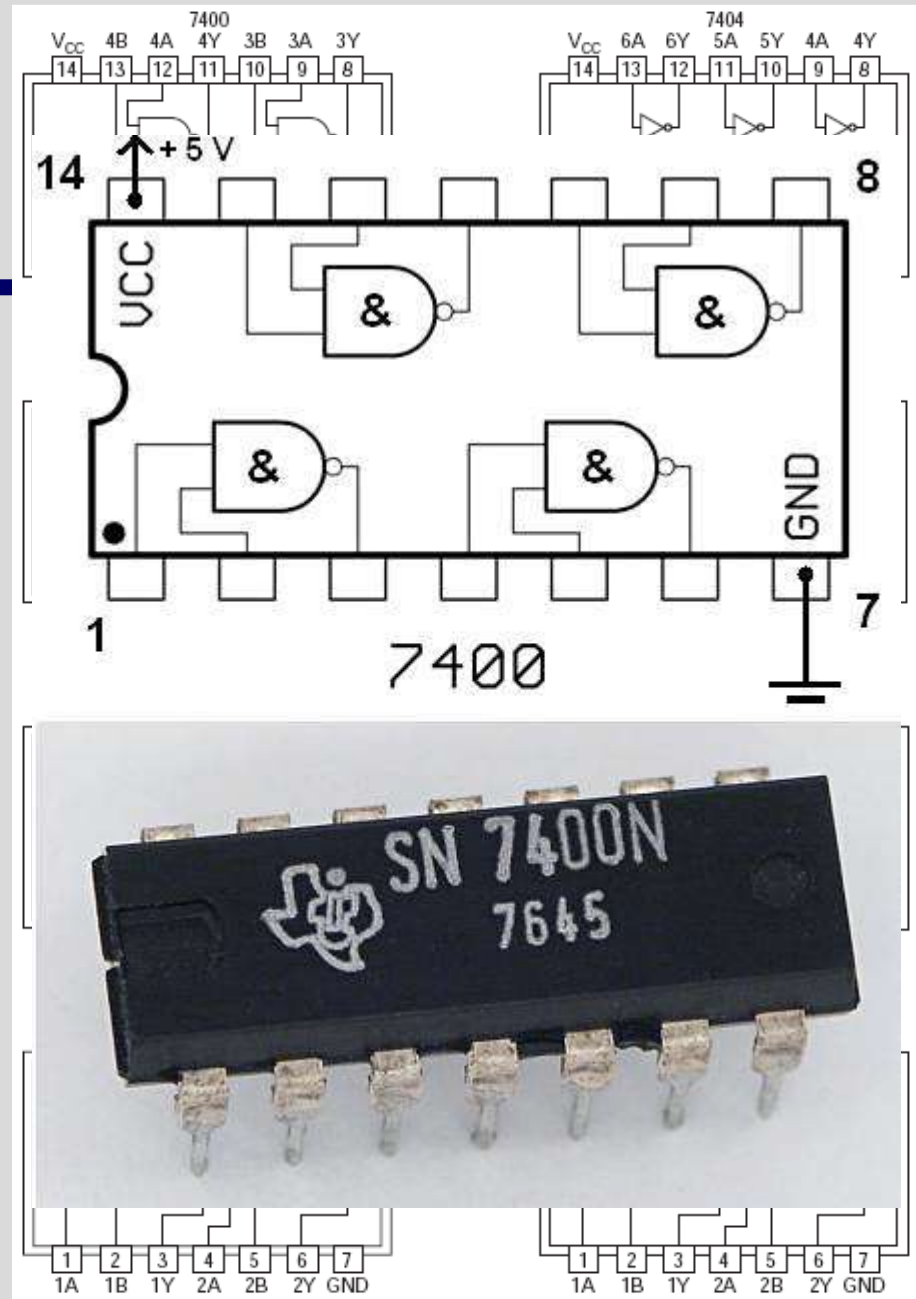
- Sequential Circuits
 - Flip-Flops
 - Registers
 - Counters
- Programmable Logic Devices
 - Programmable Logic Array
 - Field-Programmable Gate Array

Programmable Logic Devices

- So far, individual **gates are building blocks**
 - Any function can be realised

Programmable Logic Devices

- Early integrated circuits using SSI provided 1 to 10 gates on a chip
 - To construct a logic function, a **number of these chips are laid out** and the appropriate pin interconnections are made



Programmable Logic Devices

- Increasing levels of integration made it possible to put **more gates on a chip** and to make gate interconnections on the chip as well.
- Advantages:
 - Decreased cost and size
 - Increased speed

Programmable Logic Devices

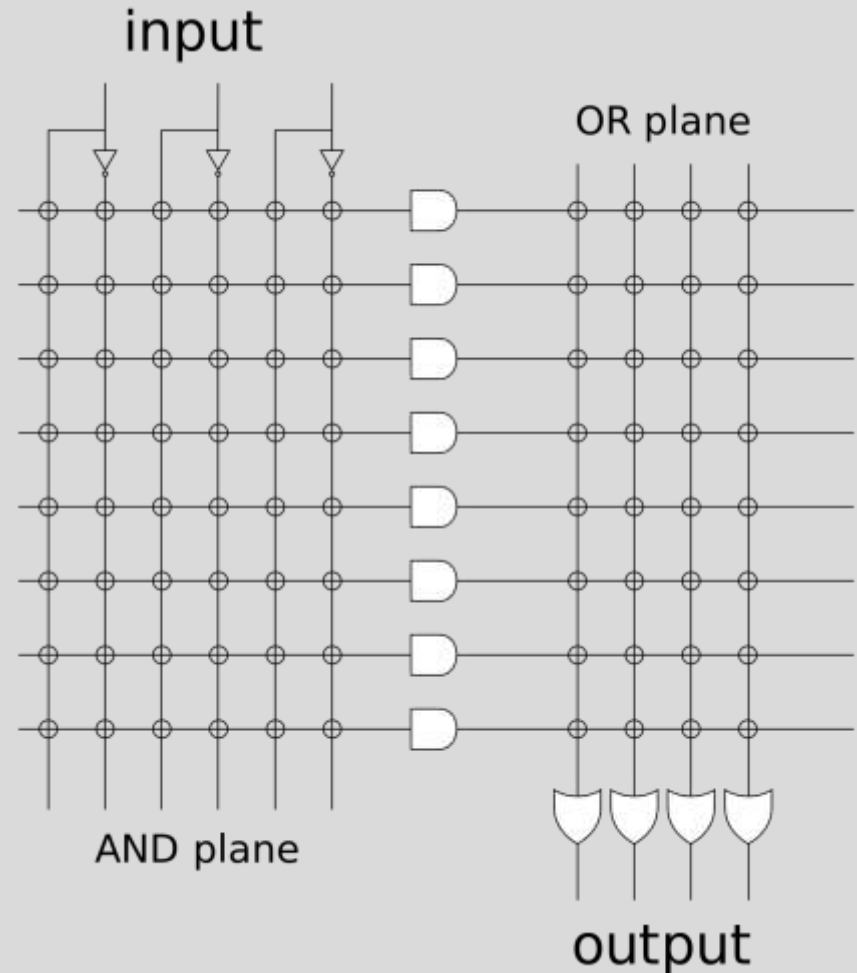
- However,
 - For each logic function, layout of gates and **interconnections must be designed**
 - **Cost and time** involved for **custom chips** design is high

Programmable Logic Devices

- Thus,
- it becomes attractive to develop a **general-purpose chip** that can be readily adapted to specific purposes
- This is the intent of the programmable logic device (PLD)

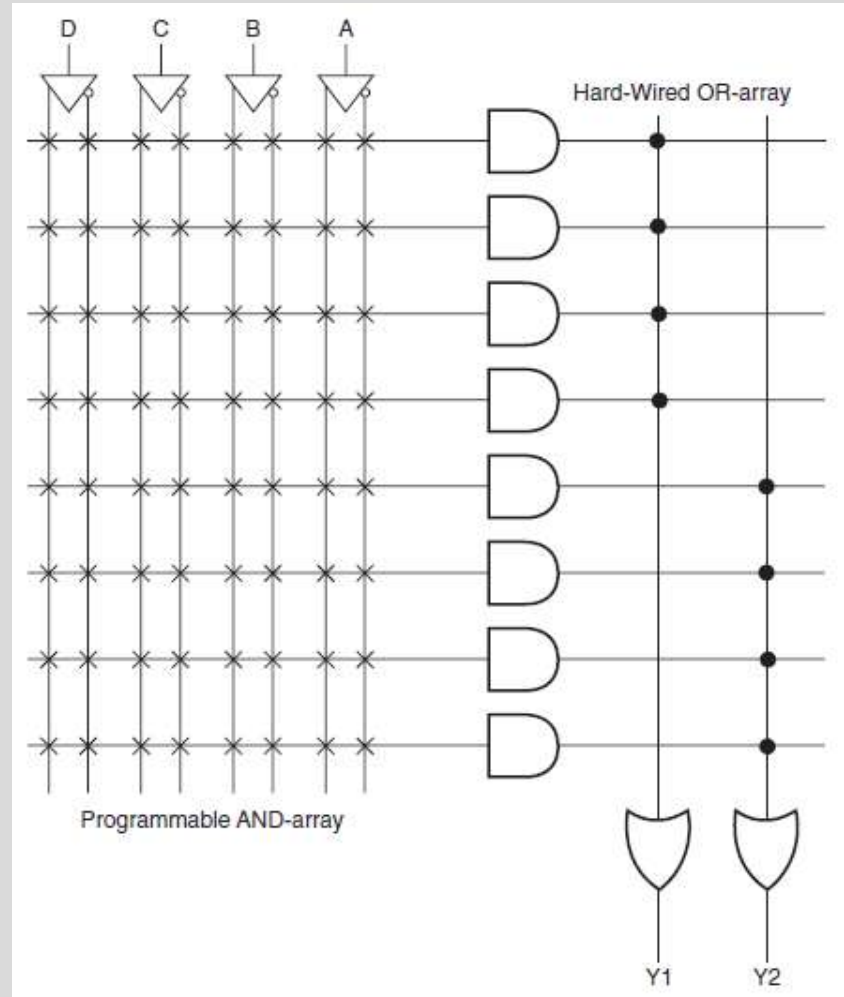
Programmable Logic Devices

- Programmable Logic Array (PLA)
 - A relatively small PLD that contains **two levels** of logic, an **AND-plane** and an **OR-plane**, where **both levels are programmable**



Programmable Logic Devices

- Programmable Array Logic (PAL)
 - A relatively small PLD that has a **programmable AND-plane** followed by a **fixed OR-plane**



Programmable Logic Devices

- **Simple** PLD (SPLD)
 - A PLA or PAL
- **Complex** PLD (CPLD)
 - A more complex PLD that consists of an arrangement of multiple SPLD-like blocks on a single chip

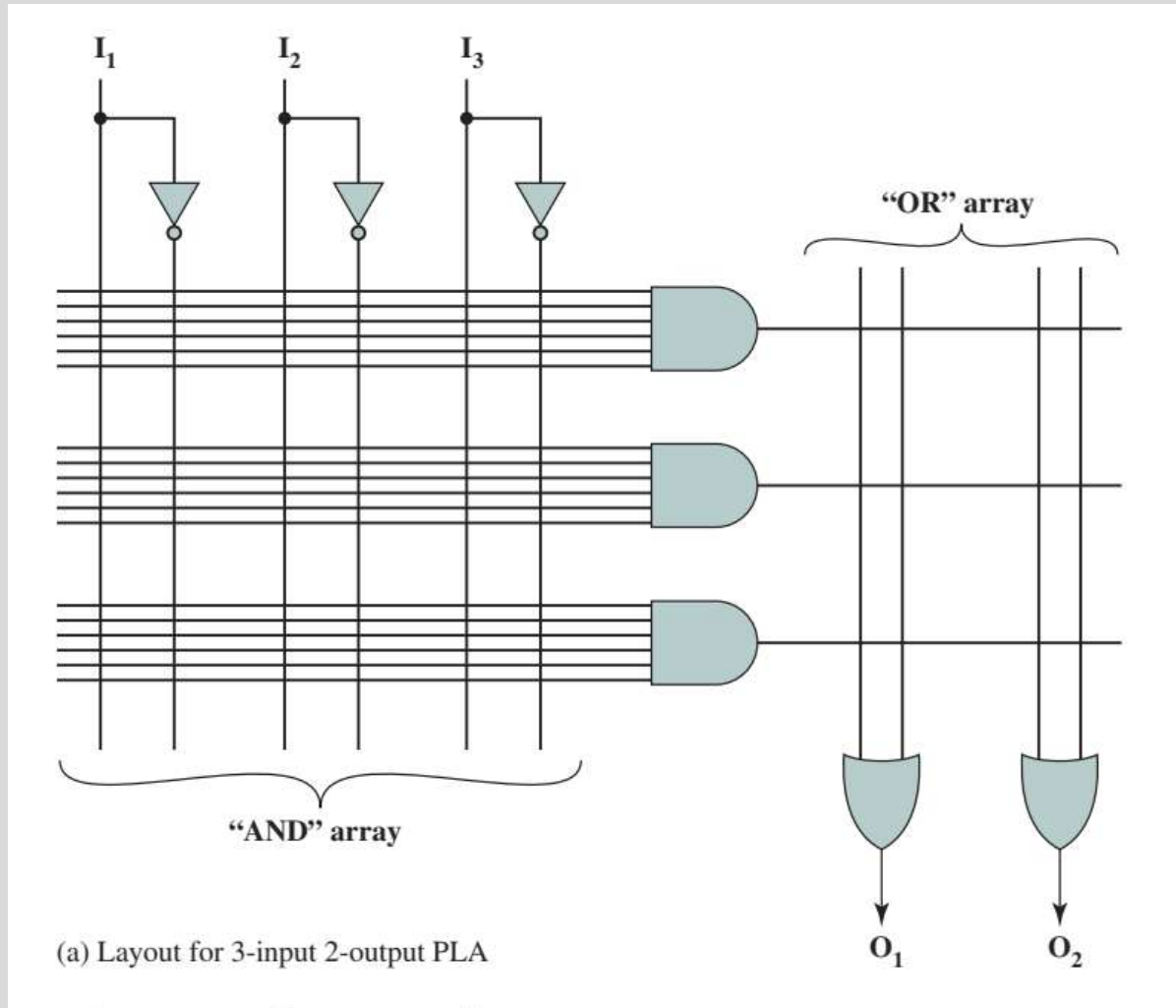
Programmable Logic Devices

- **Field-Programmable Gate Array (FPGA)**
 - A general structure that allows very high logic capacity
 - Whereas CPLDs feature logic resources with a wide number of inputs (AND planes), FPGAs offer more narrow logic resources
 - FPGAs also offer a higher ratio of flip-flops to logic resources than do CPLDs
- **Logic Block**
 - A relatively small circuit block that is replicated in an array in an FPD

Programmable Logic Array

- Based on fact that any Boolean function can be **expressed in SOP**
- Consists of NOT, AND, and OR gates
- Each chip input is NOTed
- Output of each AND gate is available to each OR gate
- Output of each OR gate is a chip output
- Appropriate connects are made to implement a particular SOP expression

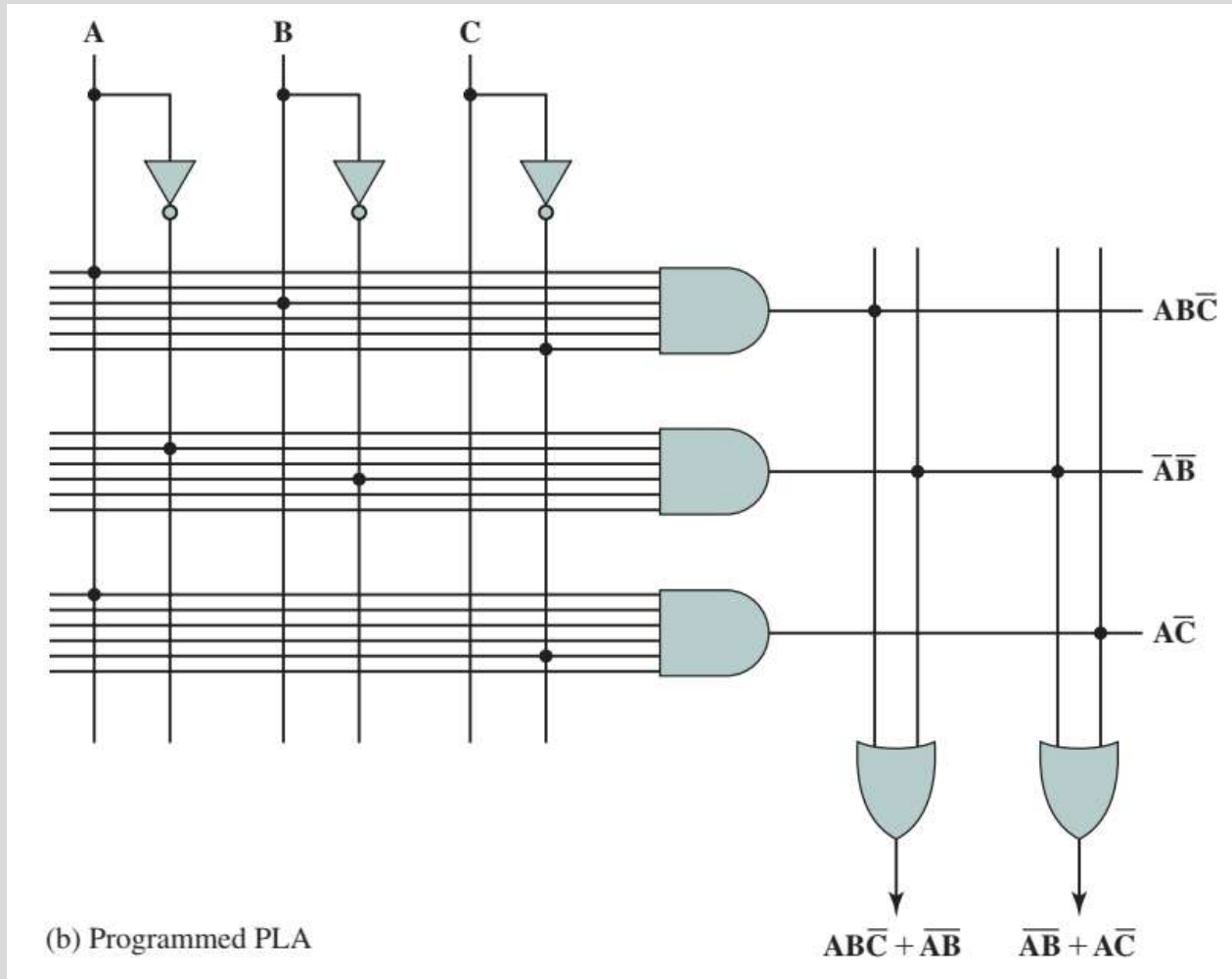
Programmable Logic Array



Programmable Logic Array

- PLAs manufactured in 2 ways
 - Every possible connection is made through a **fuse** at every intersection
 - Undesired connections are removed by blowing the fuses
 - Field Programmable Logic Array
 - Proper connections made during fabrication
- PLAs provide flexible, inexpensive way of implementing digital logic functions

Programmable Logic Array



Field-Programmable Gate Array

- PLA is an example of SPLD
- Problem is that the structure of programmable logic planes **grows too quickly in size** as number of inputs increased

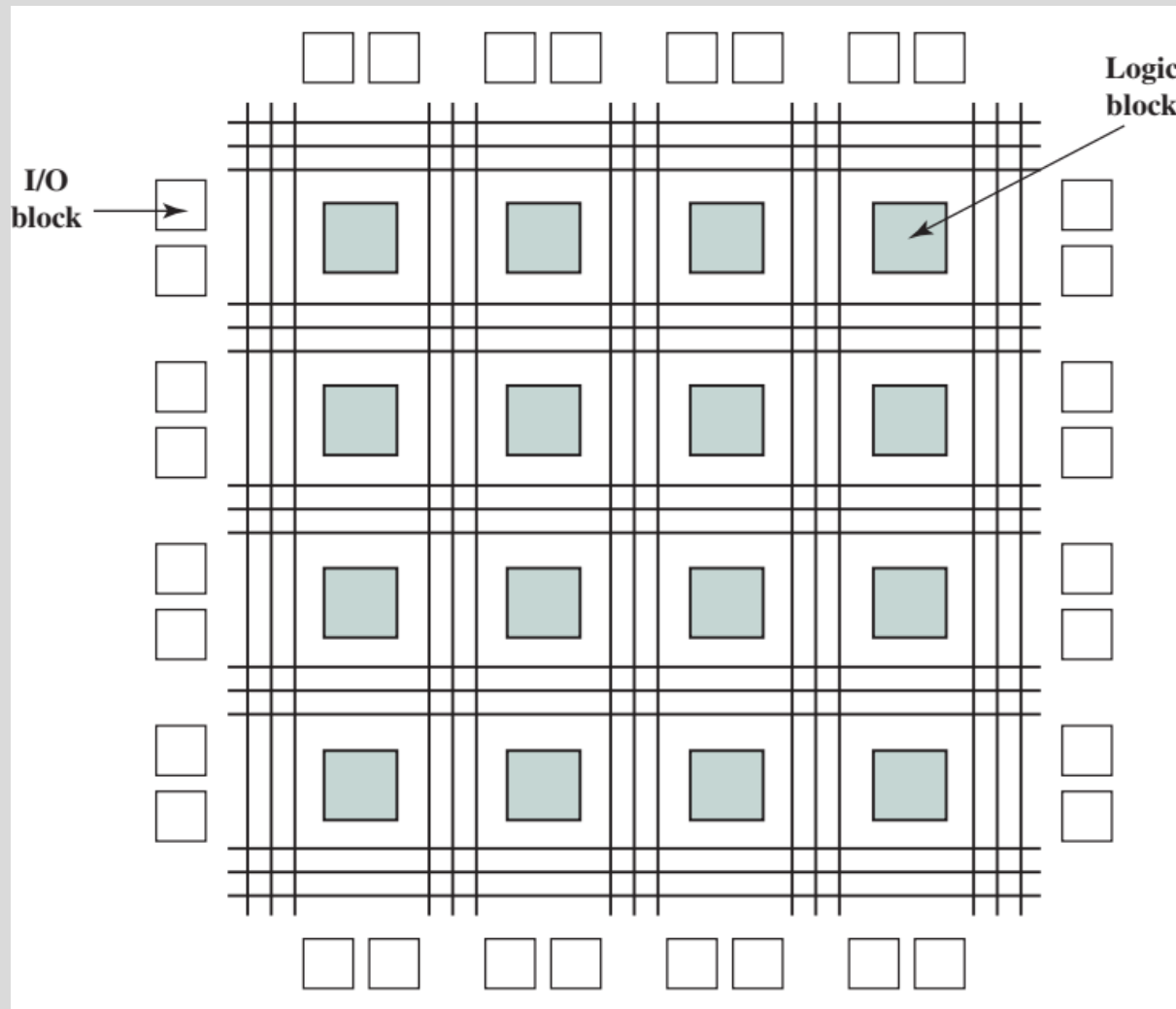
Field-Programmable Gate Array

- Solution is to integrate **multiple SPLD onto single chip**
 - Provide interconnect to programmably connect the SPLD blocks together
 - Complex PLDs (CPLDs)
 - Most important type is FPGA

Field-Programmable Gate Array

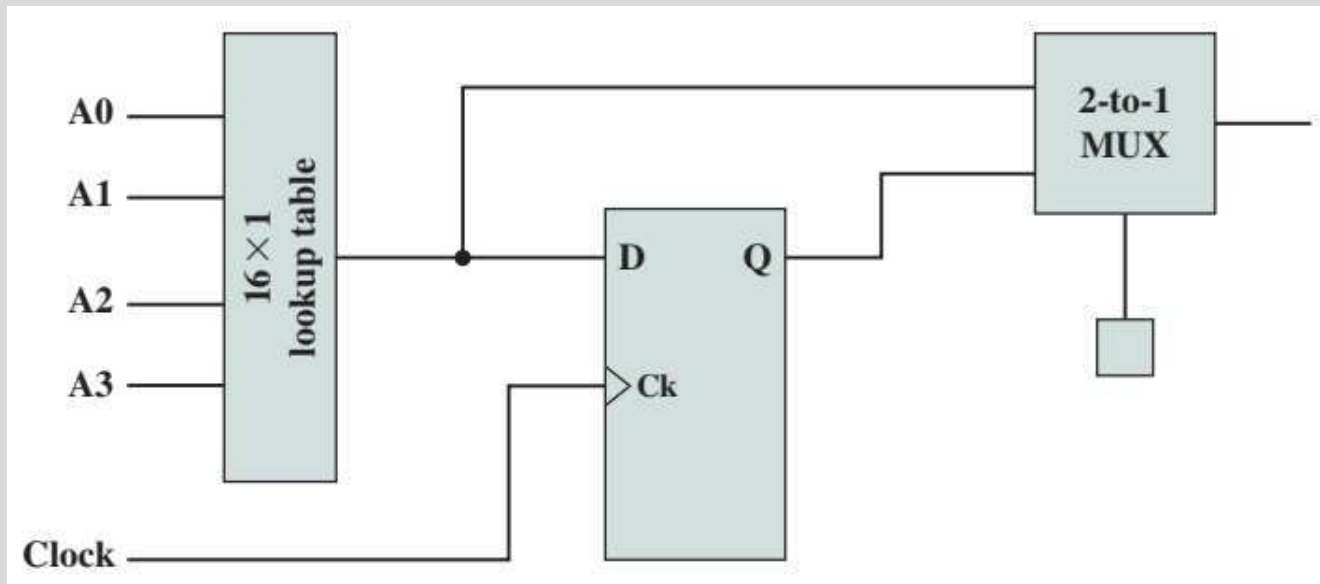
- FPGA consists of an **array of uncommitted circuit elements**
- Key components
 - Logic block – computation takes place
 - I/O block – connects I/O pins to circuitry on chip
 - Interconnect – establish connections among I/O blocks and logic blocks

Field-Programmable Gate Array



Field-Programmable Gate Array

- Logic block is either combinational or sequential circuit
 - Programming is done by downloading contents of truth table



Field-Programmable Gate Array



A person is sitting on a grassy bank next to a body of water. The person is in silhouette, wearing a dark shirt and shorts. A bicycle is parked on the grass to the right of the person. In the background, there are trees and a building under a cloudy sky.

Reflection...

From the lecture today,
I learnt...

Link on eLearn / Chat